

Onde fica a Nuvem?
(E por que você deveria se
importar).

Weder Sousa

Where is the Cloud? (And why you should care).

Weder Sousa

About Me

Weder Mariano de Sousa

Specialist Senior Java - GFT
Professor - Uni Evangélica



Technician **System Development**
Graduated **Computer Science**
Post Graduate in **Mídias UFG**



<https://www.linkedin.com/in/wedermarianodesousa/>
 <https://github.com/weder96>
 <https://twitter.com/weder96>
 <https://dev.to/weder96>



GOJava



AWS User
Group Goiânia



AWS User
Group Anápolis



Cloud

What, why, and how.



What is cloud computing?

Access to resources via the internet.



It's like renting computing power instead of owning it.

Why the cloud?

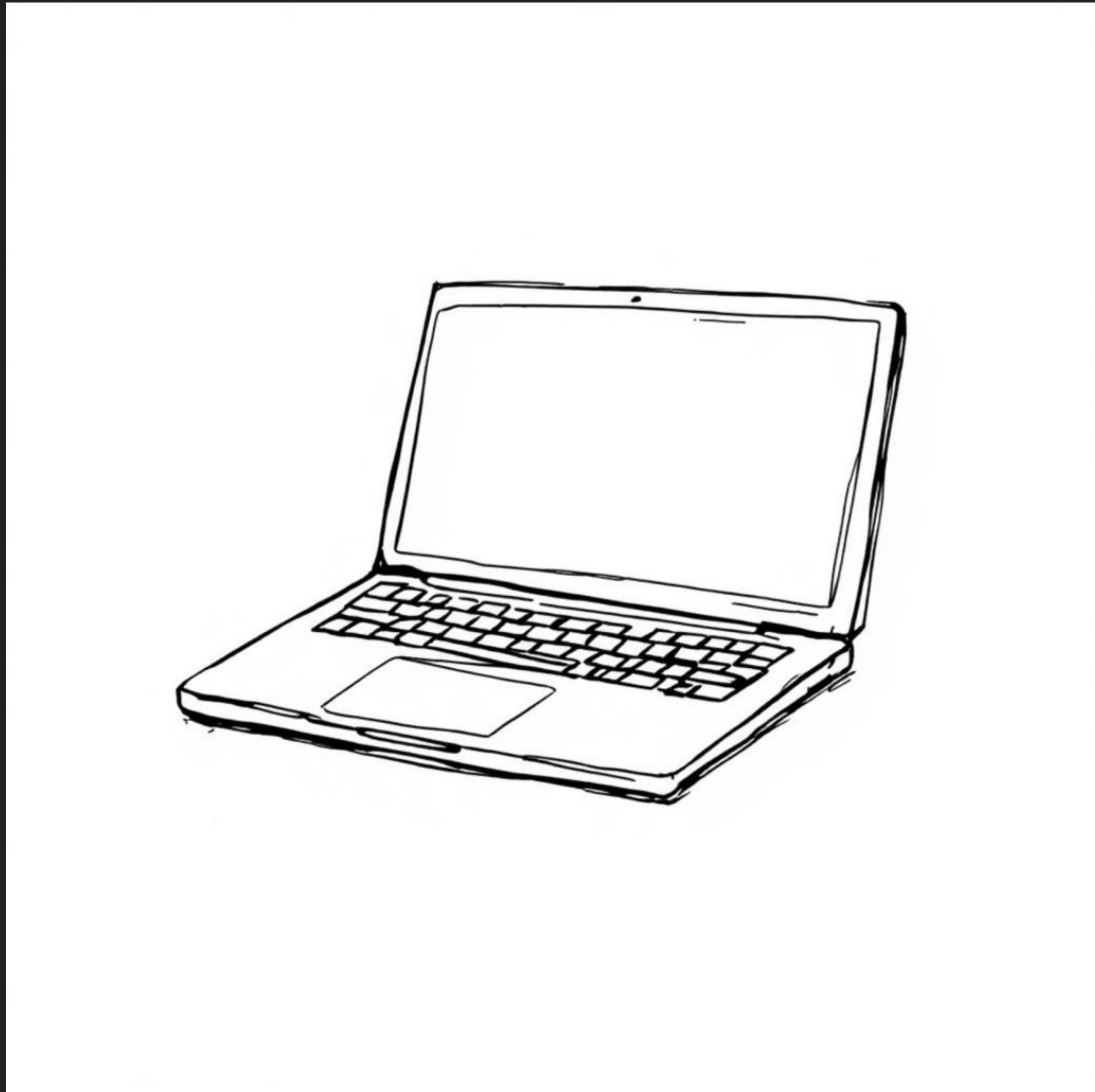
The digital world operates in the cloud, supporting your favorite applications and experiences.



Local vs. Cloud

Why should you care?

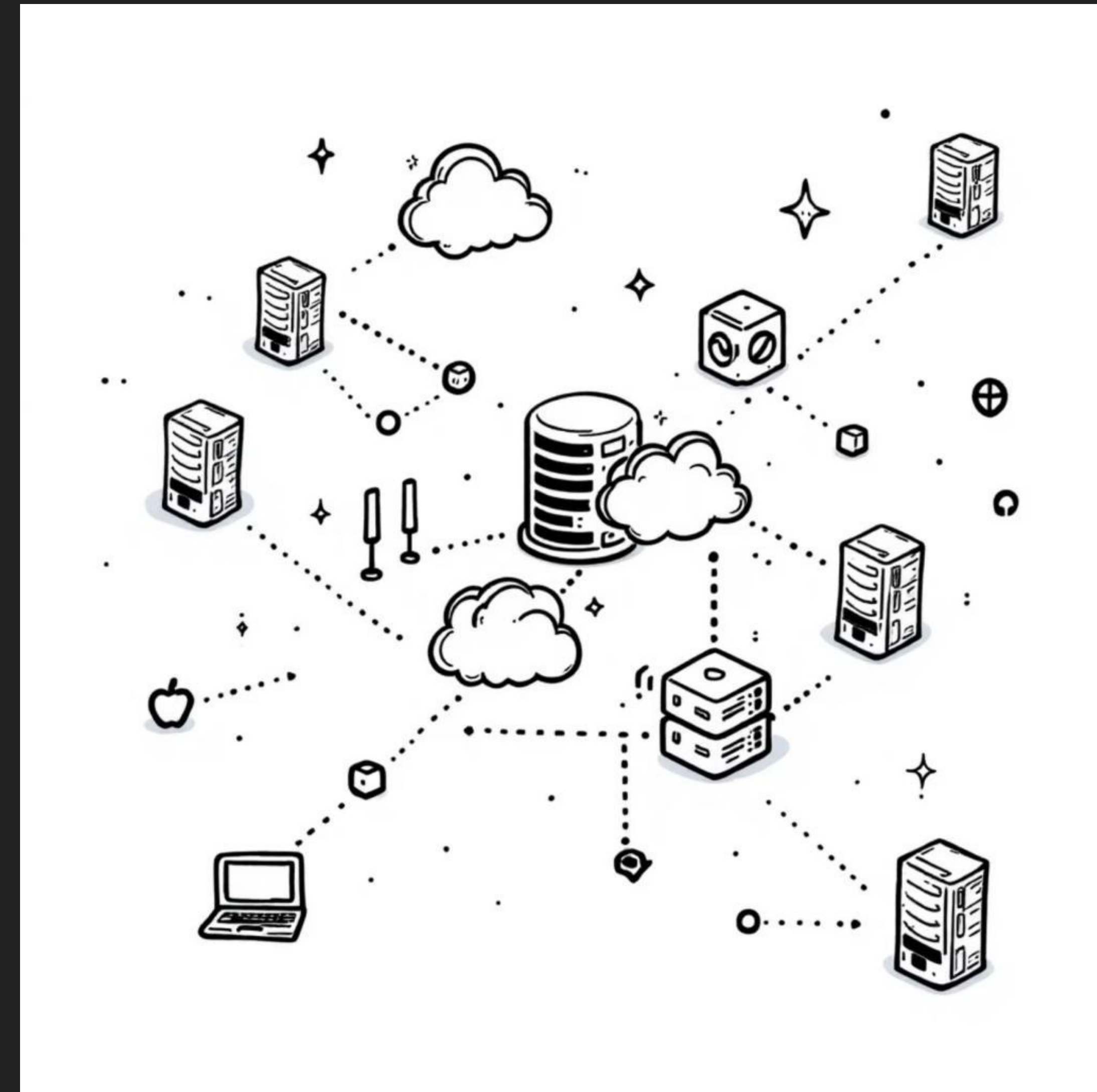
Your computer



Limited resources

Netflix

Cloud



Unlimited possibilities

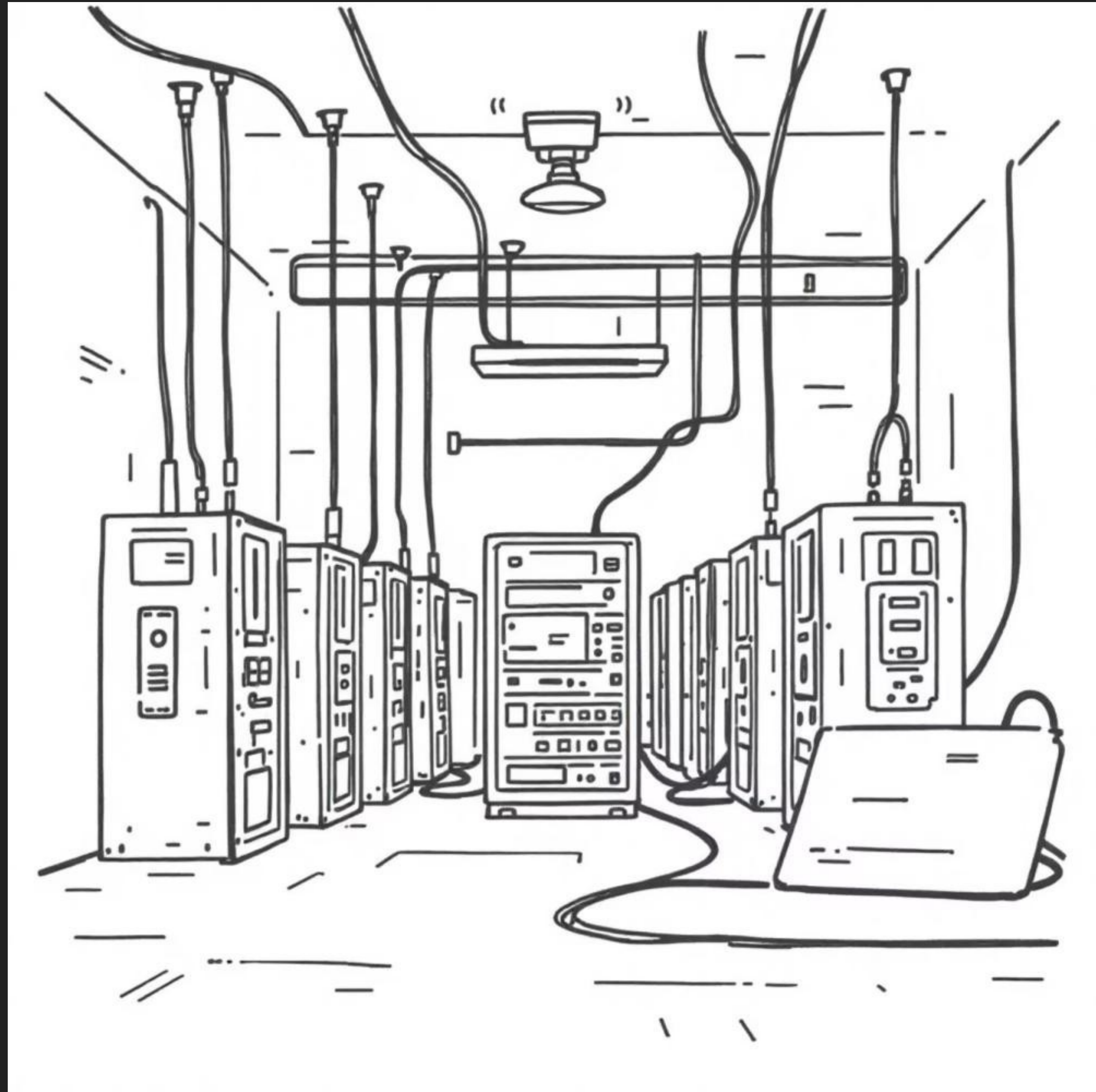
Spotify

Gaming Servers

Why Cloud Matters

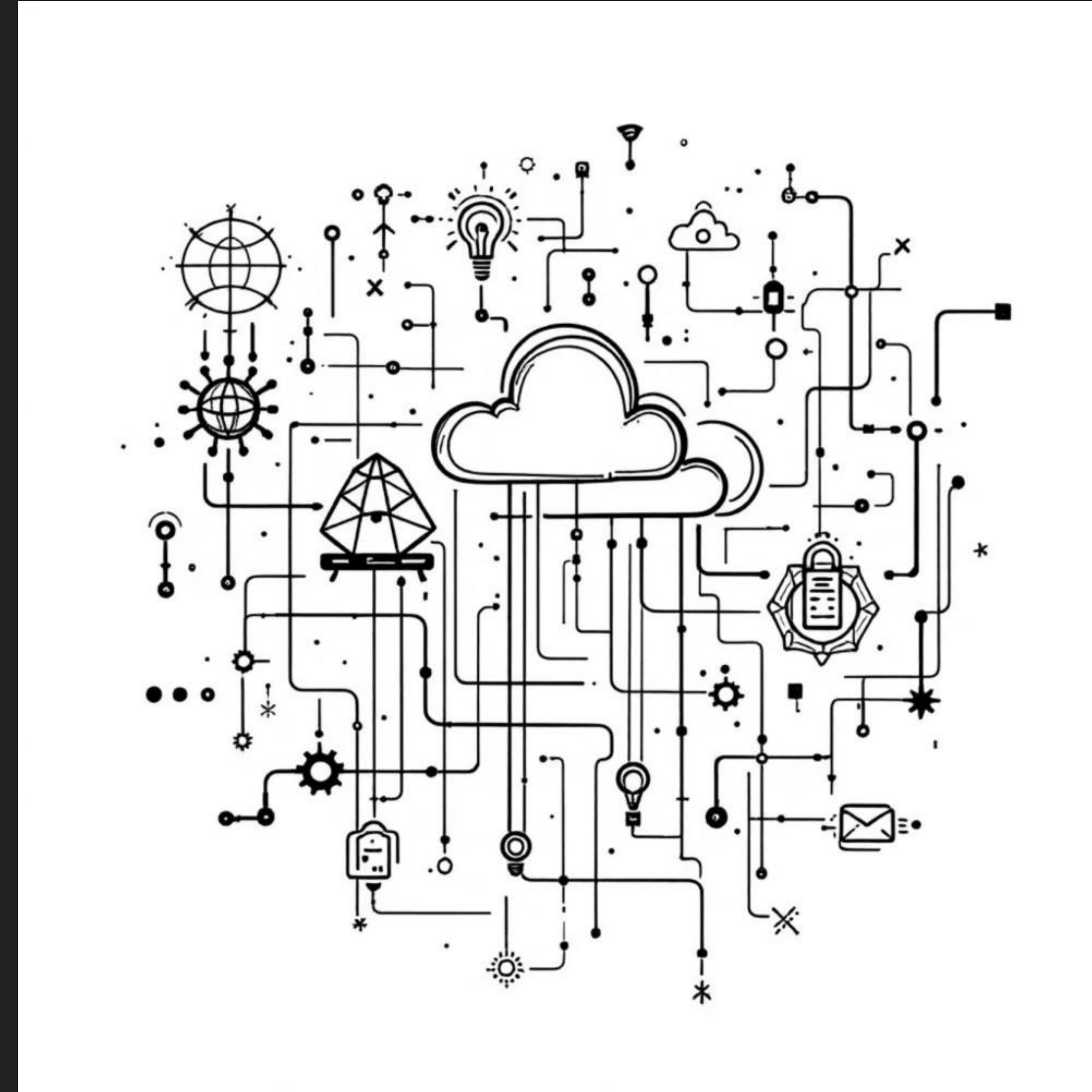
Revolutionizing how we build and scale.

Traditional Data Center



High upfront costs
Slow scaling Maintenance
burden

Cloud Computing



Pay-as-you-go
Instant scalability Managed
services



What is AWS?

Your cloud innovation toolbox.

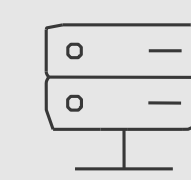
Amazon Web Services: The largest cloud provider.



Global
Reach



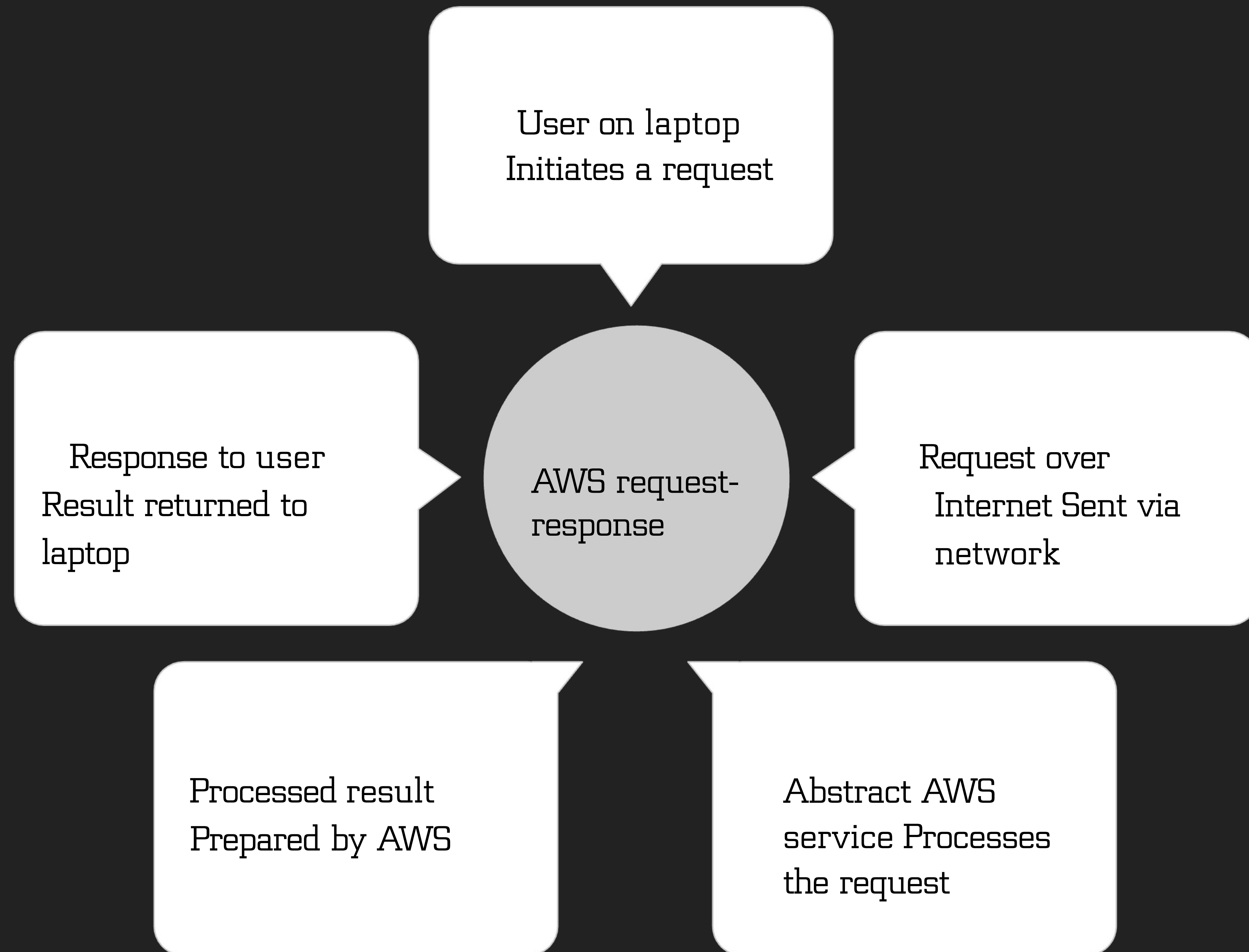
Modular
Services



Massive
Scale

How AWS Works

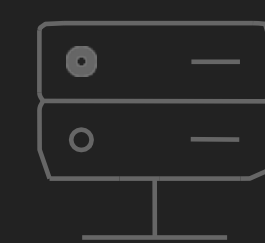
Simple interactions, powerful results.



You request, AWS delivers.

Core AWS Services

Building blocks for your ideas.



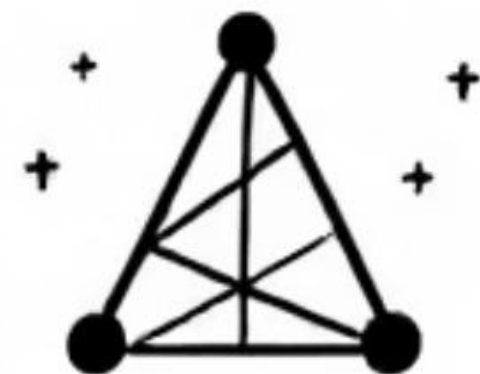
EC2: Compute
Virtual Servers



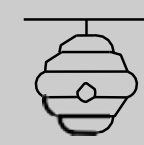
S3: Storage
Data Lake



RDS: Database
Managed Data



Today's Mission



Build a Real Backend API

Create a functional API without server headaches.



Add AI Intelligence

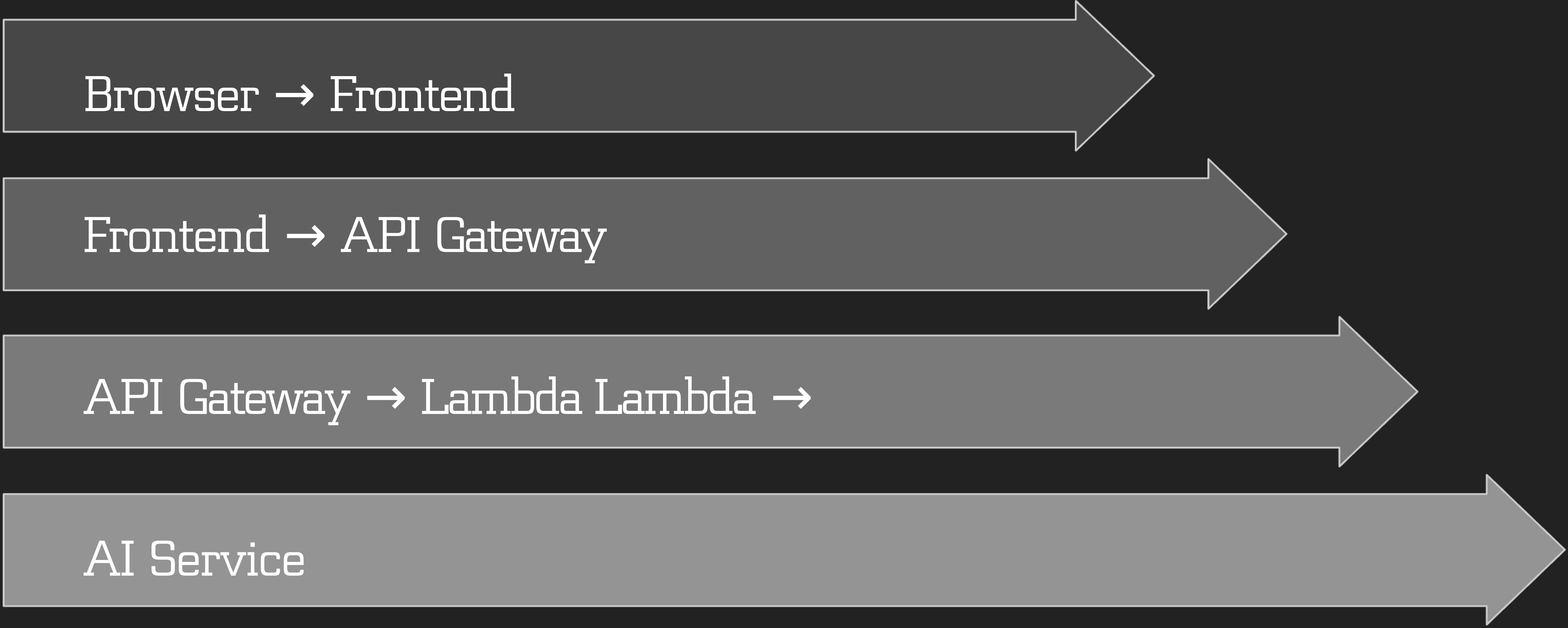
Integrate smart features effortlessly.



Connect Frontend to Cloud

Link your UI to your powerful backend.

Cloud Architecture: The Big Picture



Browser → Frontend

Frontend → API Gateway

API Gateway → Lambda Lambda →

AI Service

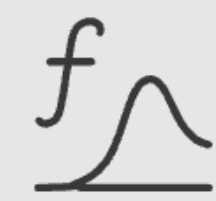
Understand how different AWS services work together.

Lab 1: Serverless API (No Servers!)



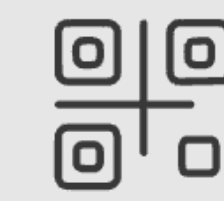
Your Code

Write functions, not manage servers.



AWS Lambda

Runs only when called, instant backend.



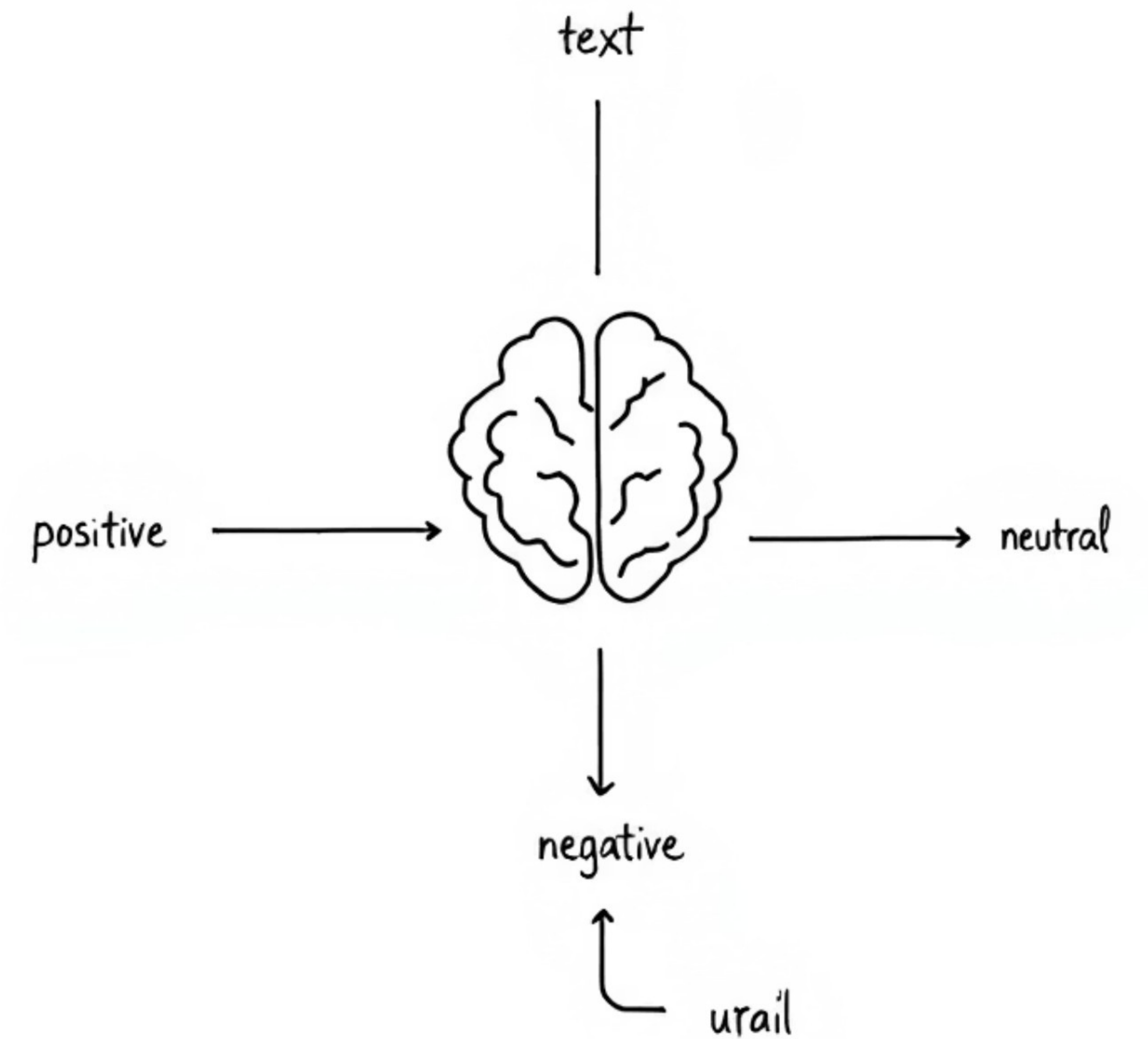
API Endpoint

Your function, now a powerful API.

Deploy your backend logic instantly, hassle-free.

Lab 2: Add AI to the API

Easily integrate advanced AI without machine learning expertise.



Frontend + Backend Together

/quote

/sentiment

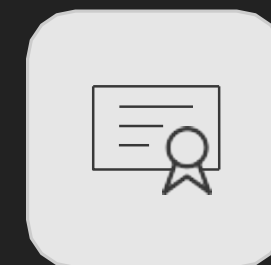
Visually connecting user actions to cloud functions.

What You Just Built



Real-World Architecture

Modern, scalable solutions.



Resume-Worthy Skills

Hands-on cloud expertise.



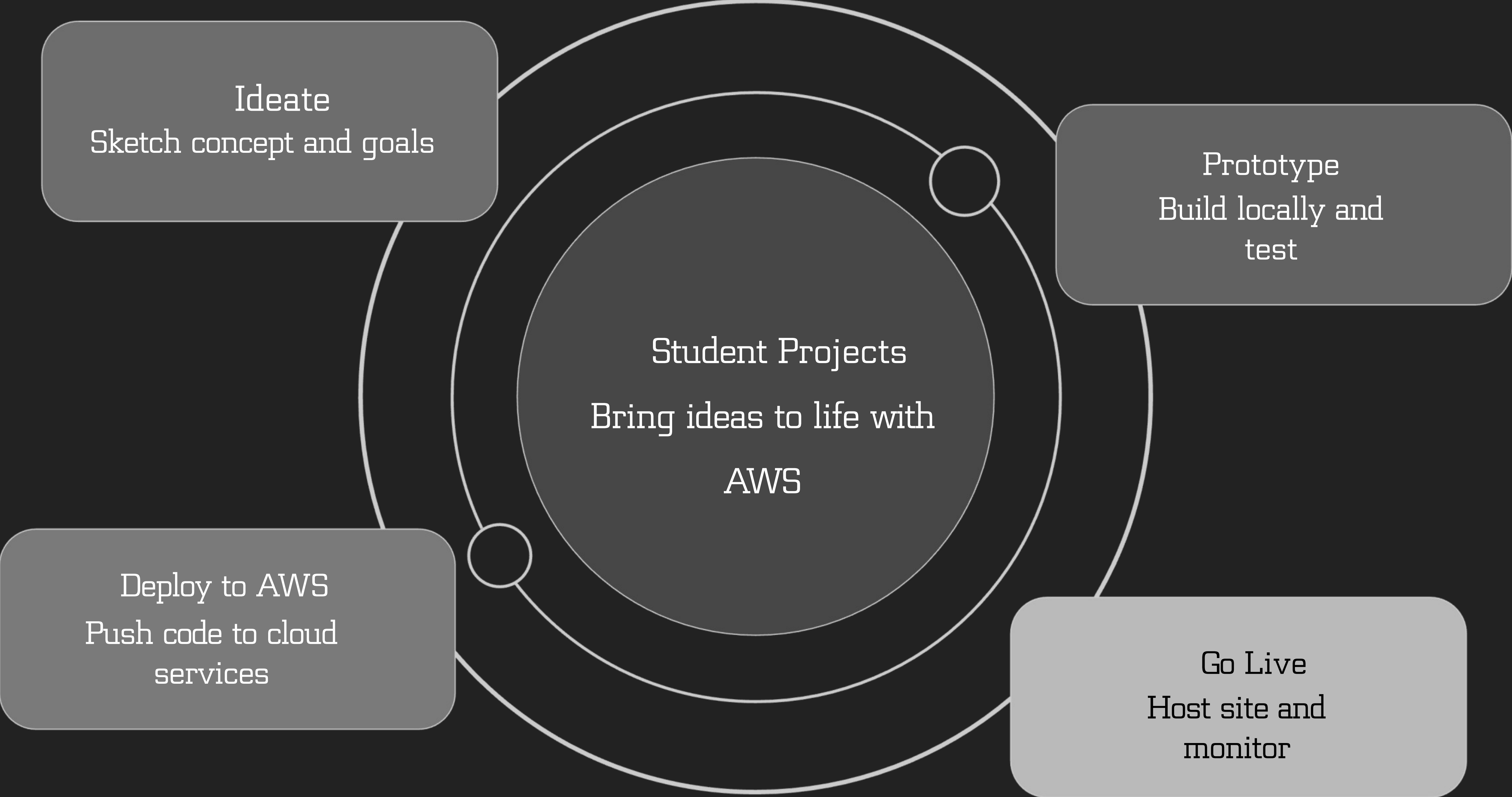
Modern Cloud Stack

Ready for future projects.

From concept to deployment: you built a functional cloud application!

Students & AWS

Bring your ideas to life!



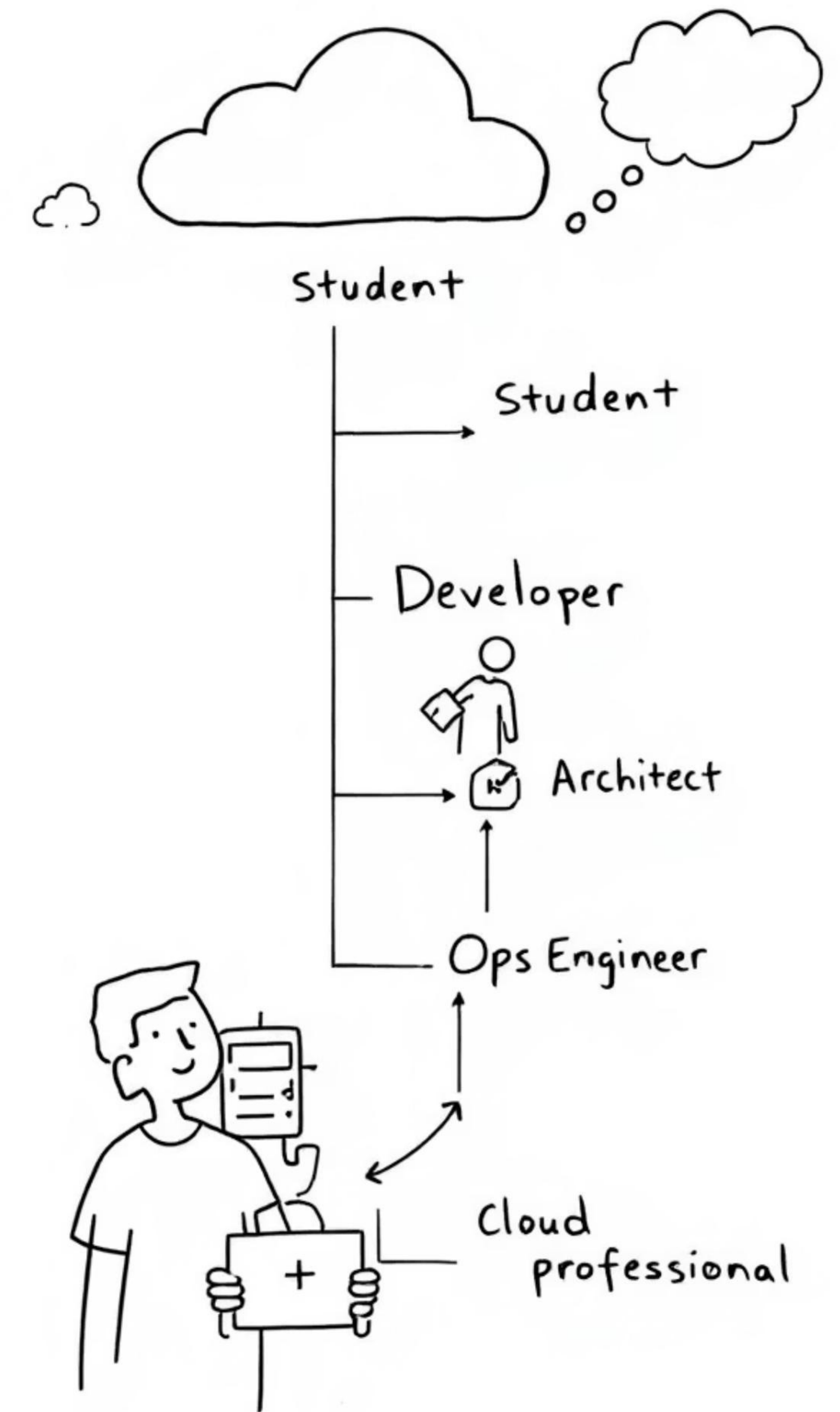
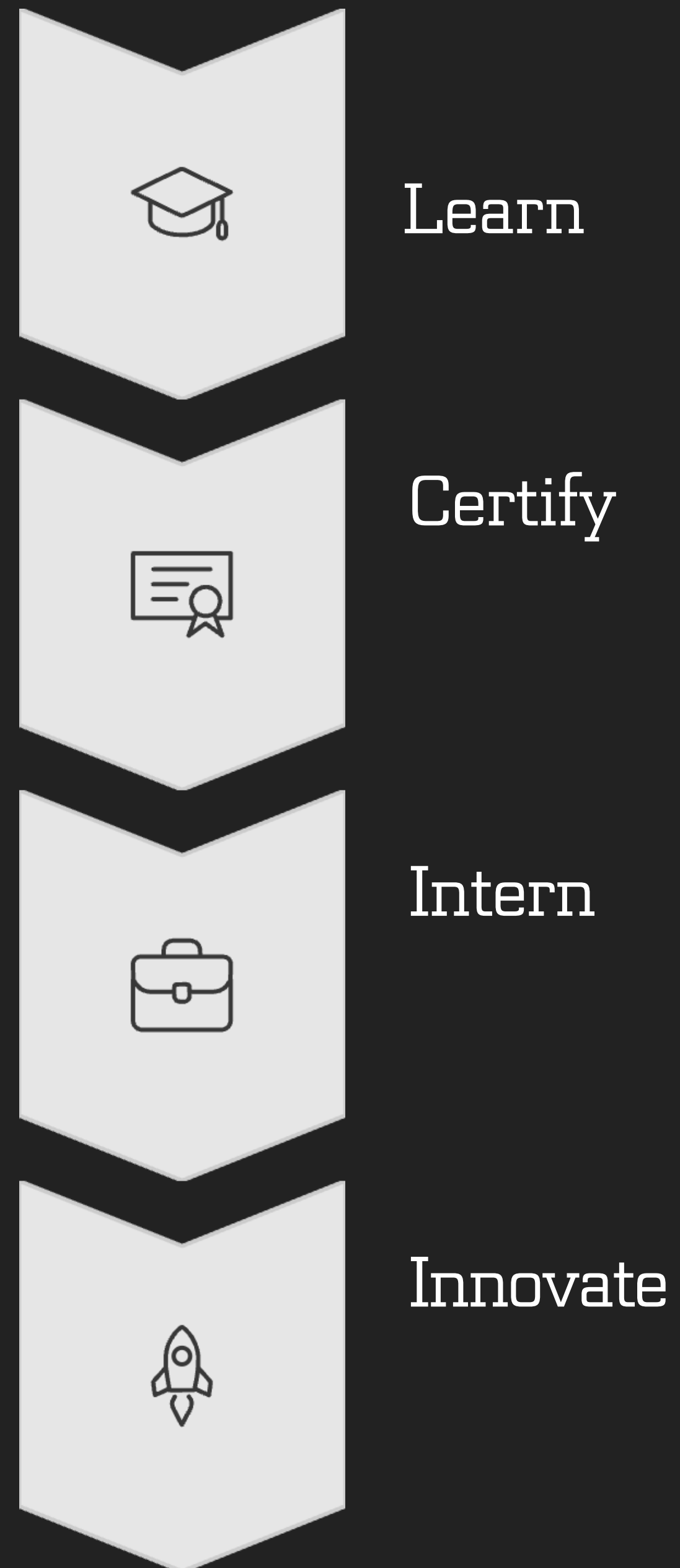
Host a Website

Deploy Projects

Build APIs

Cloud Careers

Your path to innovation.



Where is the Cloud ?



AWS Global Infrastructure

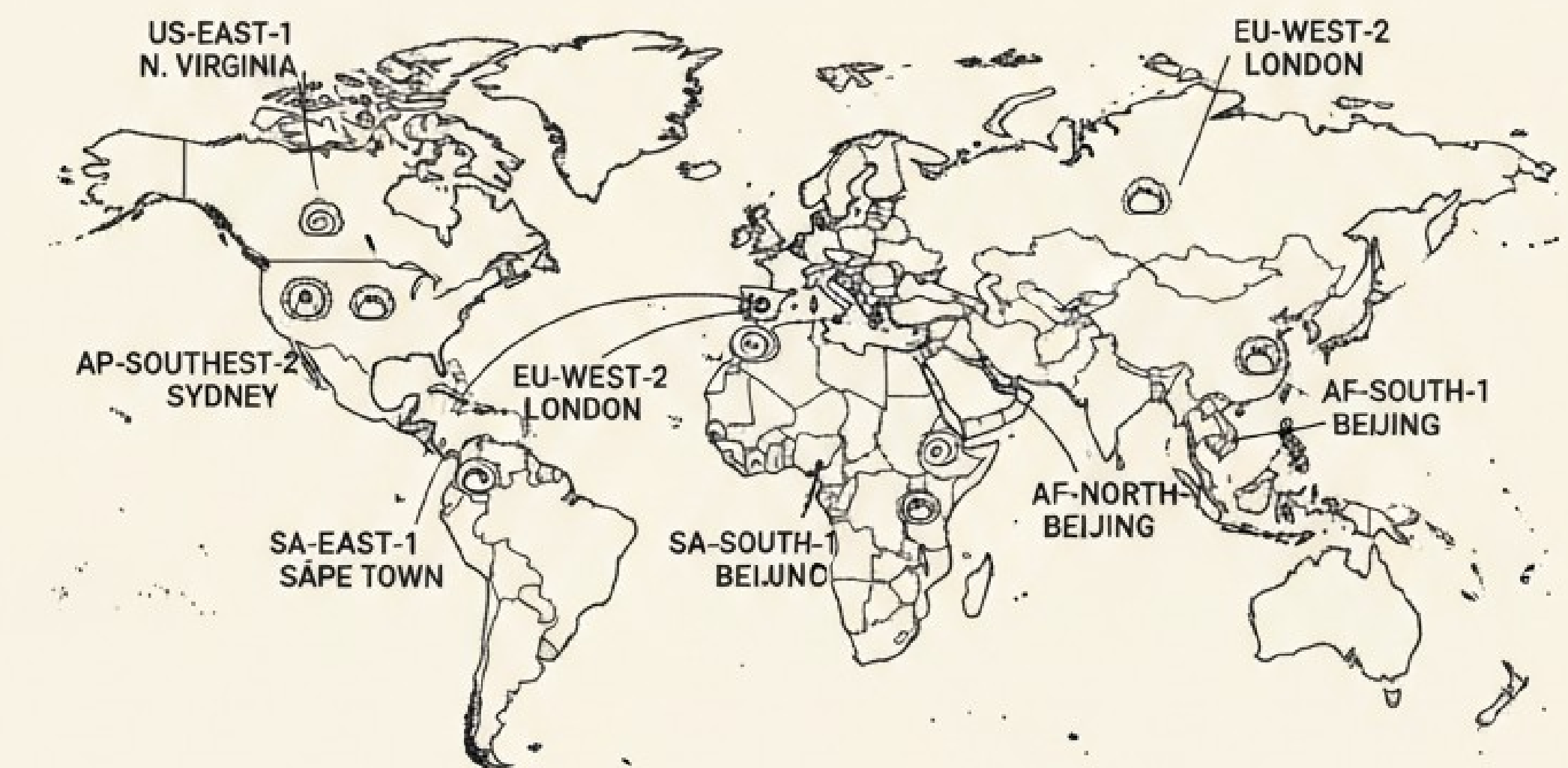


The AWS Cloud spans 123 Availability Zones within 39 Geographic Regions, with announced plans for 7 more Availability Zones and 2 more AWS Regions in the Kingdom of Saudi Arabia, and Chile.

The Backbone of Microsoft Cloud: Our Datacenters



80+ Azure regions: Cloud regions distributed worldwide.
500+ Datacenters: Physical facilities operating across the globe.
34 Countries: Nations hosting Microsoft datacenters.
800k+ Kilometers: Total length of network fiber optic cables.
190+ Network PoPs: Points of Presence to ensure low latency.
40+ Gigawatts: New renewable energy contracted, spanning across 26 countries.



Inside a data center



The Backbone of Microsoft Cloud: Server Room



The server room is the heart of the Microsoft Cloud's massive computing and storage capacity.

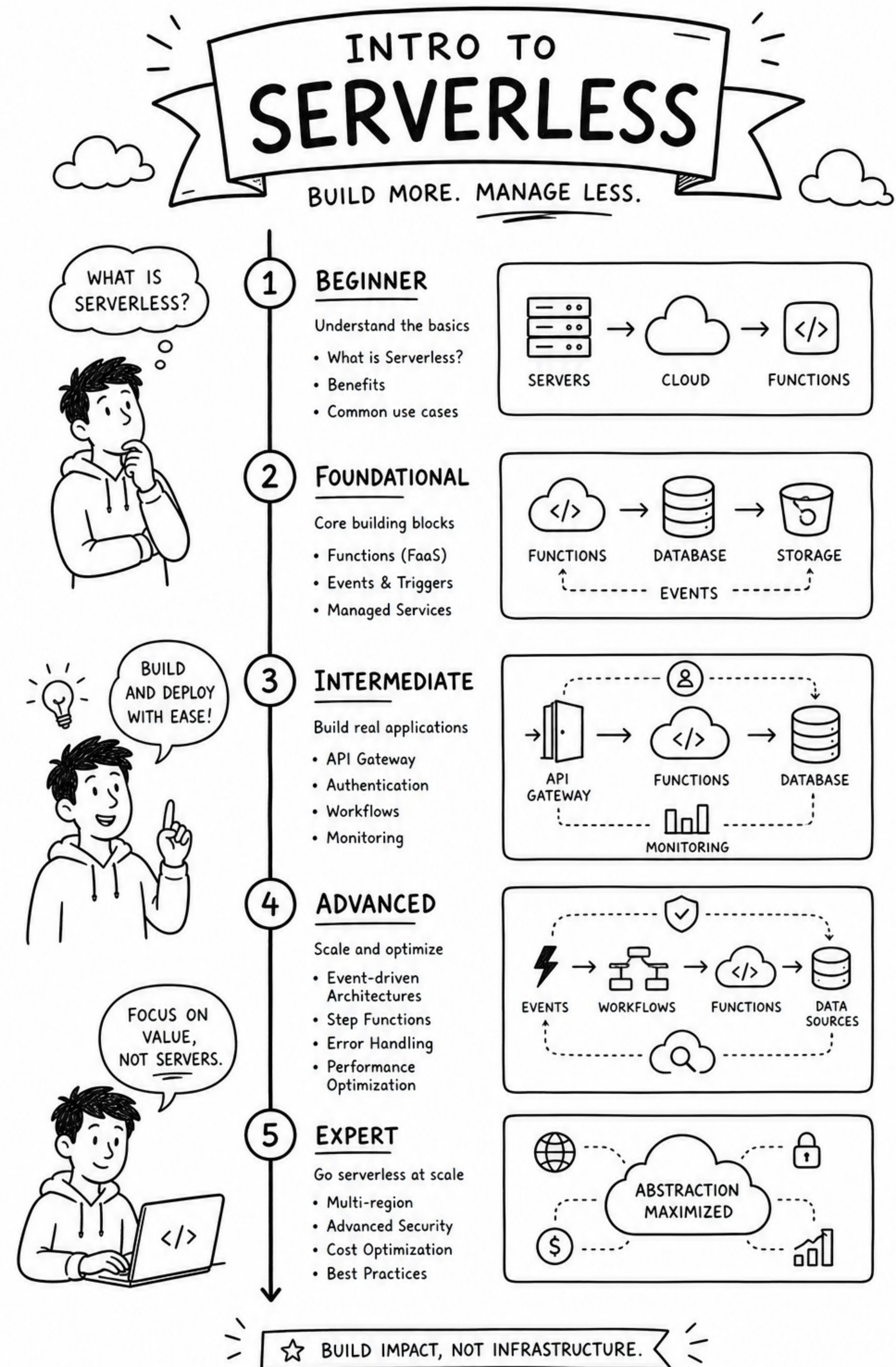
Innovative server blades, designed and built under Project ...

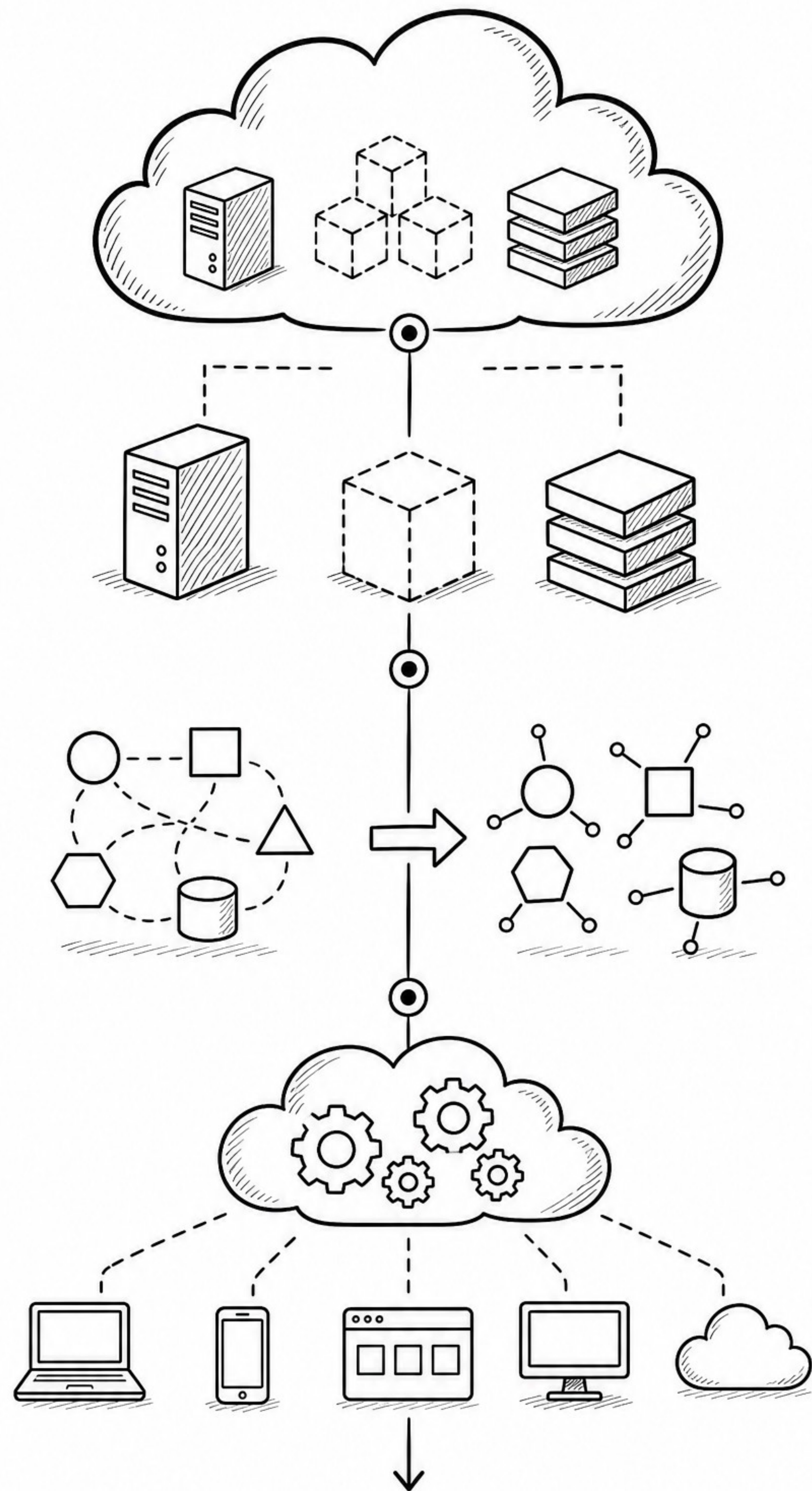
Data Centers

bare-metal servers

Monolithic Architecture

Tightly coupled components, deployed as single-unit





Cloud

bare-metal, VMs, IaaS

Service-oriented Architecture

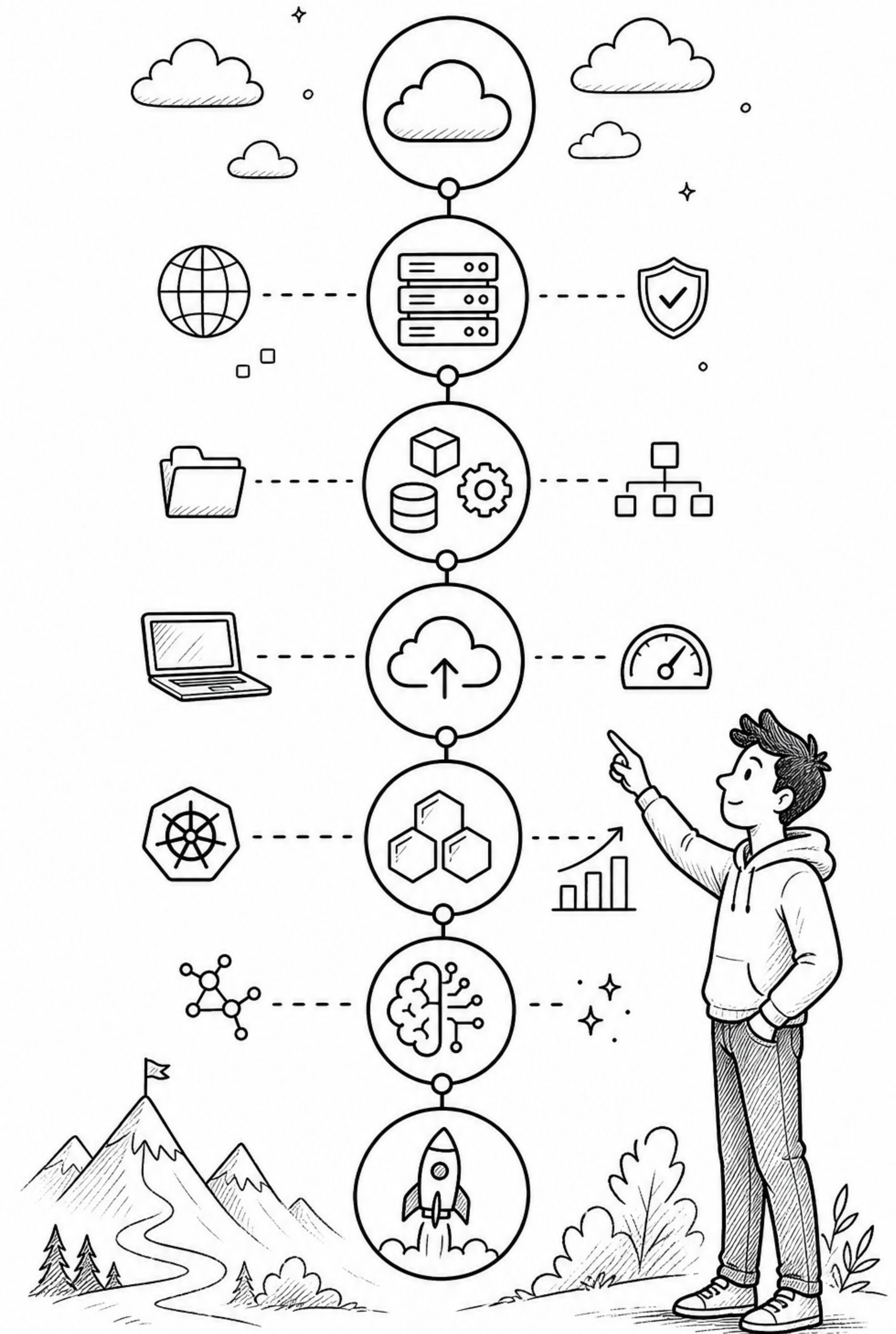
decoupled components, deployed as web services

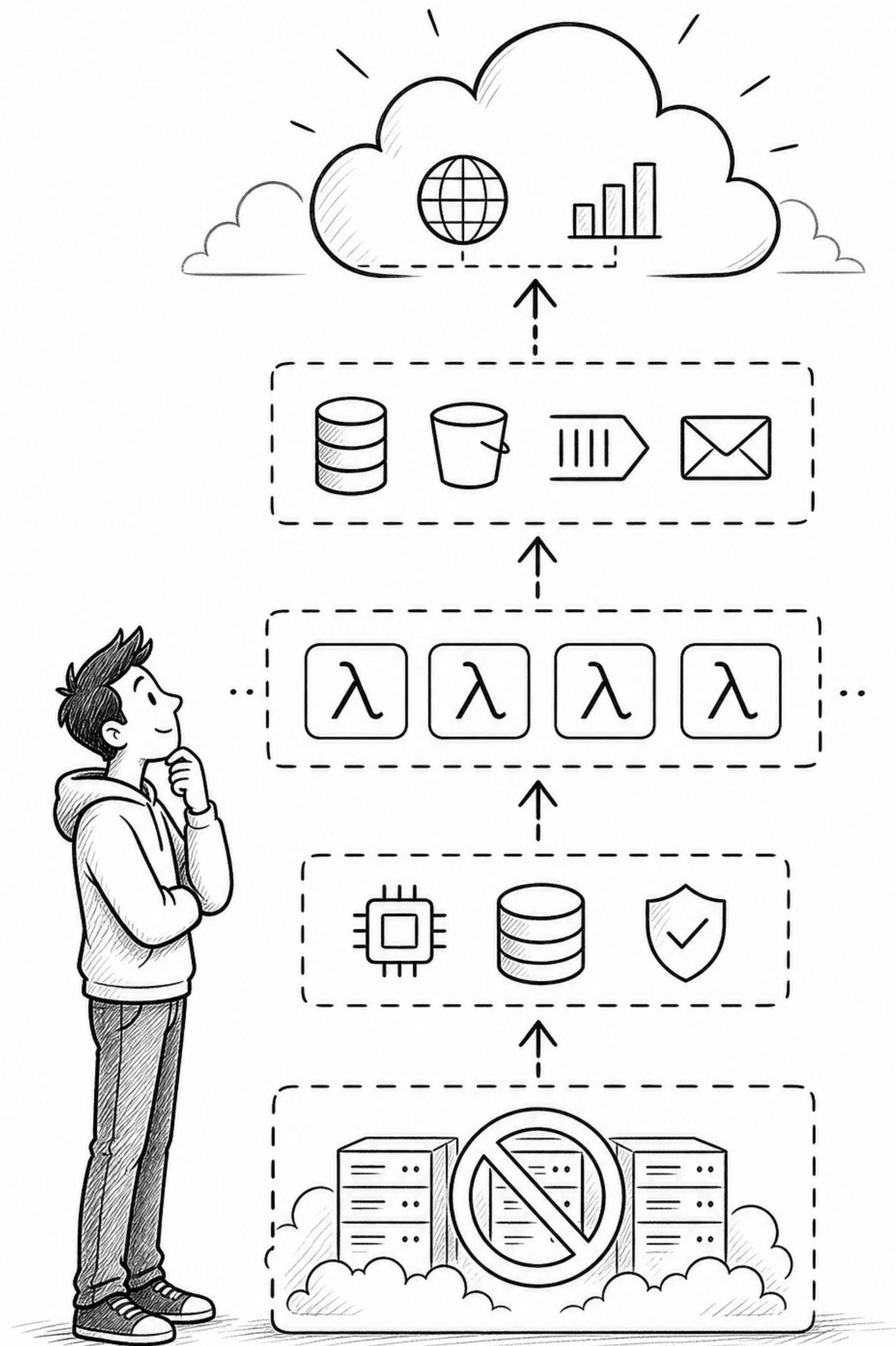
Platform-as-a-Service

shift away from infrastructure management

Micro-Services Architecture

API centric, single responsibility services





functions-as-a-Service

zero administration of infrastructure

Serverless Architecture

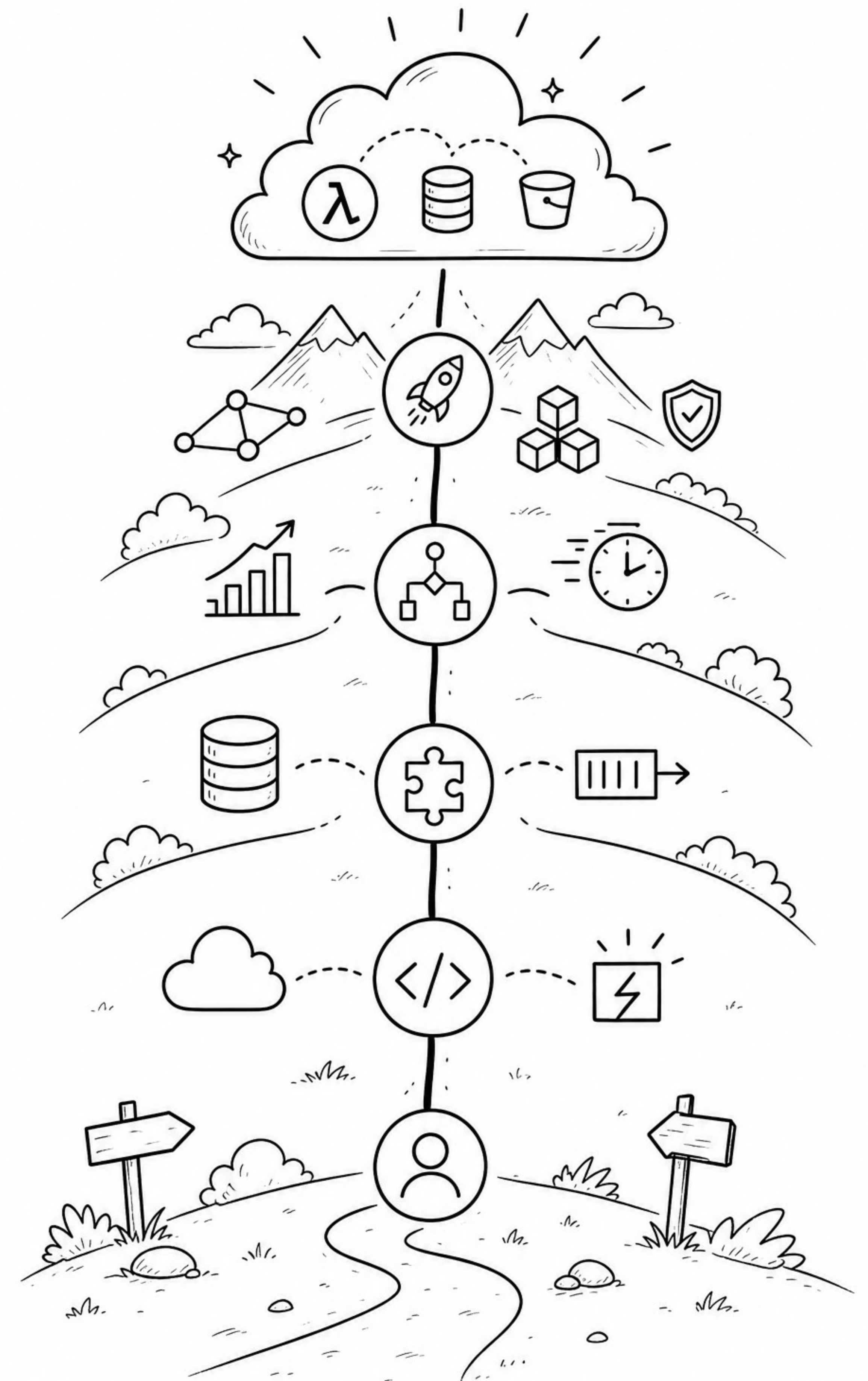
functions as unit of deployment

Intro to Serverless

basic understanding of serverless computing and why you should be interested

We will look at:

- *Evolution*
- *Tenets of serverless computing*
- *Use cases & challenges*
- *Serverless costs compared*

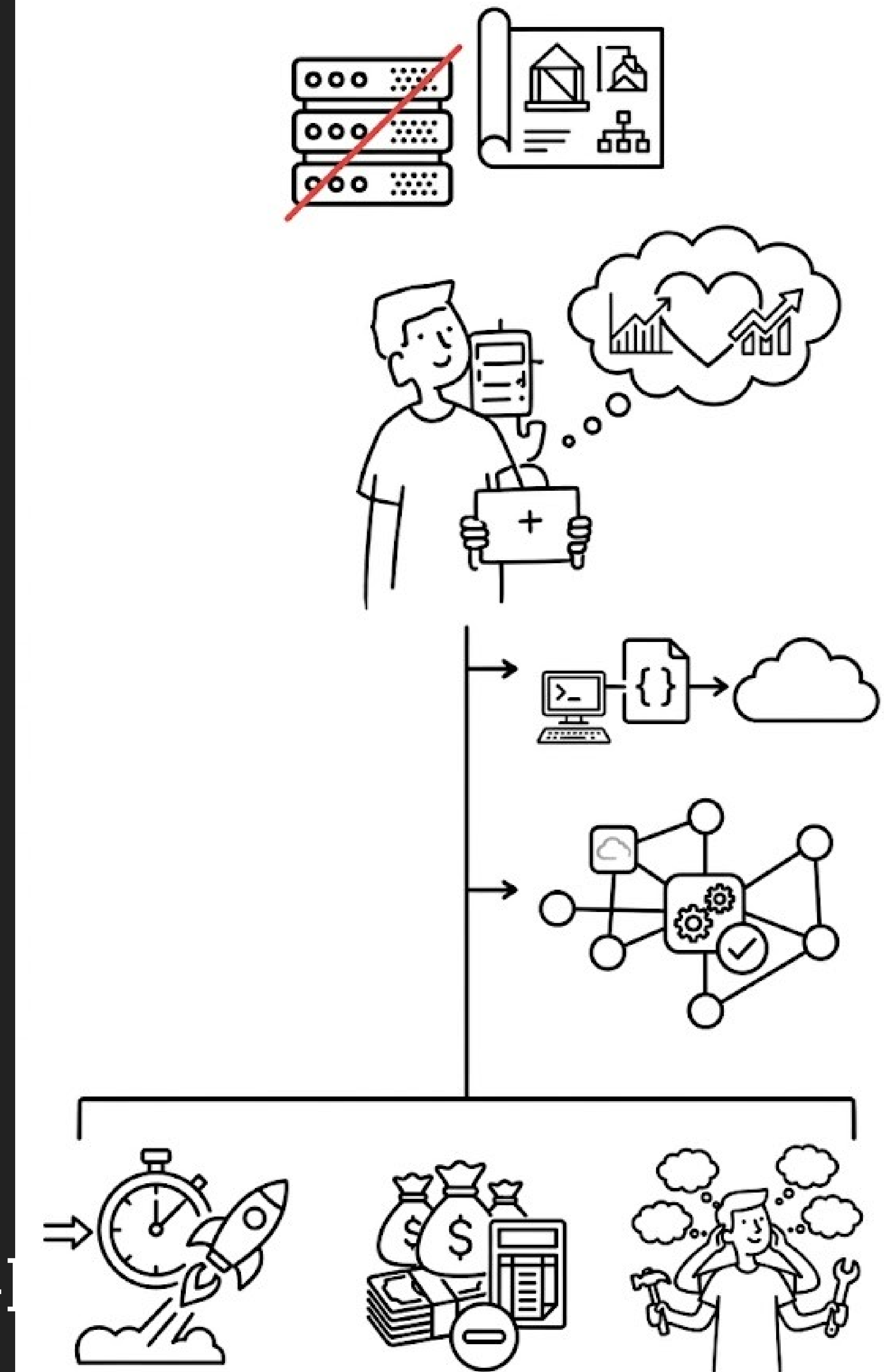


Serverless Architecture

applications run code in **FaaS** services
and leverage
other managed services

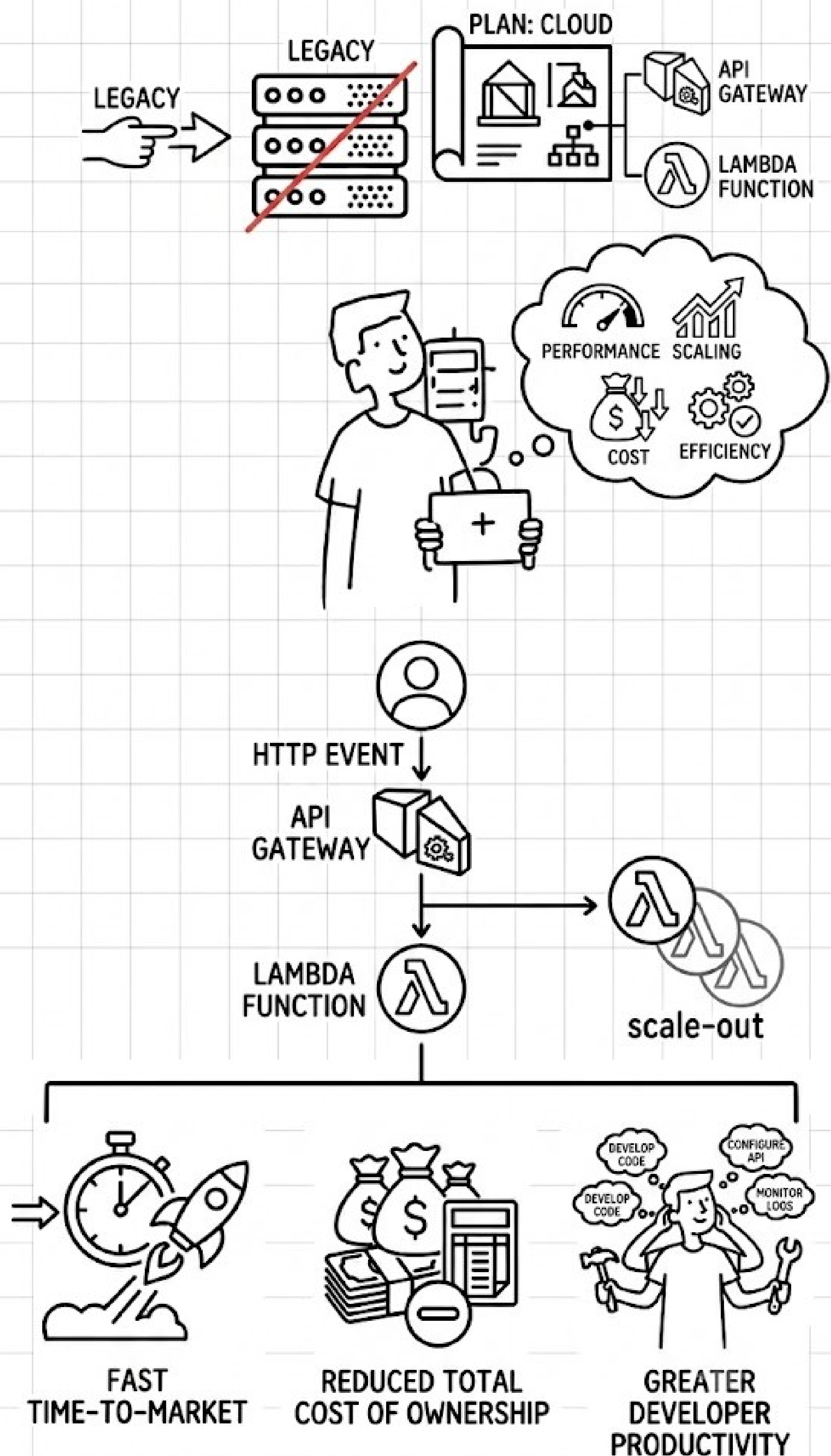
values

shortest time to market lowest total cost of ownership
greater developer productivity



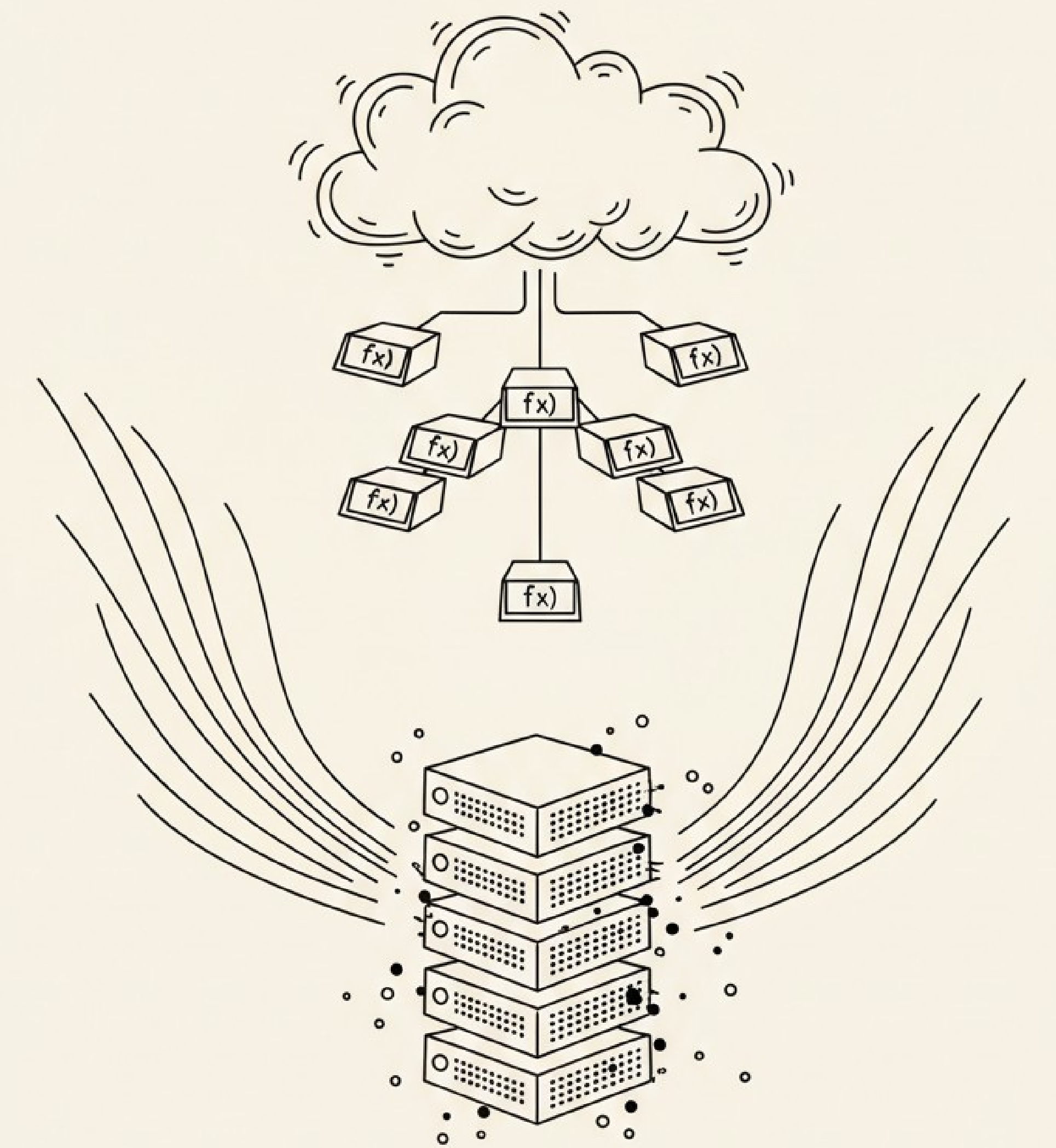
Serverless Architecture

it's all about events



Serverless Computing

myths, benefits and qualities



COST EFFICIENCY



RAPID
DEPLOYMENT

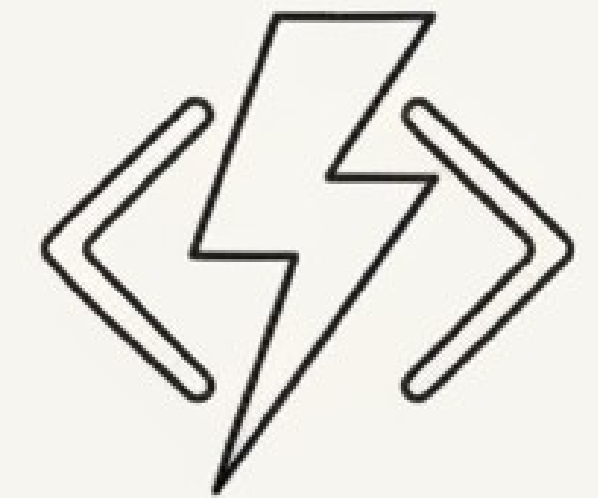
serverless definition

serverless: though servers exists, the
developer does not have to think about
them

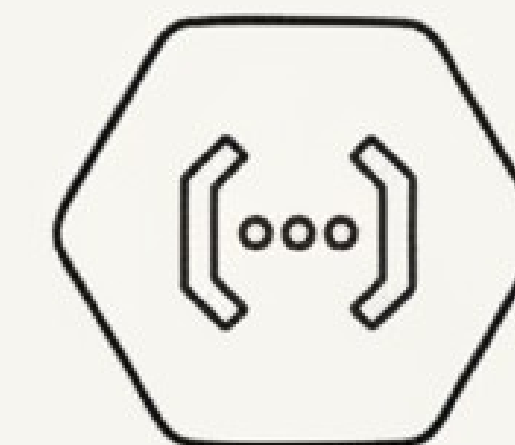
FaaS Providers



**AWS
Lambda**



**Azure
Functions**



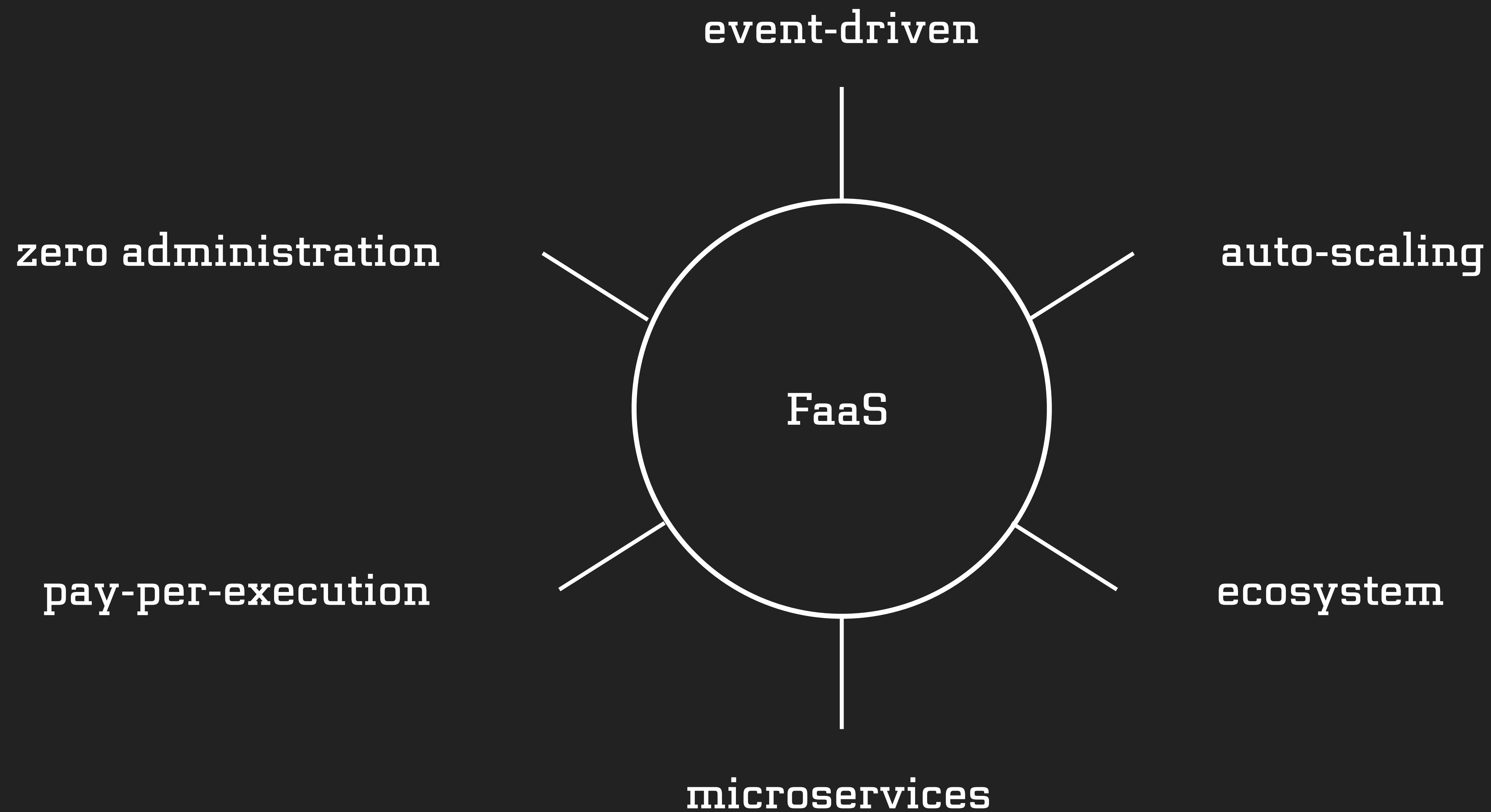
**Google Cloud
Functions**



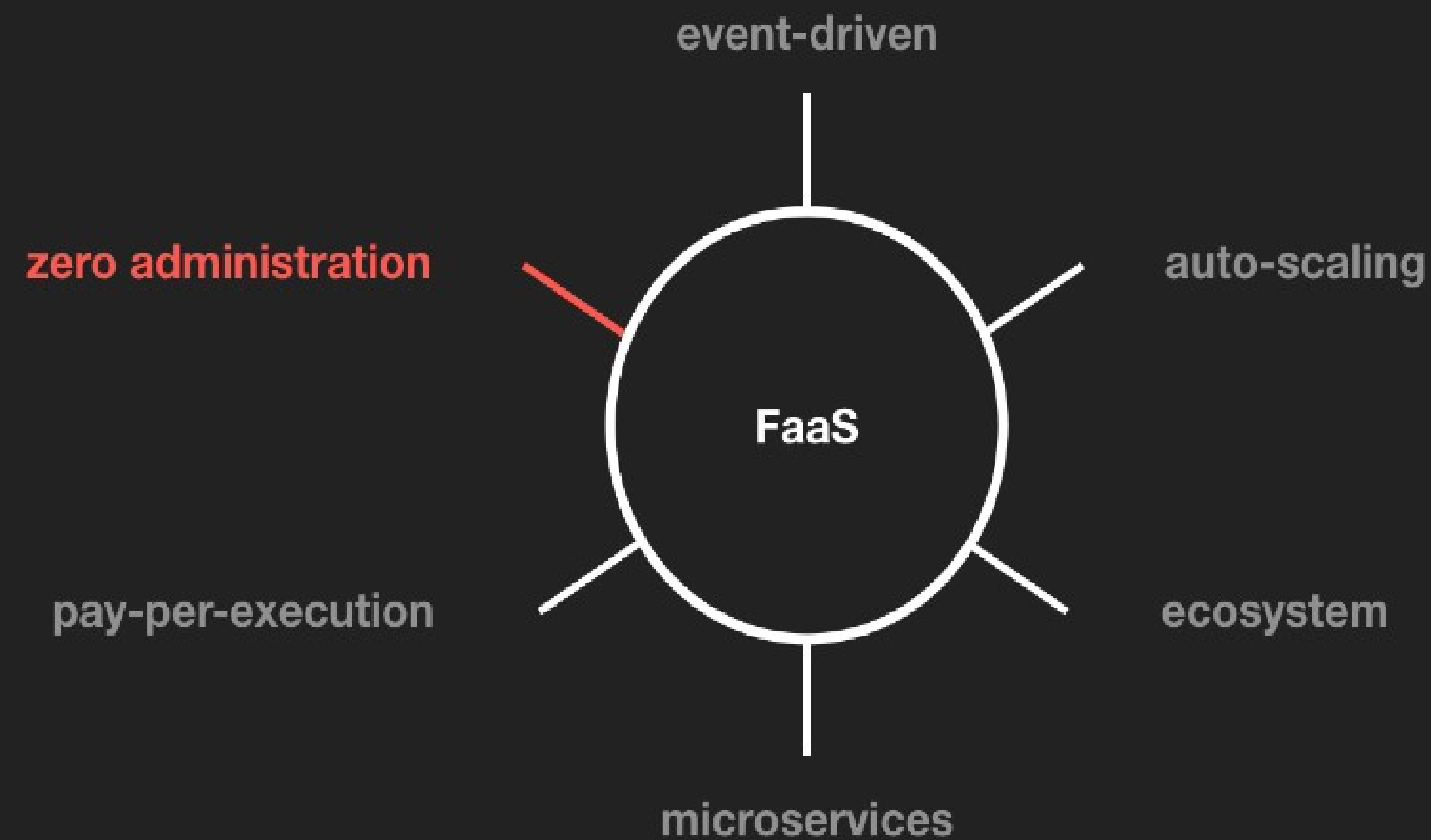
**IBM Cloud
Functions**

Serverless Computing

FaaS qualities

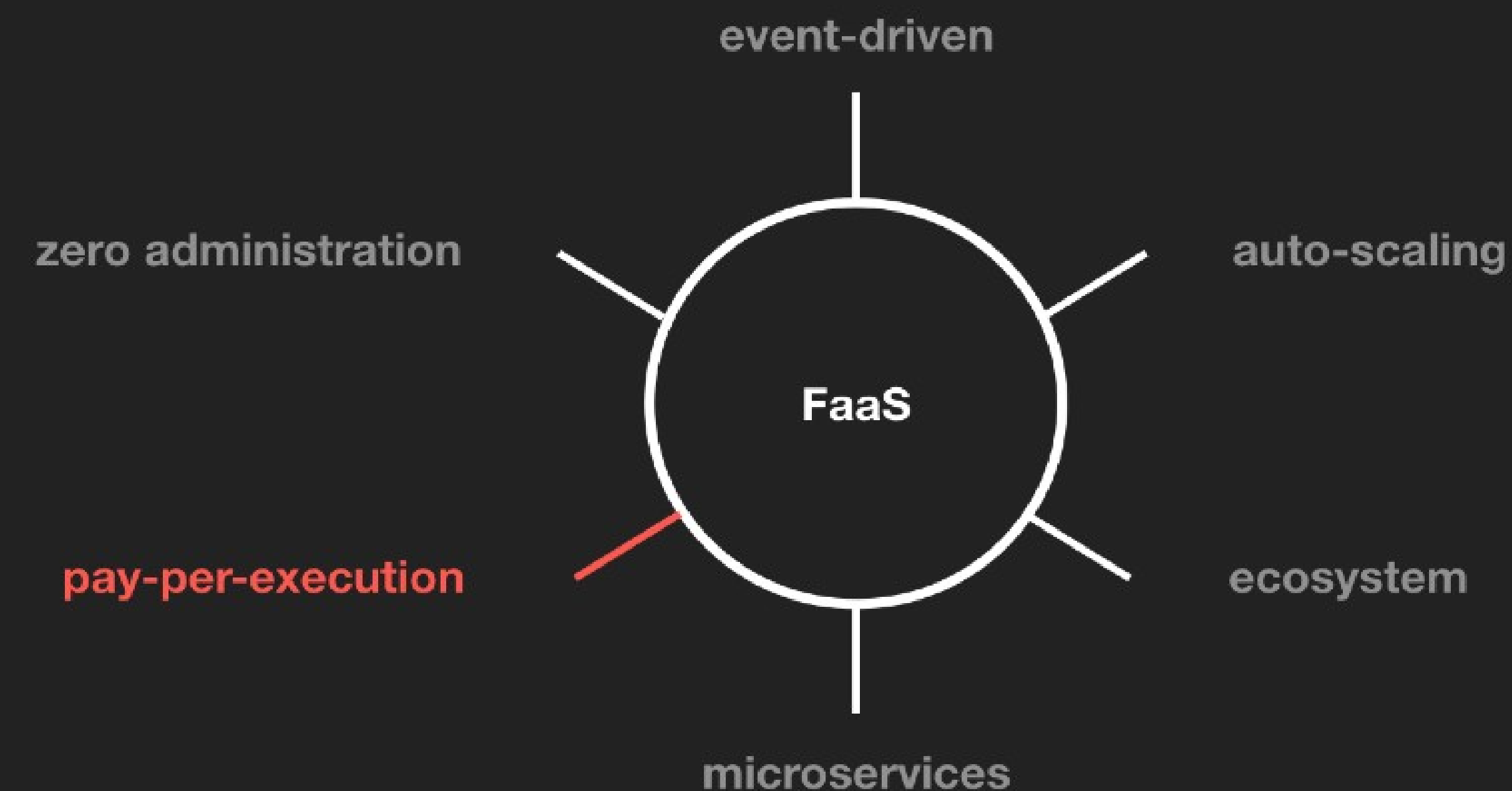


Zero administration



- No servers to maintain
- No OS upgrades, No patches
- Renewed role of devops teams
- Paradigm shift, No ssh
- No control over infrastructure

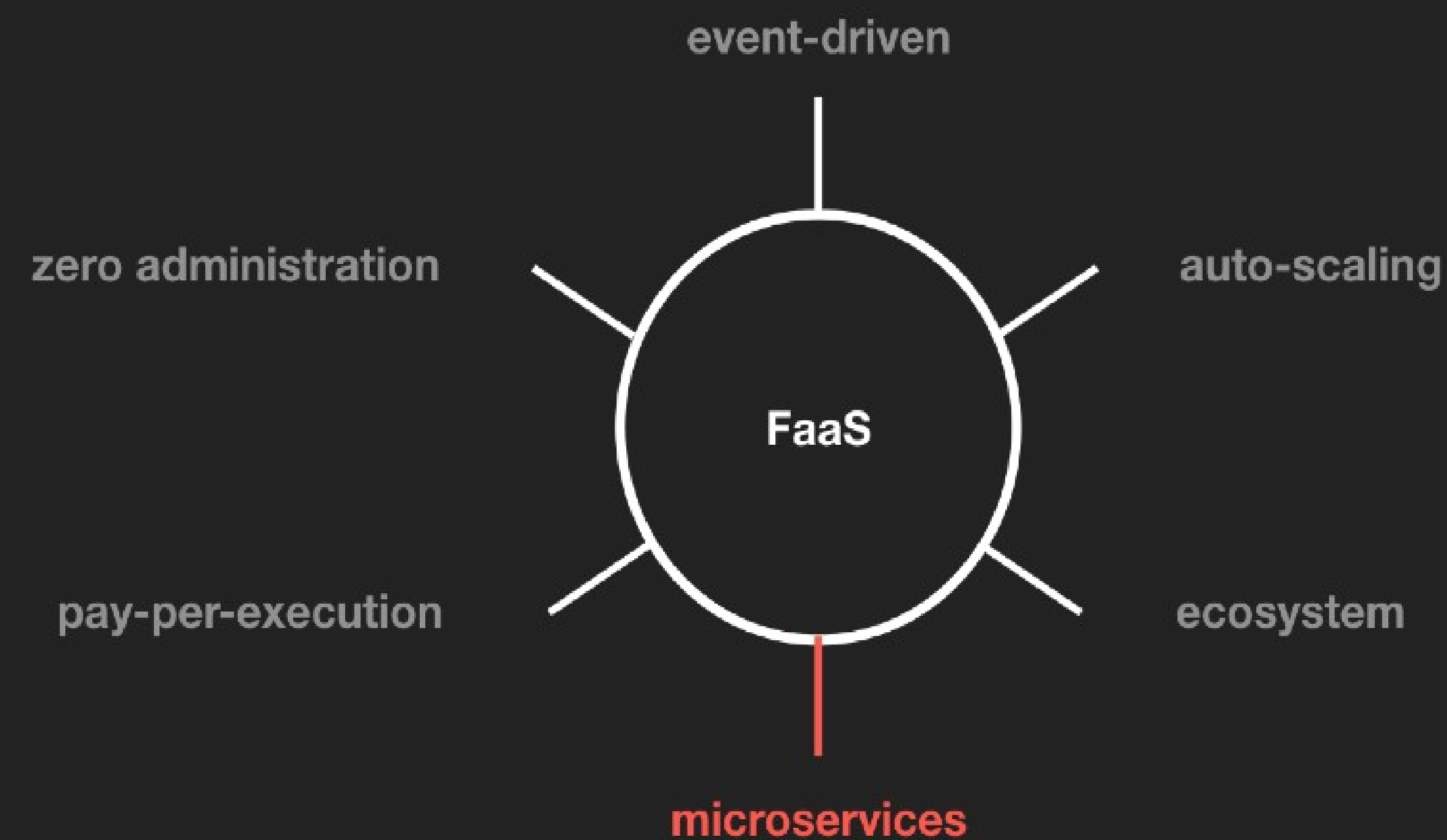
Pay-Per-Execution



- No paying for idle
- upfront cost savings
- micro-billing, per 100 ms *
- 1 million requests free tier *
- \$0.20 per million requests post free tier *

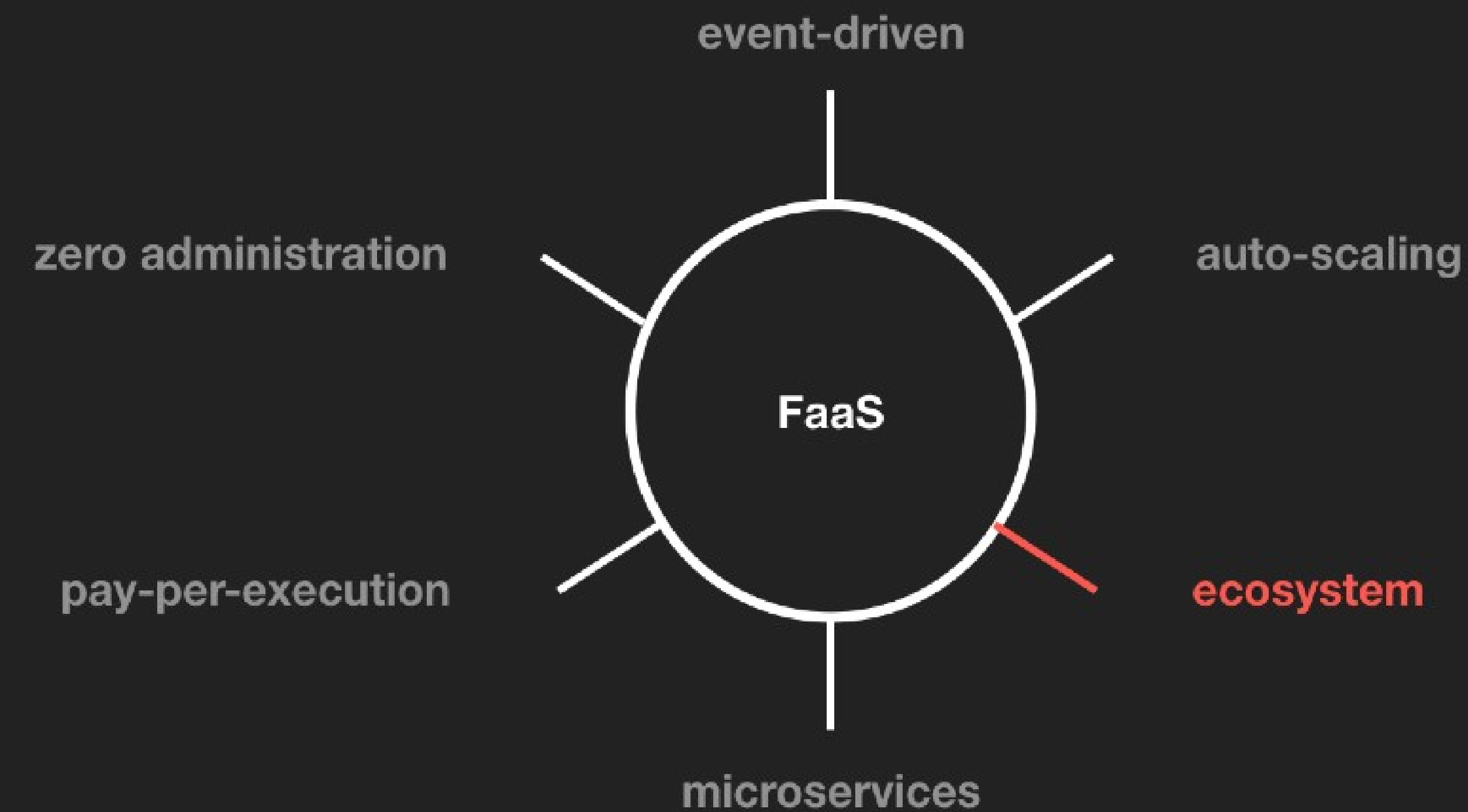
* AWS Lambda pricing

Microservices



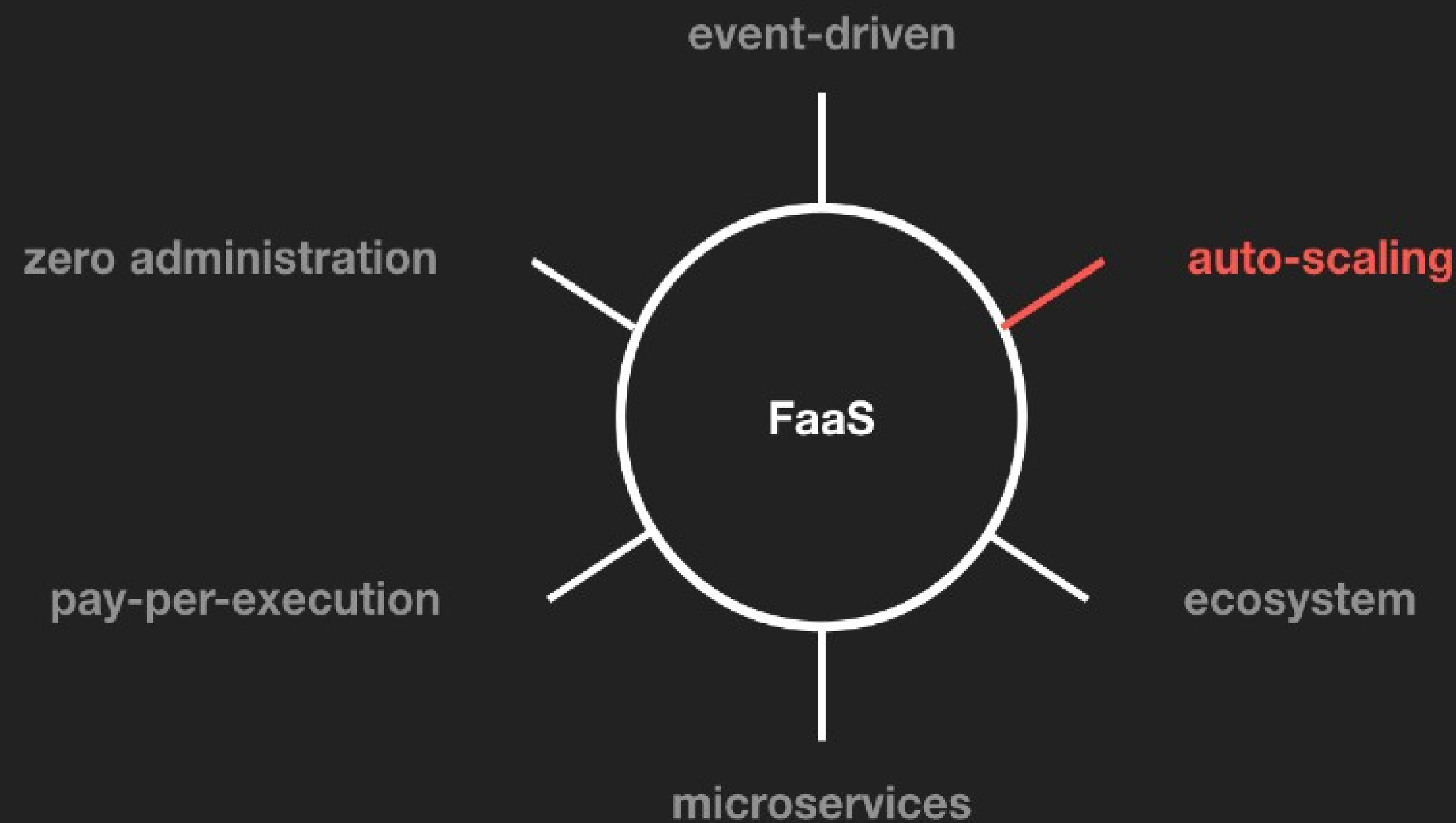
- promotes single responsibility principle based services
- no more death stars, monolithic apps
- varied language support
- small, autonomous, polygot teams
- agility, quick & frequent deployments

Ecosystem



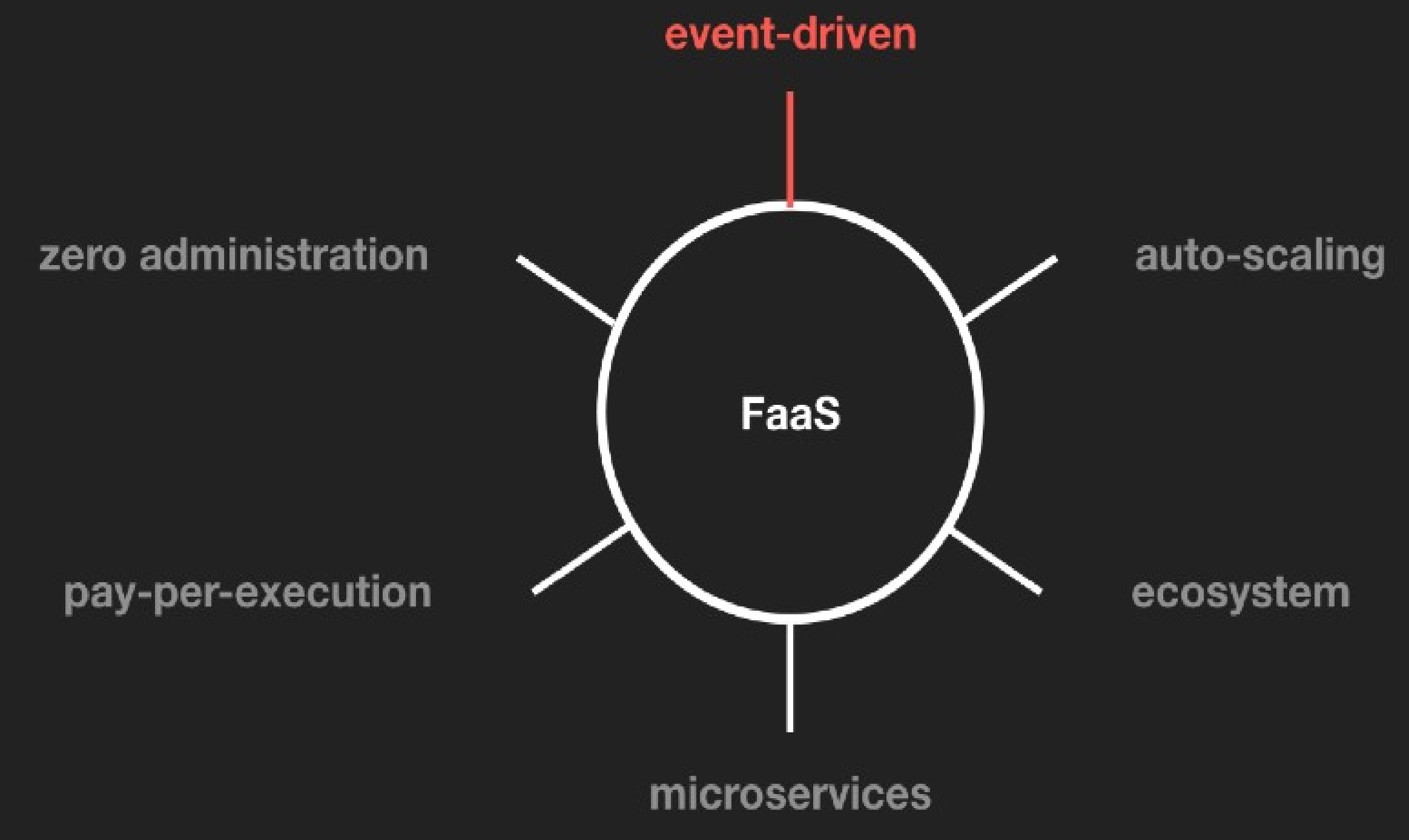
- use provider services
- use of third-party services
- S3, Kinesis, SNS, SQS
- DynamoDB, Firebase, GraphQL
- Cognito, Auth0, Synk, PureSec
- CloudWatch, X-Ray, IOPipe

Auto-scaling



- auto scaling Lambda
- auto scaling DynamoDB
- based on load, throughput consumption
- no need for auto-scaling group policies
- enables high-availability

Event-driven

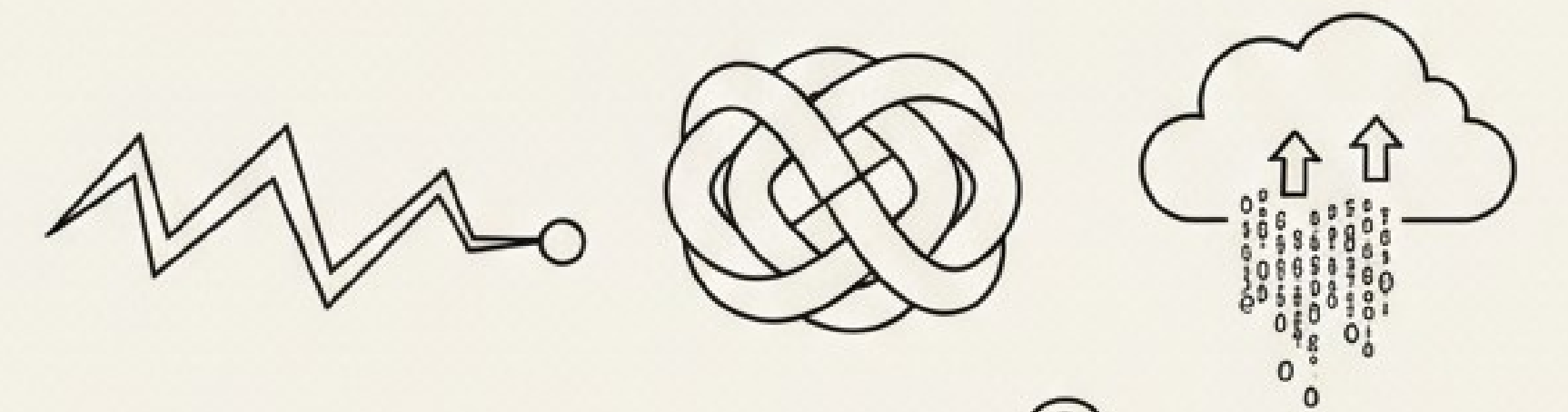


- trigger & respond to events
- http events (API Gateway)
- webhook events
- scheduled events
- rich ecosystem of event sources

SECURITY ASPECTS

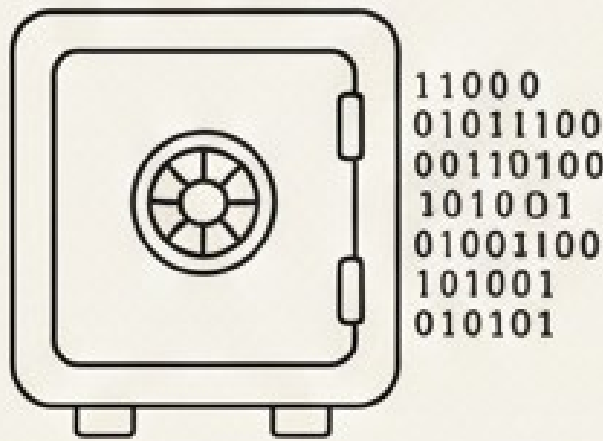


USE CASES

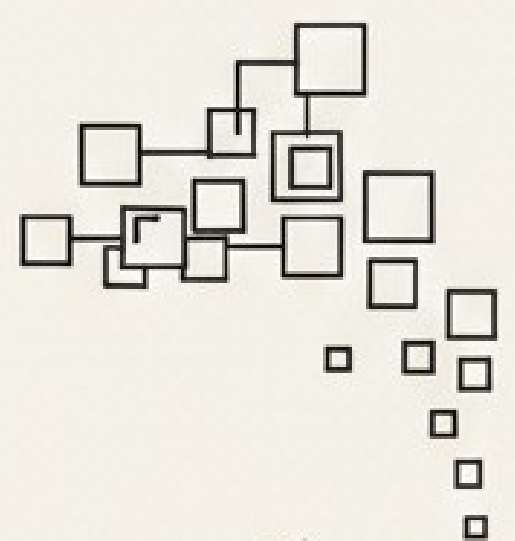


CHALLENGES

DATA PROTECTION



INTEGRATION CHALLENGES



COST MANAGEMENT



PERFORMANCE OPTIMIZATION



VENDOR LOCK-IN CONCERNS

Considerations

security, use cases & challenges

Security

guidelines & recommendations

- Least privileges across resources
- Reduce access to sensitive data, handle exceptions and validate input
- Avoid embedding secrets, access keys and credentials in code
- Throttle and define quotas
- Encrypt data on the wire and data stored at rest
- Scrutinize 3rd-party libraries & APIs for vulnerabilities
- Be aware of new input/event sources

use cases

what are others building

- Real-time/streaming, batch processing: kinesis -> lambda
- REST APIs, Graph APIs: lambda -> API Gateway endpoint
- Event-driven workflows, scheduled tasks, data transforms

use cases

what are others building

web, mobile & IoT backends

form processing

authentication

devops automation

chatbots

file manipulation

voice apps (Alexa)

ETL workloads

image resizing

video transcoding

security audits

dynamic websites

web hook listeners

CRON jobs

CI/CD pipelines

log analytics

challenges

it's not all rosy

cold start latency, more with VPC

price at high volumes

developer experience

limitations

fear of vendor lock-in

application lifecycle management

metrics & monitoring

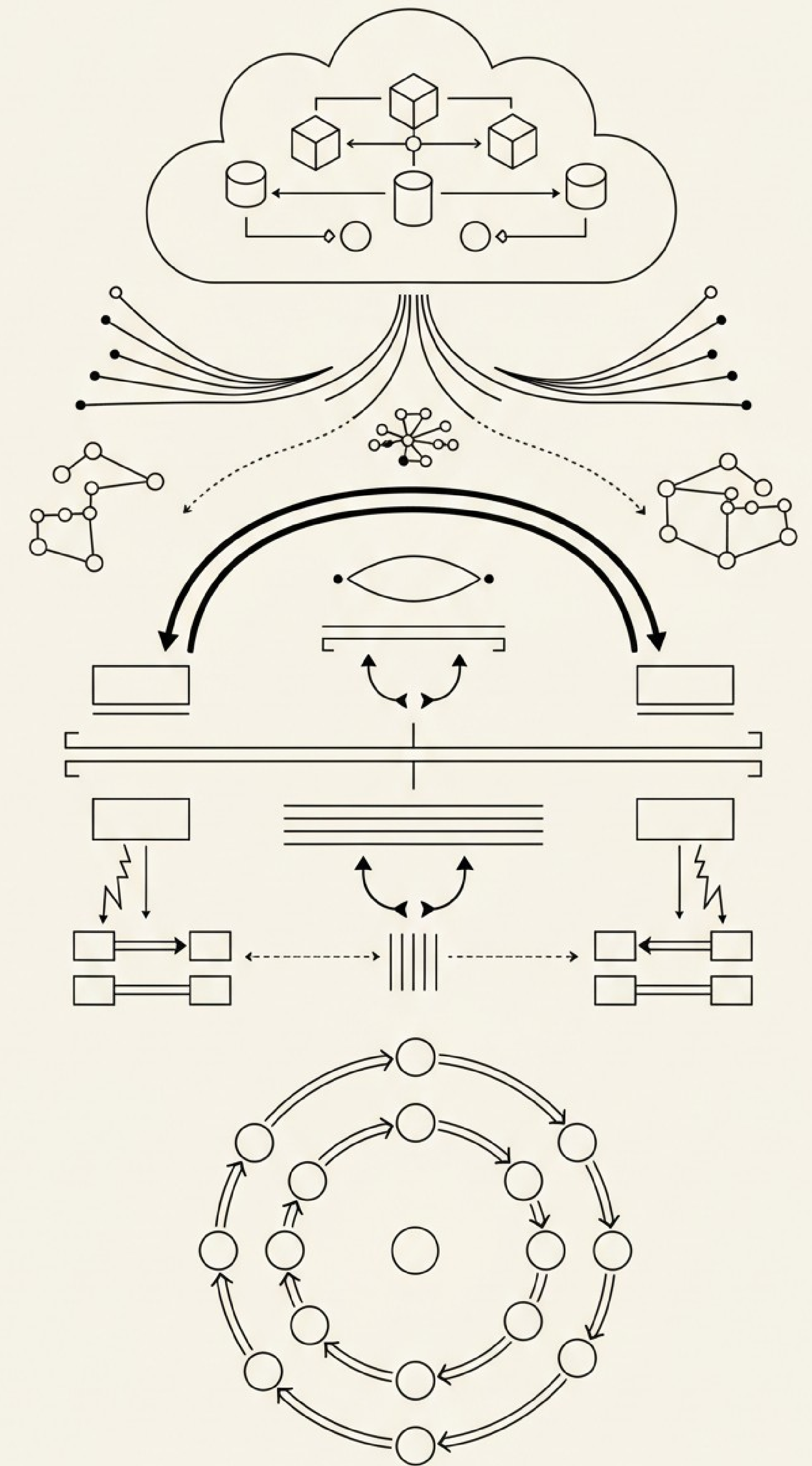
service discovery

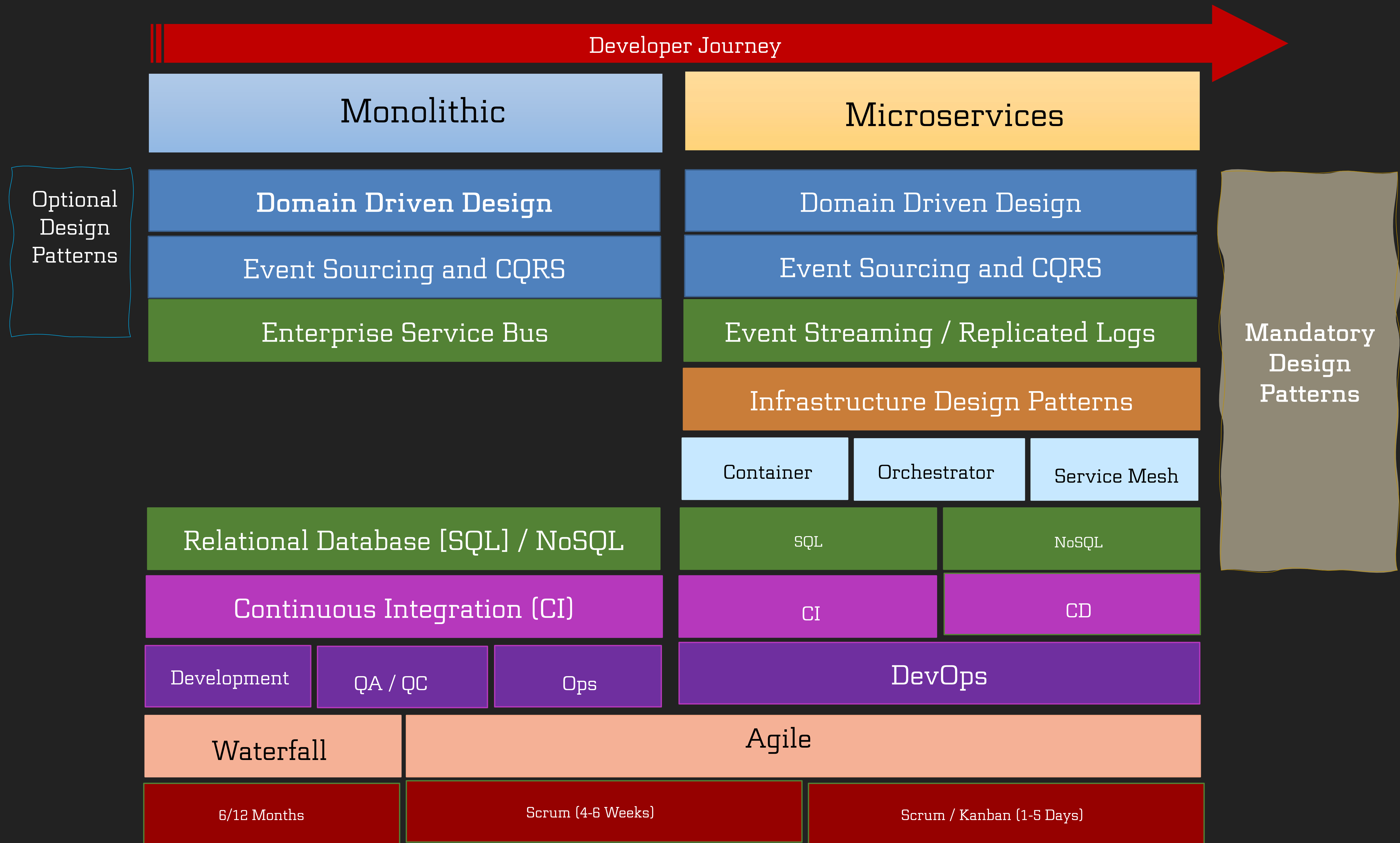
team collaboration

standardization

Cloud Adoption App

Transformat

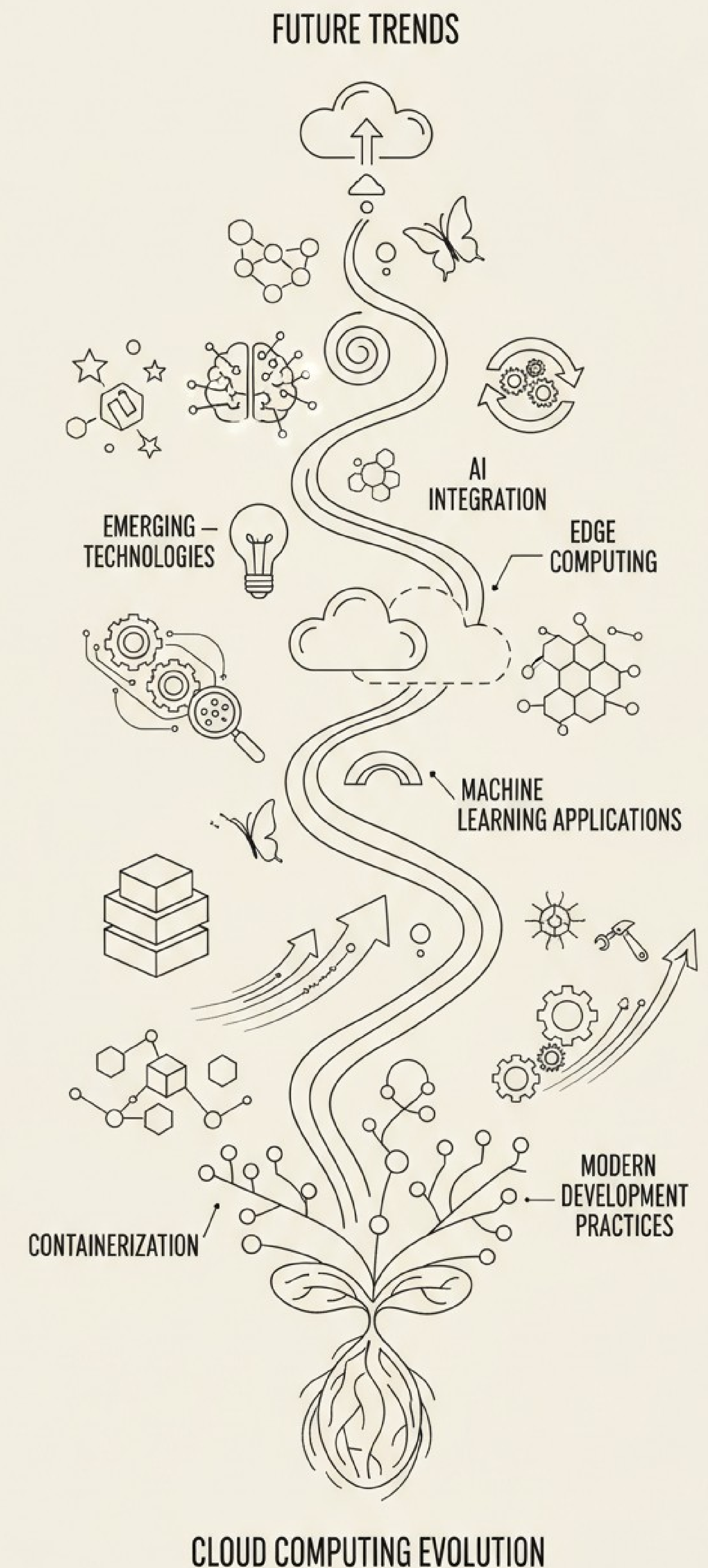




Cloud Features

1. Fault Tolerance
2. Scalability
3. Agility
4. Total Cost of Ownership

1. Hybrid Cloud
2. Multi Cloud



Cloud Migration Strategies

REHOST

Lift and Shift

Move the assets to cloud without any code change

Less Risk

No Additional Benefits other than being in Cloud



Risk



Benefits



Cost

Cloud Migration Strategies

REPLATFORM

Lift & Re-Shape

Move the assets after
Refactoring the assets
to take advantage of cloud.



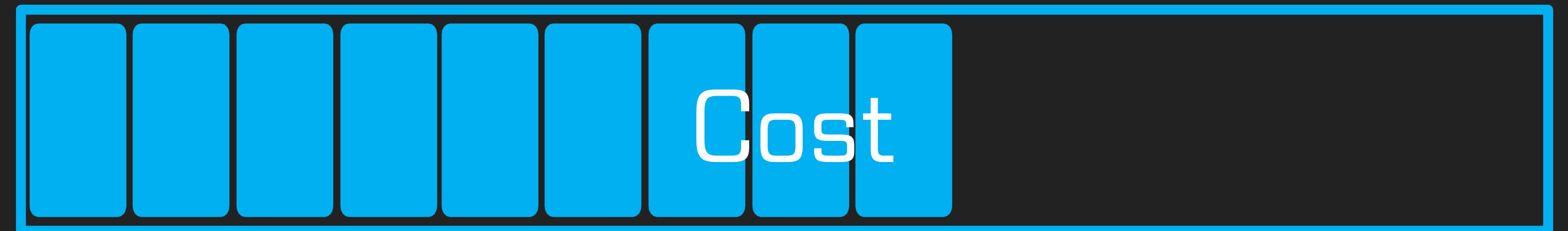
Cloud Migration Strategies

REPURCHASE

Drop & Shop

Abandon the existing
feature / module and
Shop for a new Service
available in the Cloud

Ex. HR / CRM etc.

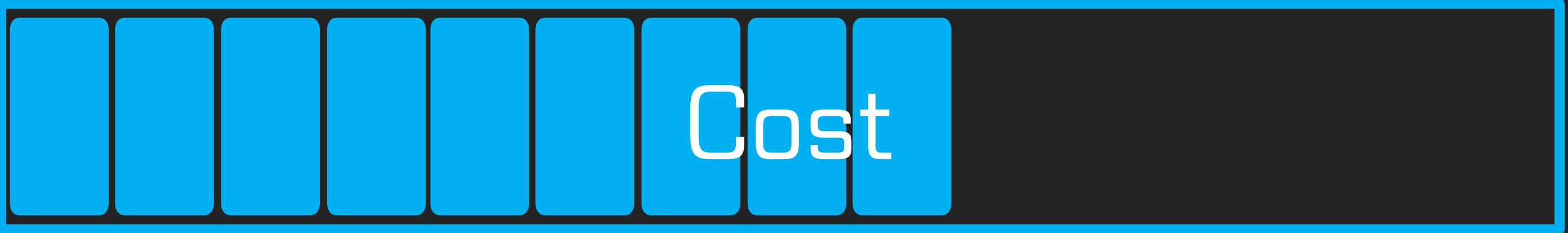


Cloud Migration Strategies

REARCHITECT

REARCHITECT

Redesign and transform the monolith to a Cloud Native App.



Cloud Migration Strategies

RETIRE

RETIRE

Sunset
applications that
are not relevant
anymore.



Cloud Migration Strategies

RETAIN

RETAIN

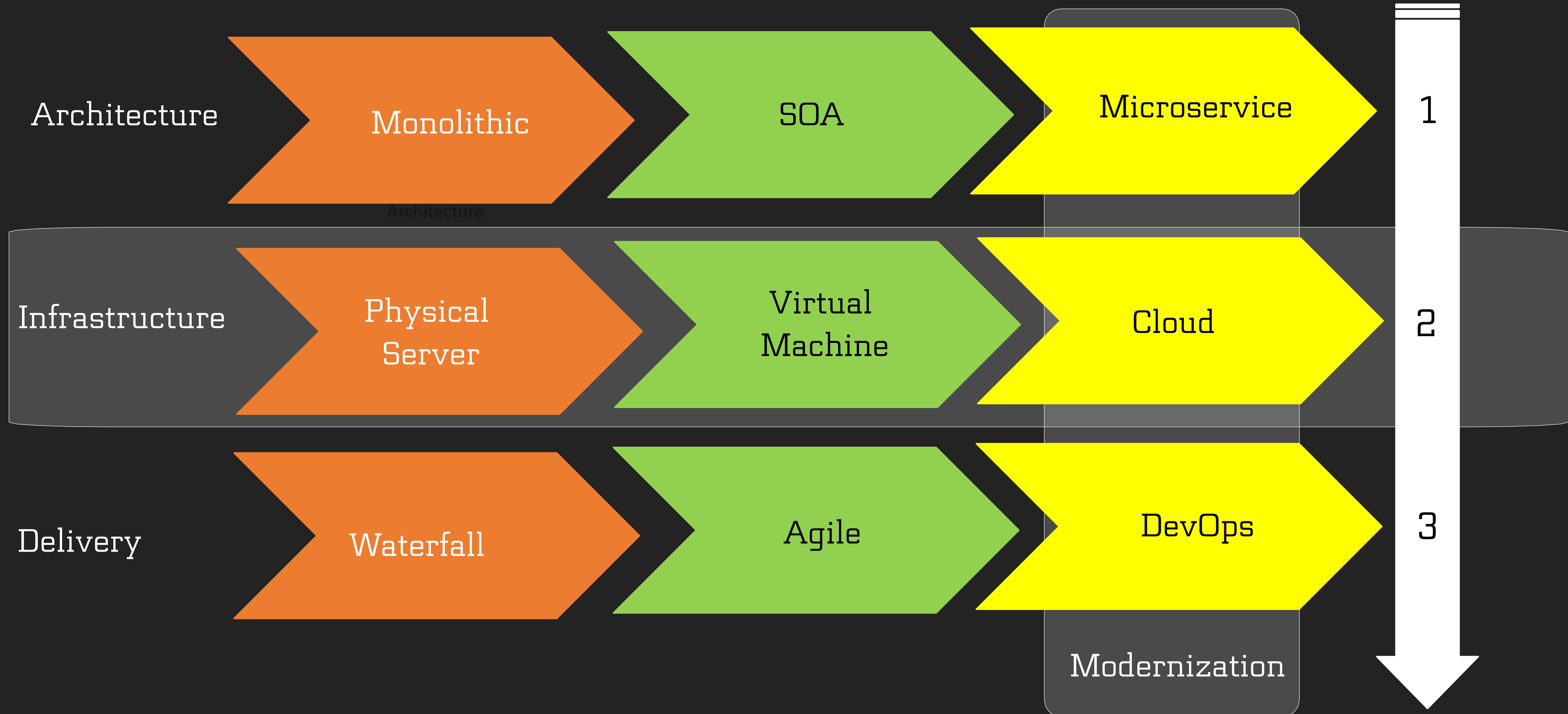
Do Nothing approach.

Evaluate the criticality
of the application
later.



Application Modernization

Transformations



Modernization Journey

▶ Start new features as Microservices

Incrementally establish the success early.

▶ Expose Legacy On-Premise Apps API's

If legacy Apps can't be shifted to Cloud

▶ Refactor Monolithic features to Microservices

Breakdown and Deploy Feature by Feature

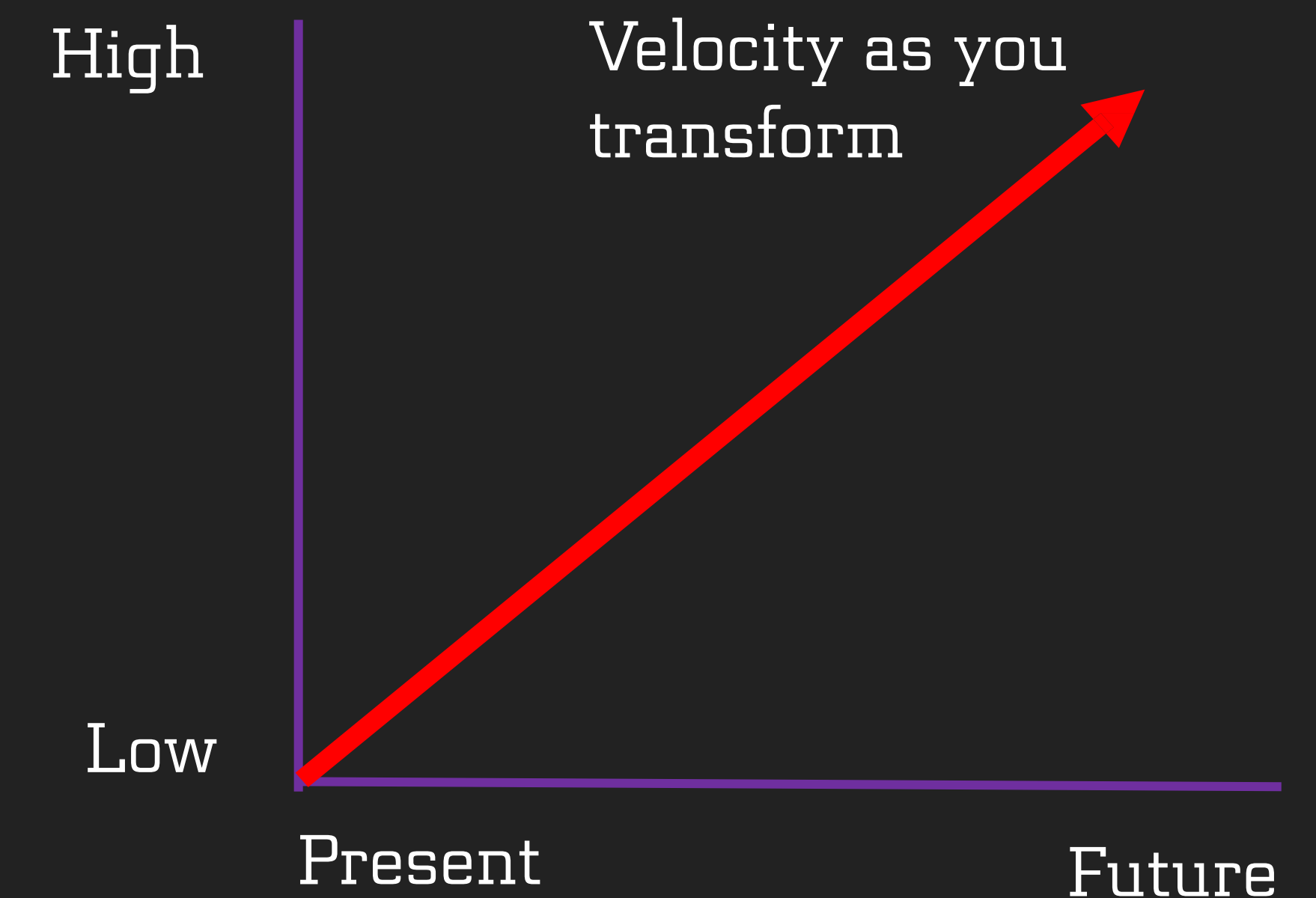
▶ Containerize the Microservice

Reduce costs, simplifies the operations and consistent environment between Dev, QA and Production

▶ Monolith De-commission Plan

Breakdown and Deploy Feature by Feature

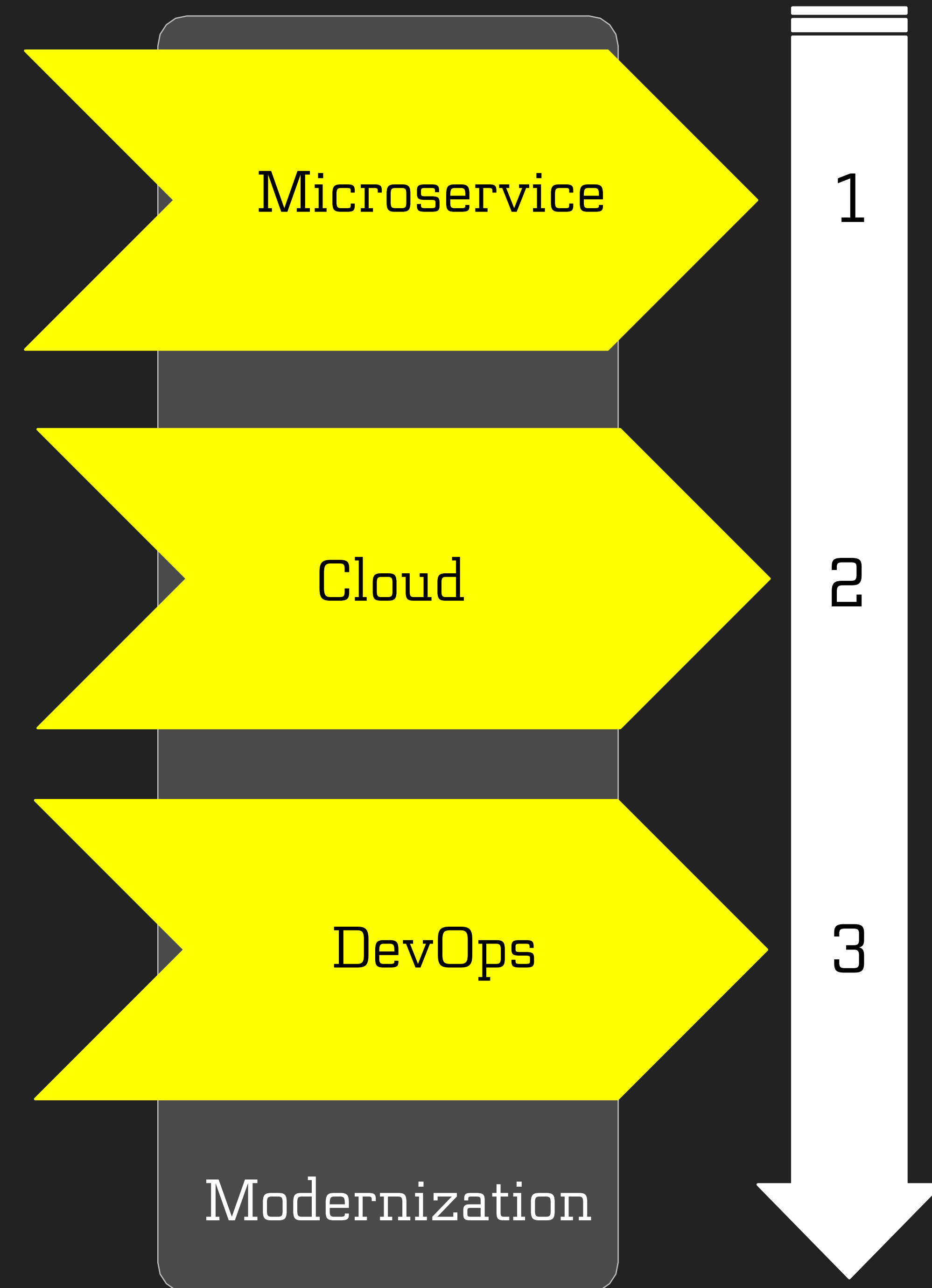
Increase your
Delivery Velocity
along the Journey



Summary – Cloud Adoption

- ▶ Re-Host
- ▶ Re-Platform
- ▶ Re-Purchase
- ▶ Re-Architect
- ▶ Retire
- ▶ Retain

3 Transformations



Cloud

Infrastructure

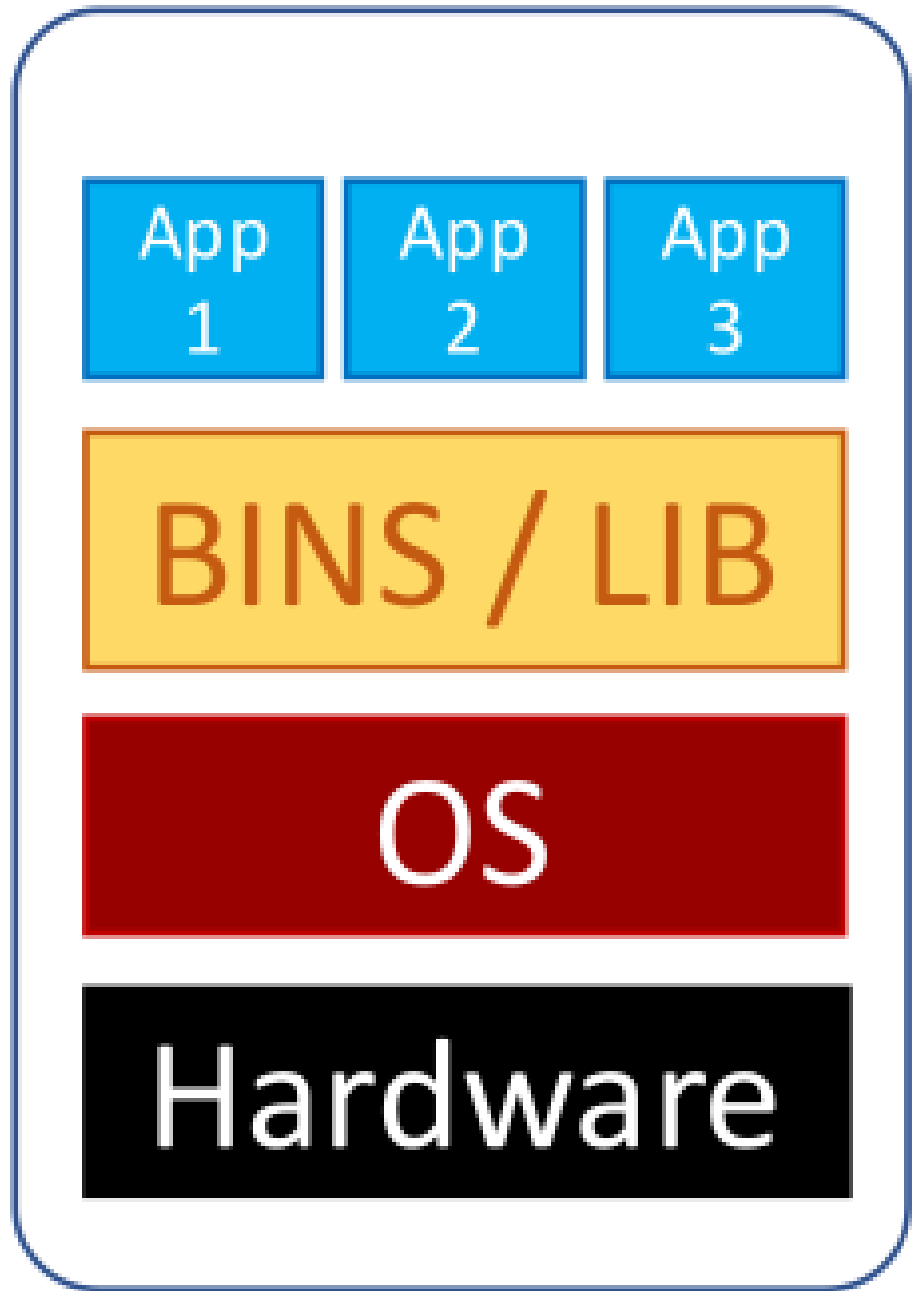


Servers / Virtual Machines / Containers

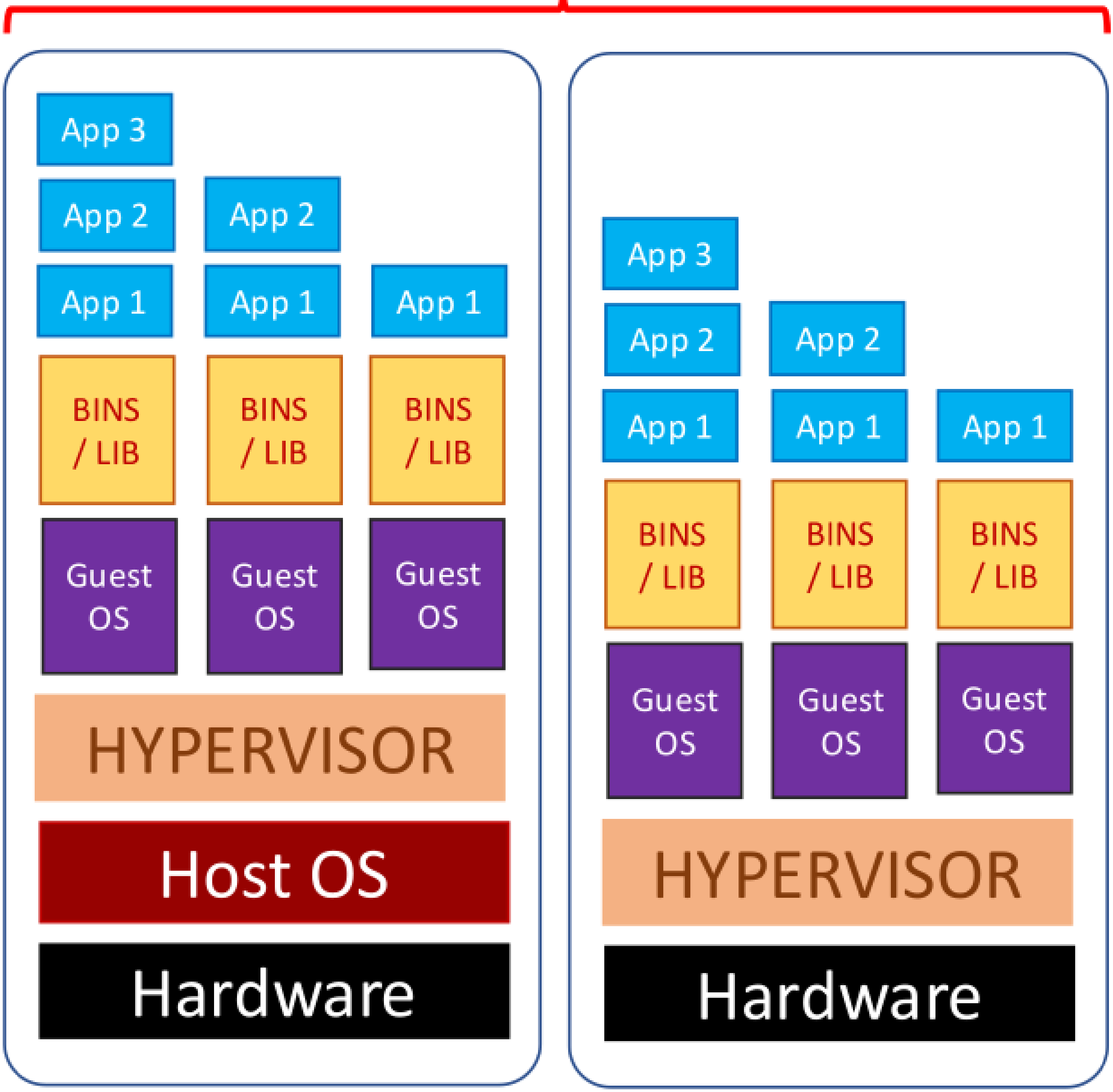
Virtualizes the Hardware
Creates Virtual Machines

Virtualizes the OS
Create Secure Sandboxes in OS

Data Center
No Virtualization

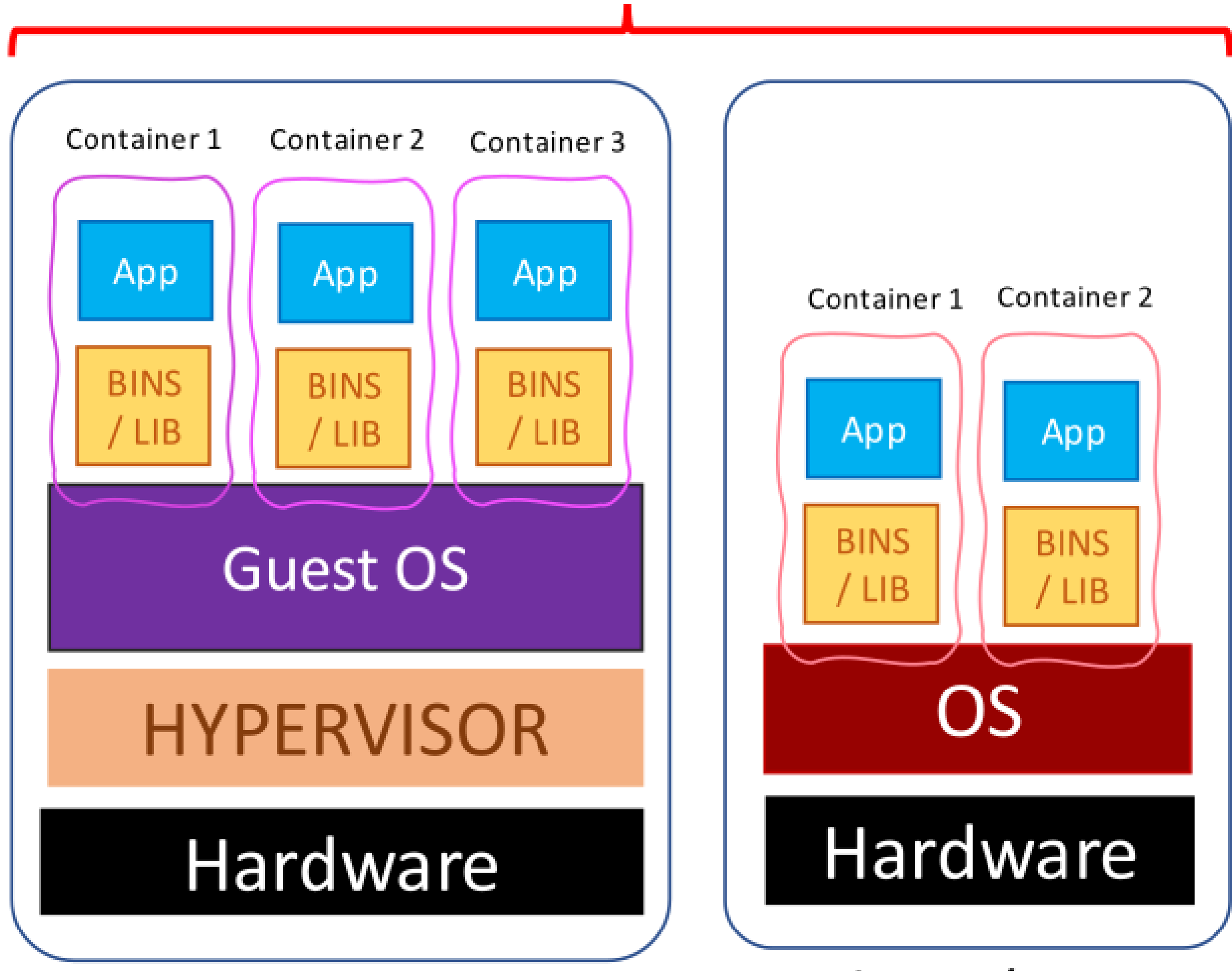


Server



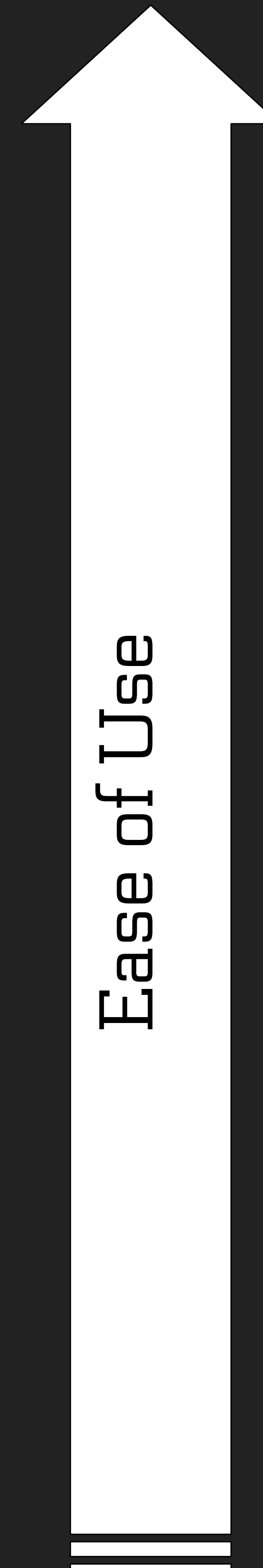
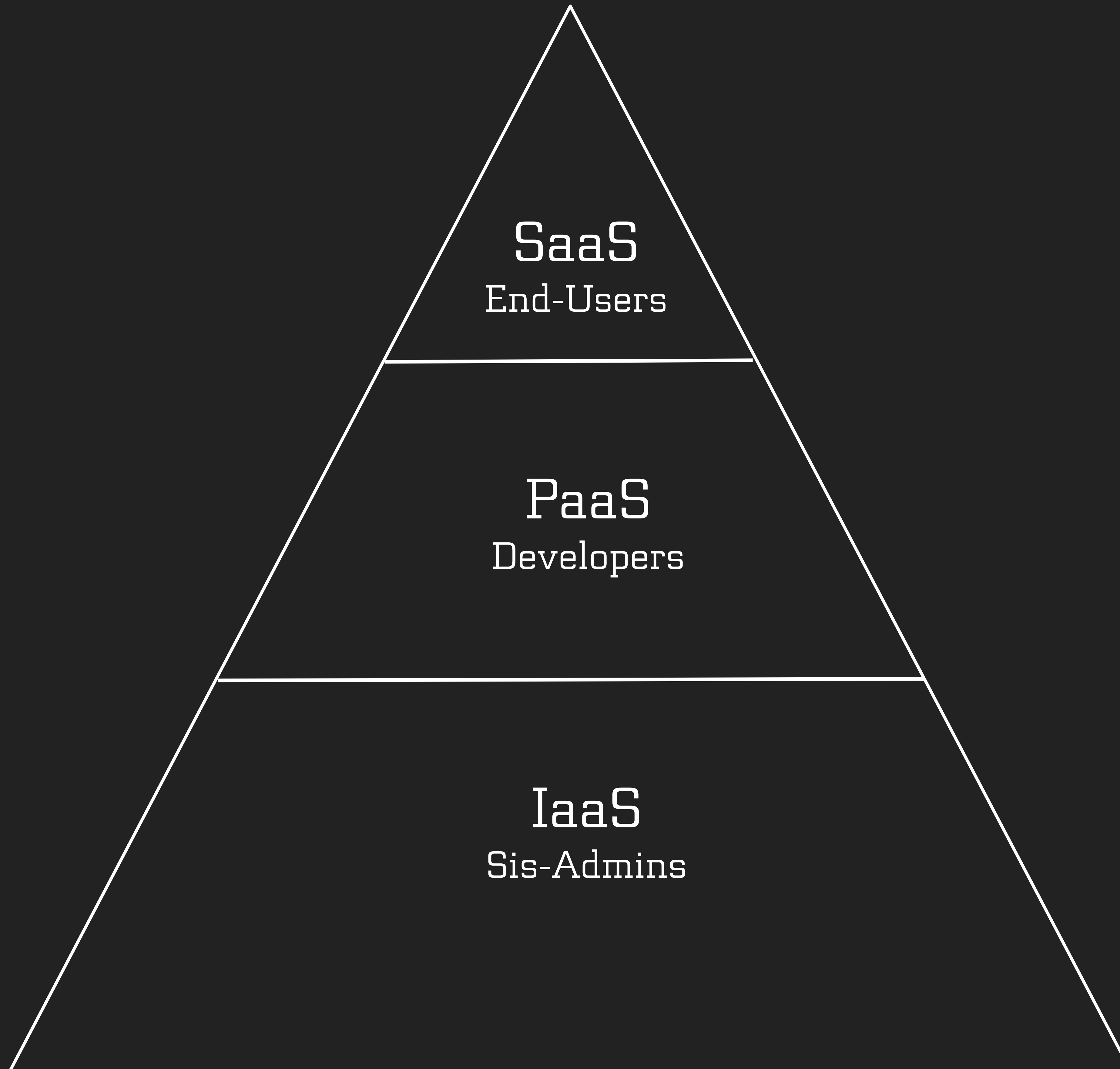
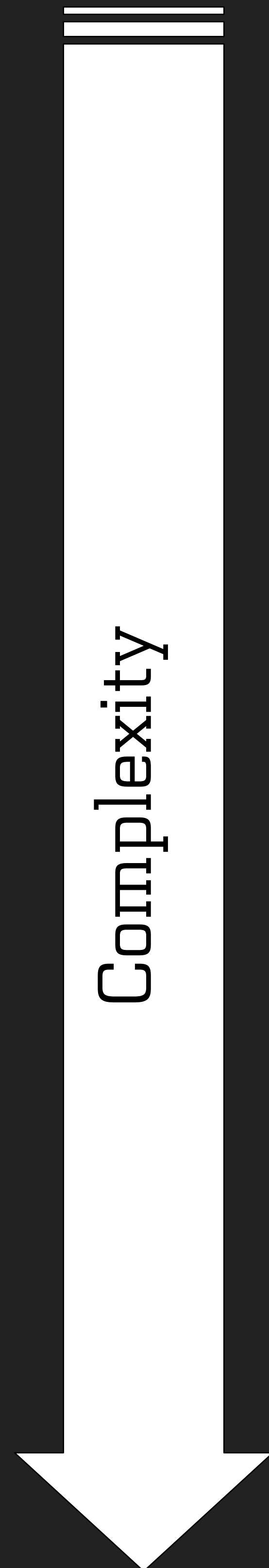
Type 2 Hypervisor

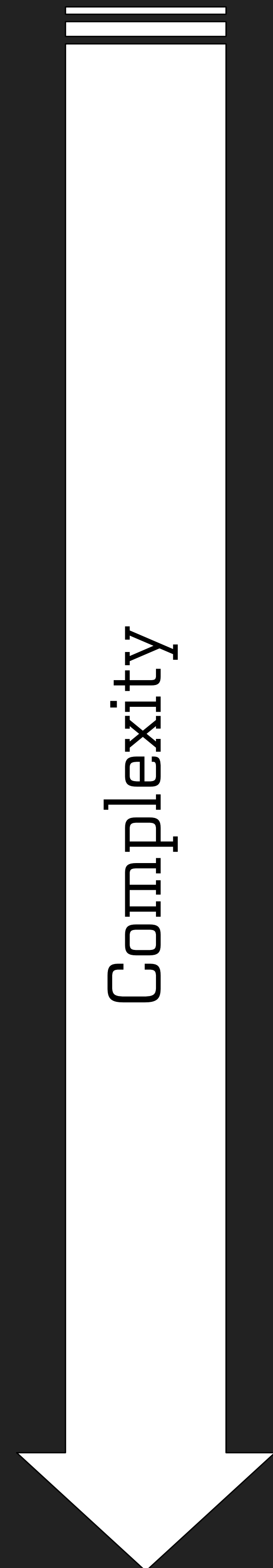
Type 1 Hypervisor



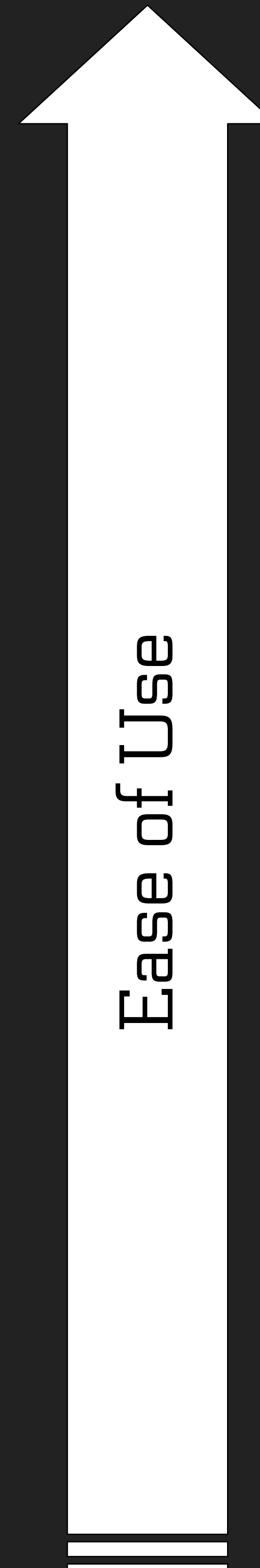
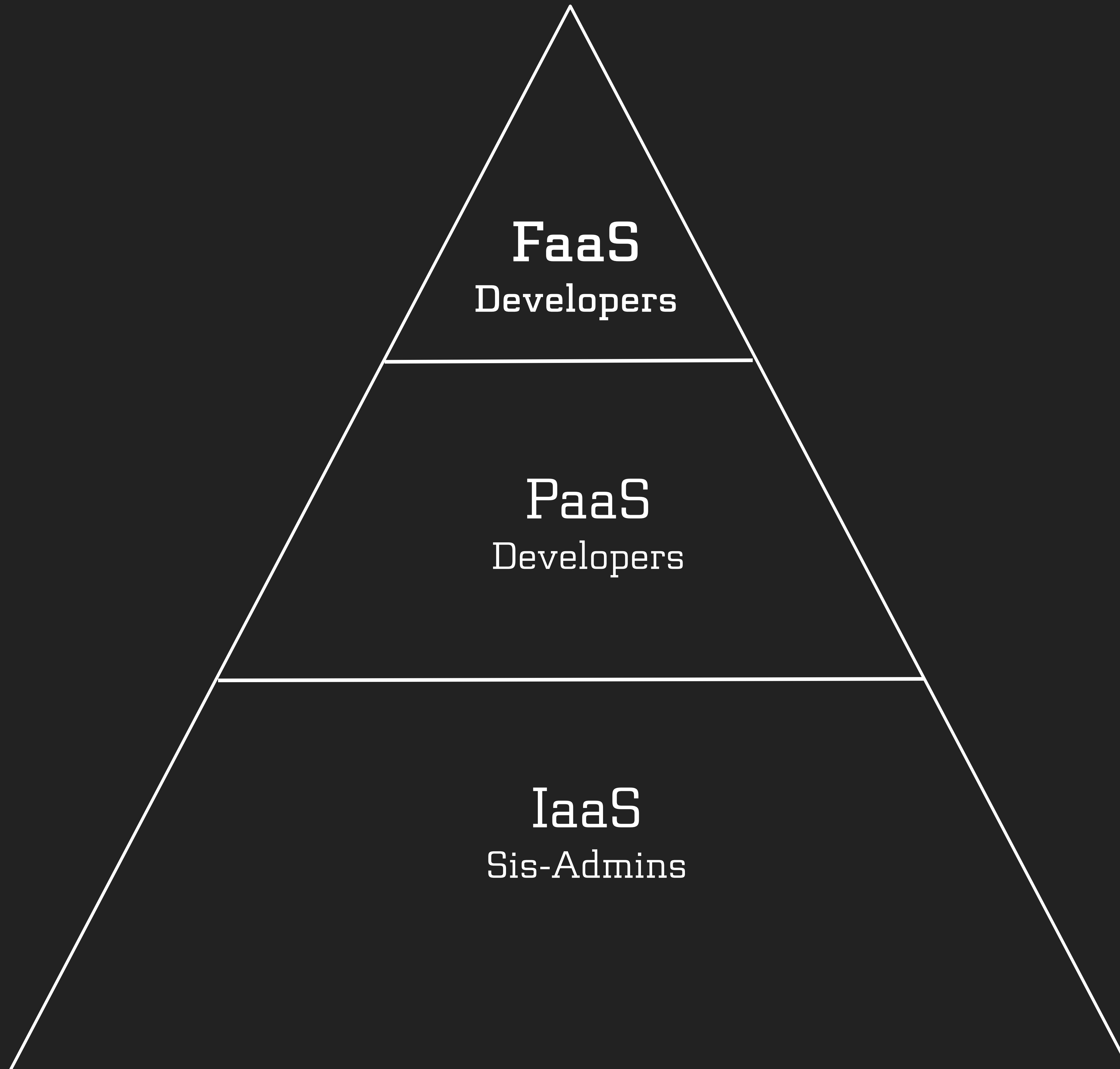
Type 1 Hypervisor

Desktop / Laptop





Complexity



Ease of Use

Infrastructure as a Service (IaaS)

Networking

Generic Compute

GPU

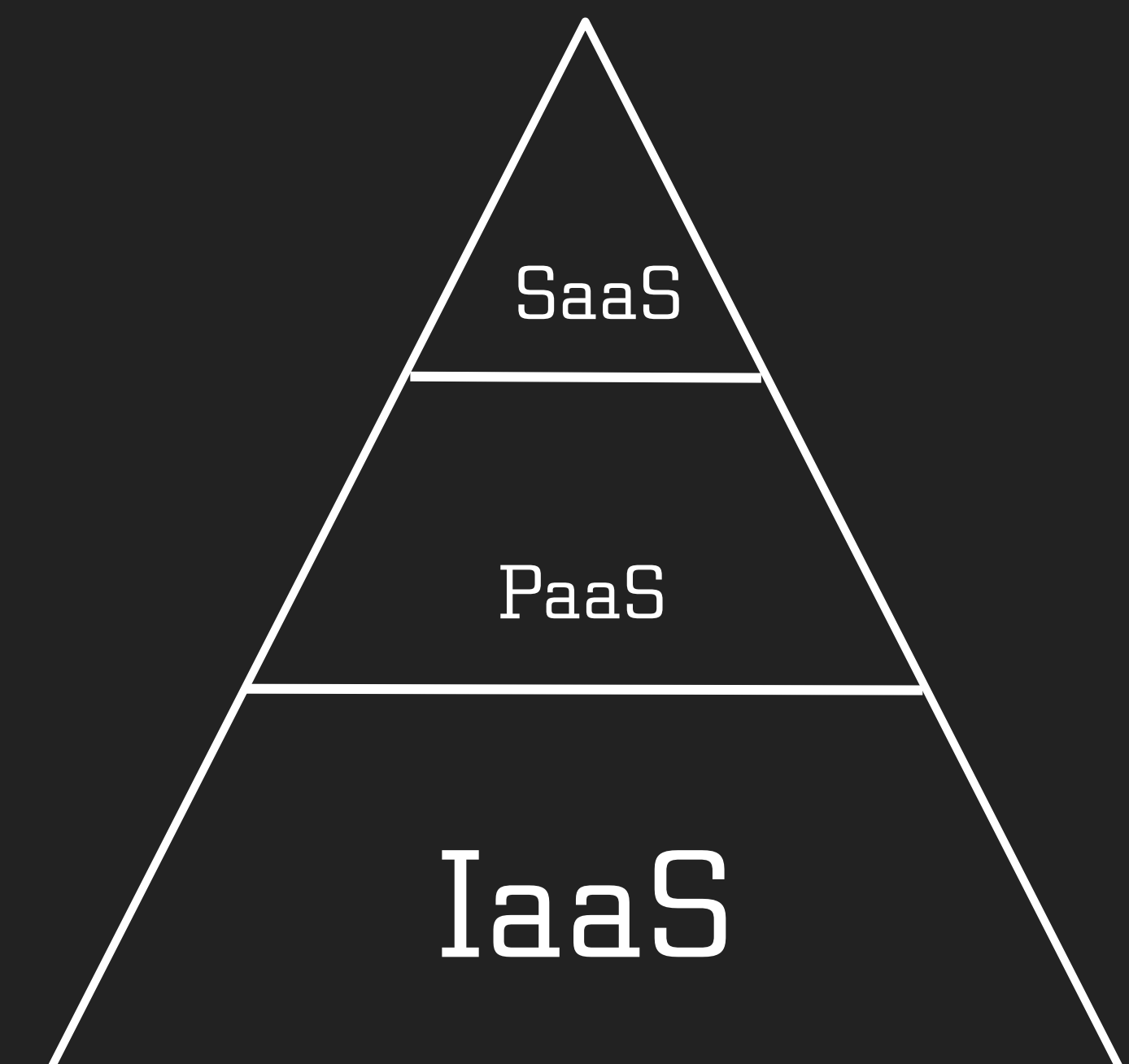
HPC

Object Storage

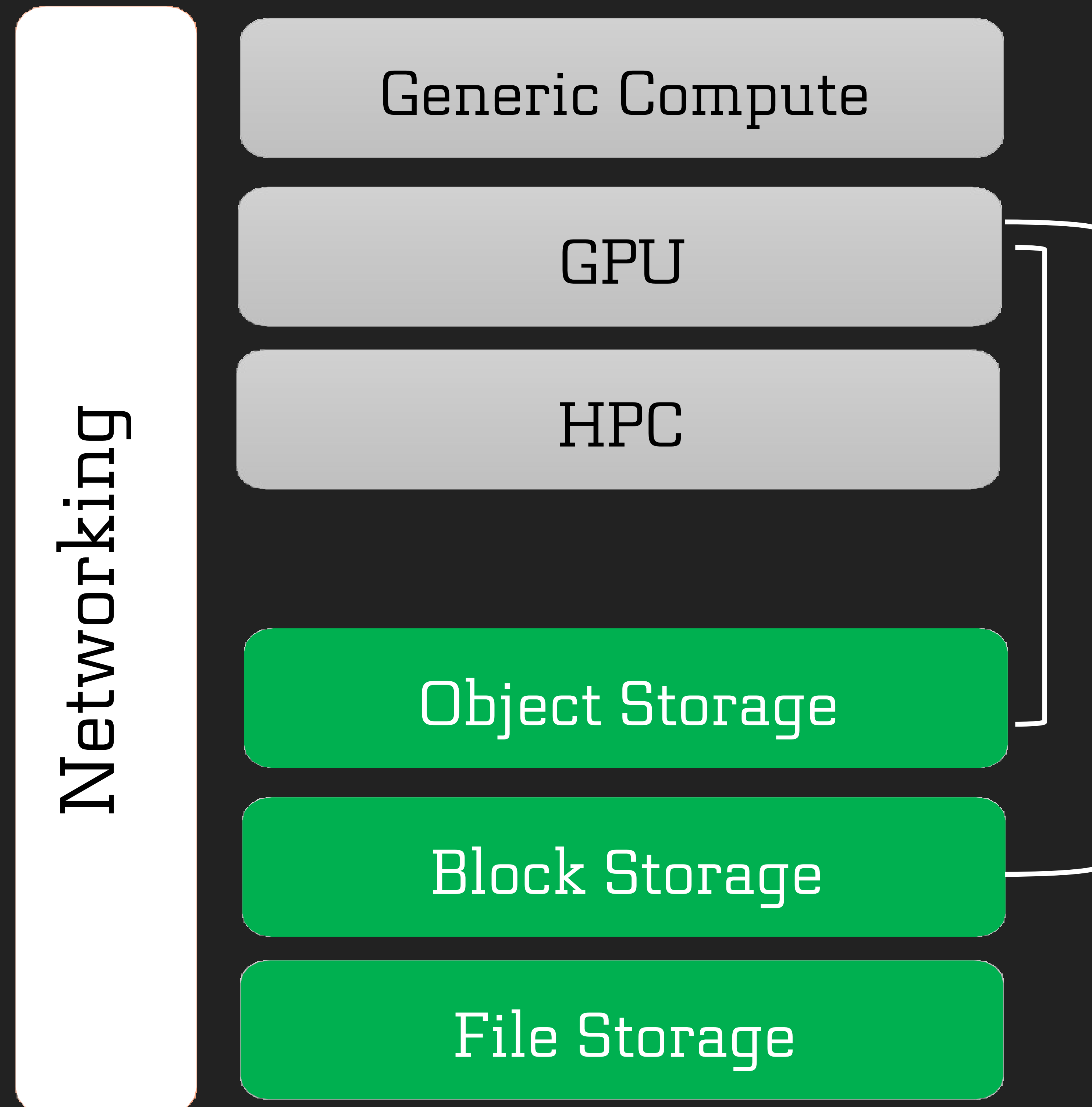
Block Storage

File Storage

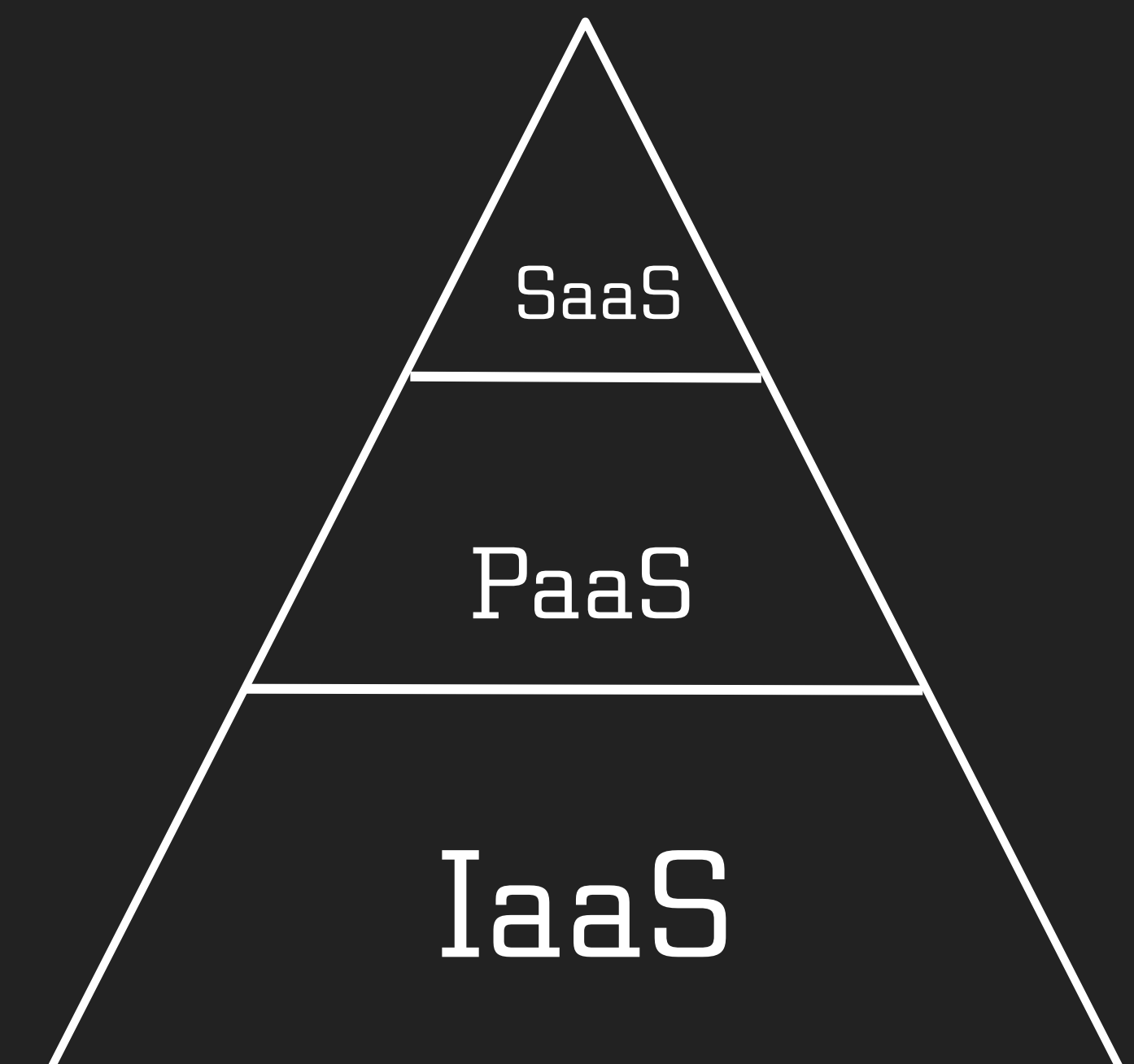
- ▷ Shared
- ▷ Hourly / Monthly
- ▷ No Contracts
- ▷ Self Service



IaaS – Ex. Image Processing



- ▷ Image Stored in Object Storage
- ▷ For Processing by different ML Algorithms Block Storage is used.
- ▷ Once the processing is done the result is stored back in Object storage or in DB
- ▷ Storages are attached using Networking Infrastructure.



Infrastructure as a Service (IaaS)

Infra as a Service

Applications

Data

Runtime

Middleware

OS

Virtualization

Servers

Storage

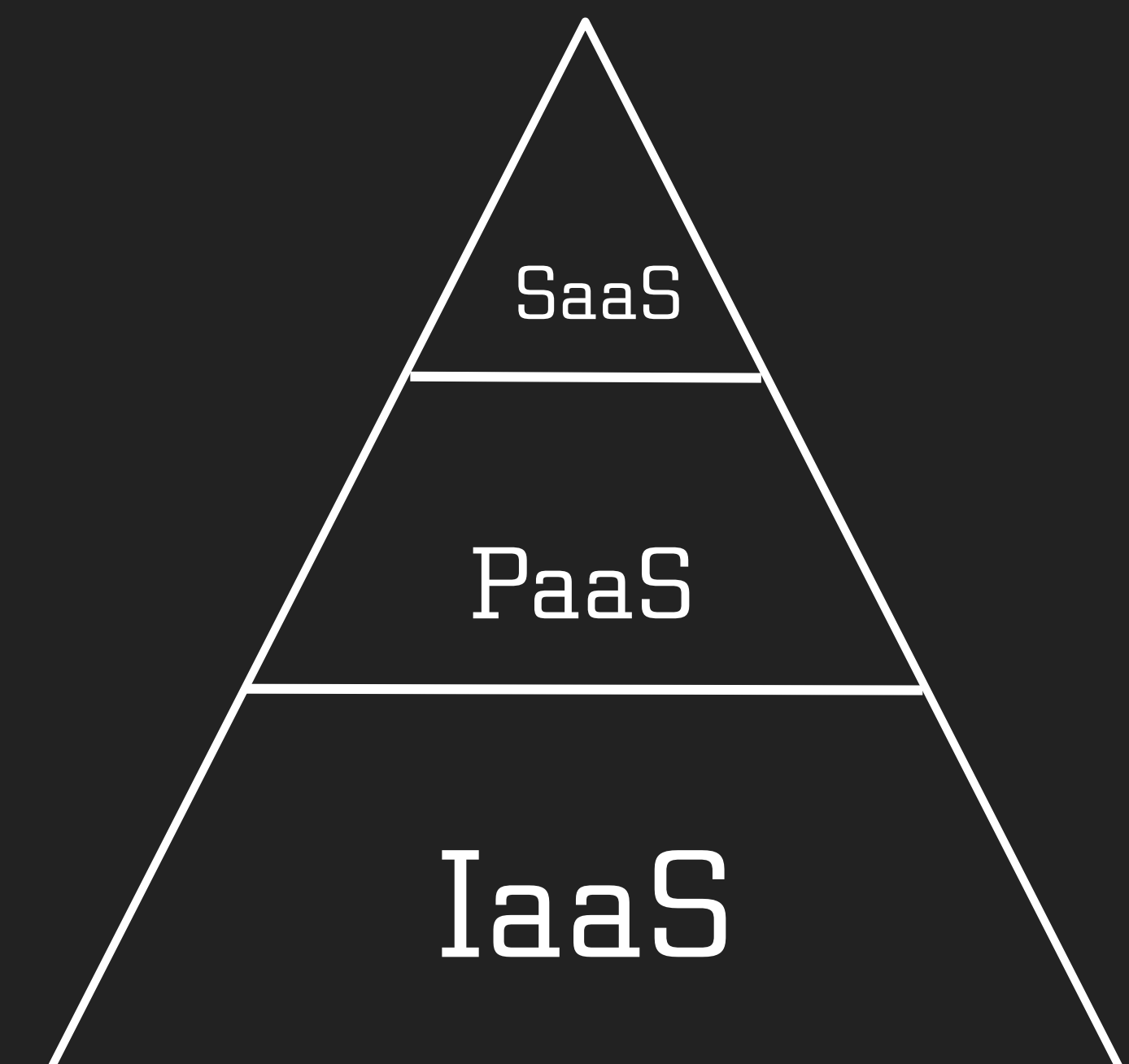
Networking

Pros

- ▶ Build your infra from Hours to minutes (From Weeks to Days in On-Premise Case)
- ▶ Pay per usage (As long as infra is running)
- ▶ Cost Effective compared to On-Premise upfront cost.

Cons

- ▶ Not stable than On-Premise Systems
- ▶ Vendor Lock In



Infrastructure as a Service (IaaS)

Infra as a Service

Applications

Data

Runtime

Middleware

OS

Virtualization

Servers

Storage

Networking

Examples

1. Amazon Web Services (AWS)

2. Microsoft Azure

3. Google Compute Engine (GCE)

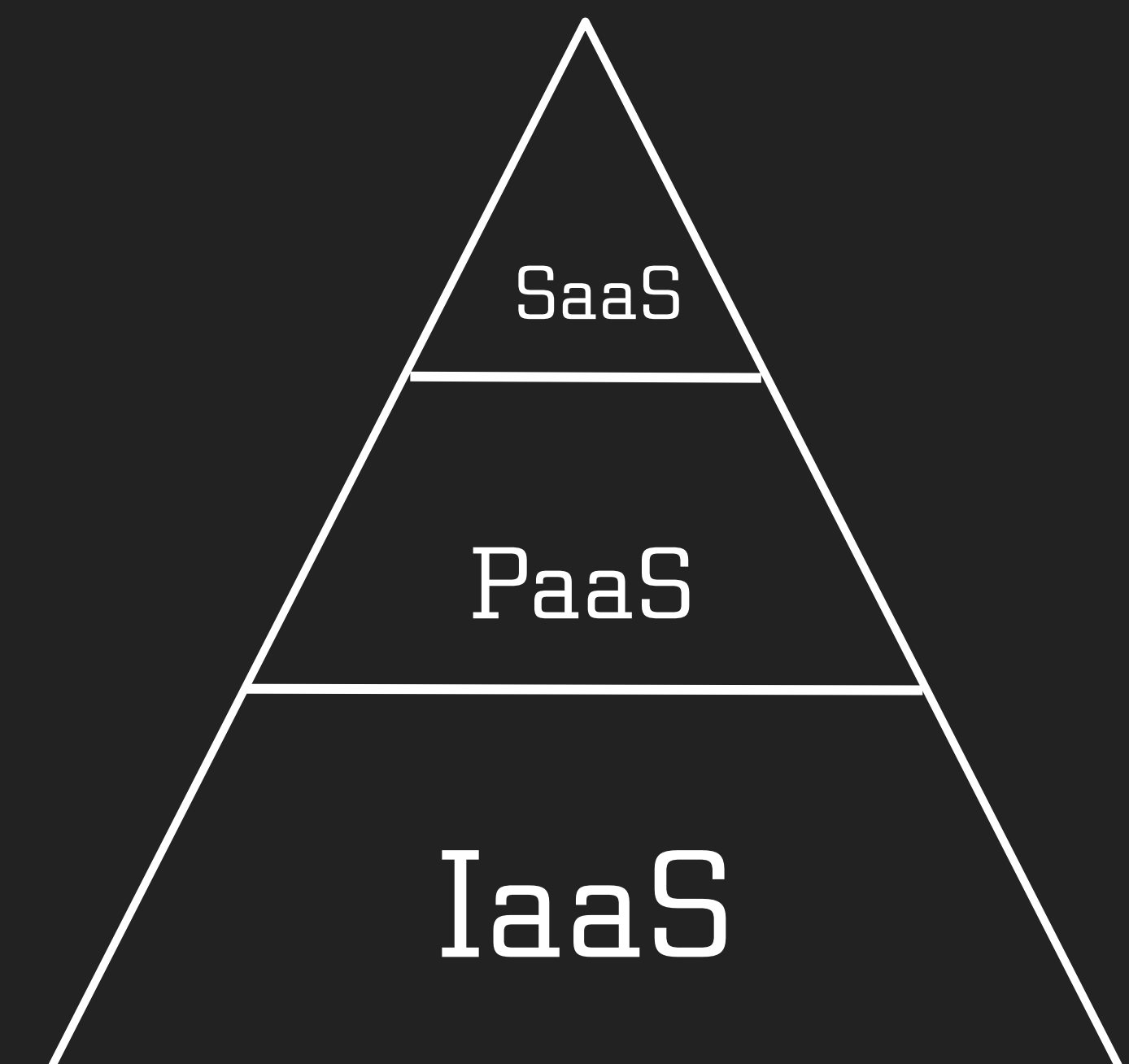
4. IBM Cloud

5. Cisco Metapod

6. Digital Ocean

7. Linode

8. Rackspace



Platform as a Service (PaaS)

Infra as a Service

Applications

Data

Runtime

Middleware

OS

Virtualization

Servers

Storage

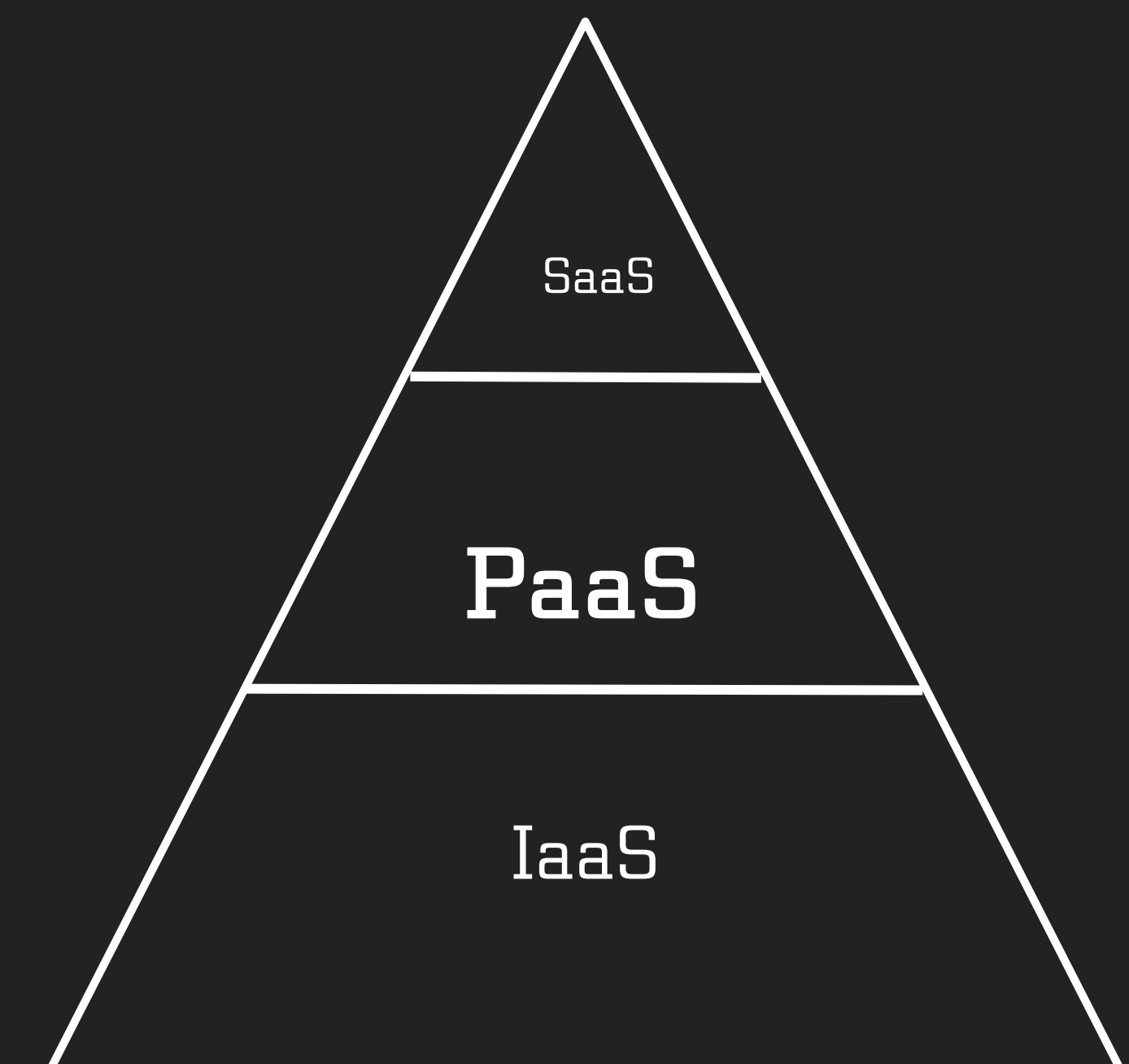
Networking

Pros

- ▶ Build your infra from Minutes to Seconds (From Days to Hours in On-Premise Case)
- ▶ Easy Deployment of Applications / Services
- ▶ You don't have to worry about Middleware's, Runtimes etc.

Cons

- ▶ Developers are constraint to PaaS capabilities.
- ▶ Vendor Lock In



Platform as a Service (PaaS)

Paas

Applications

Data

Runtime

Middleware

OS

Virtualization

Servers

Storage

Networking

Paas

Applications

Data

Container

K8s

OS

Virtualization

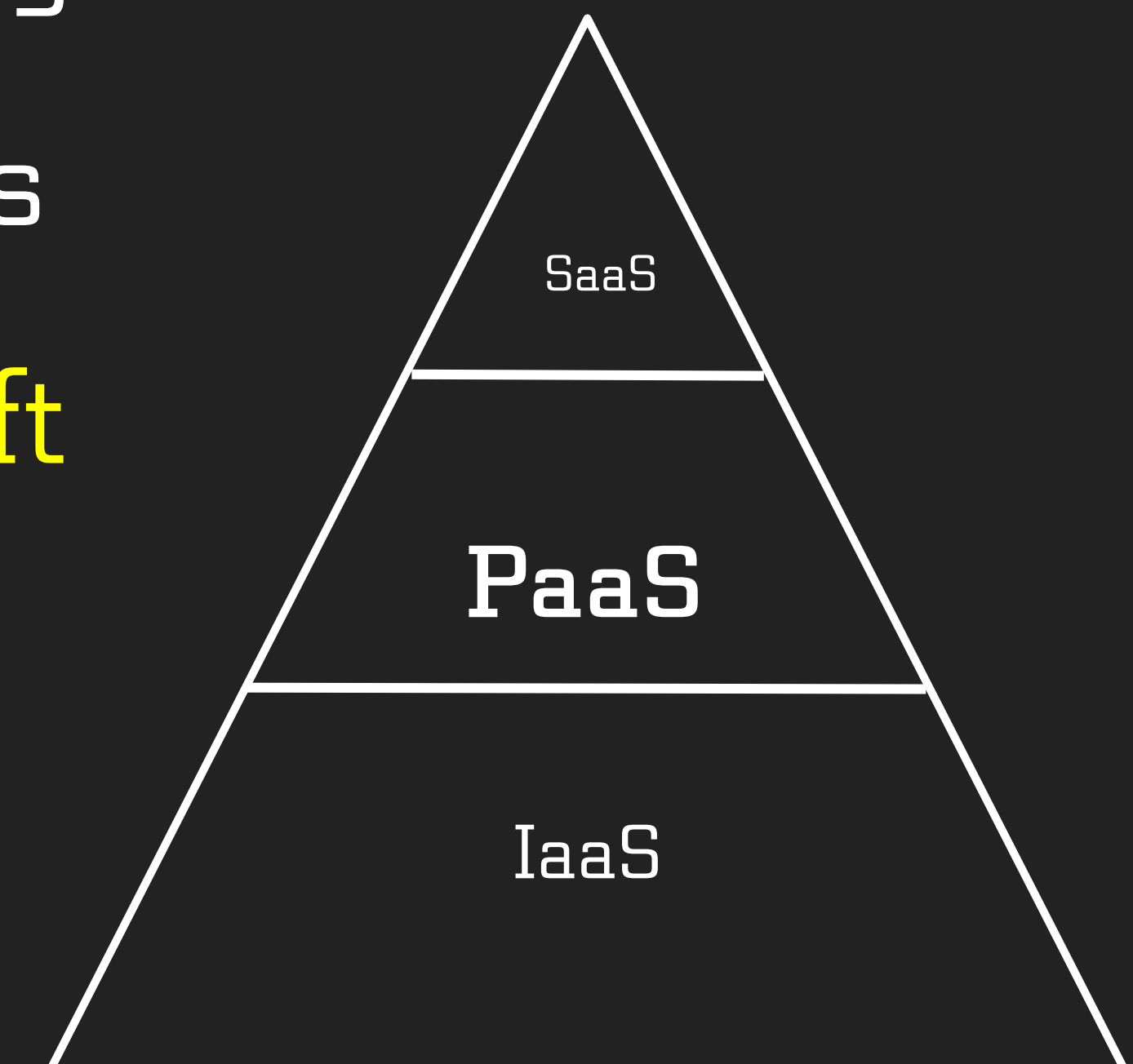
Servers

Storage

Networking

Examples

1. AWS Elastic Beanstalk
2. Windows Azure
3. Heroku
4. Force.com
5. Google App Engine
6. Apache Stratos
7. K8s / OpenShift



Software as a Service (SaaS)

Software as a Service

Applications

Data

Runtime

Middleware

OS

Virtualization

Servers

Storage

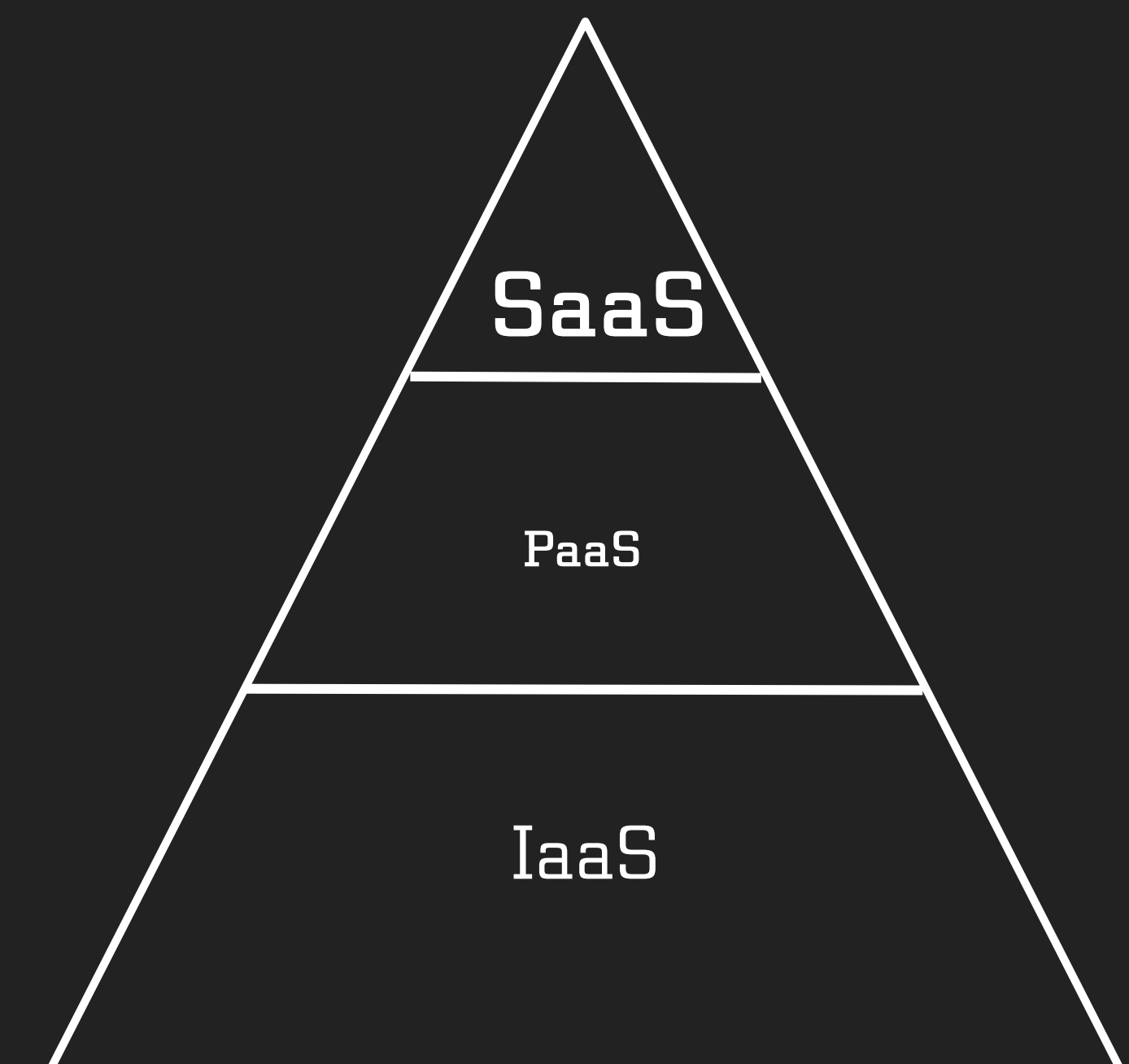
Networking

Pros

- ▶ 1. No Installation. Get Started asap. Multi Tenant
- ▶ 2. Accessible from Anywhere / Any Device
- ▶ 3. Excellent for Collaborative working
- ▶ 4. Cost Effective & Pay Per Usage

Cons

- ▶ 1. Vendor Lock In.
- ▶ 2. Compliance Restrictions
- ▶ 3. Depends on Internet
- ❑ Speed for Performance



Software as a Service (SaaS)

Software as a Service

Applications

Data

Runtime

Middleware

OS

Virtualization

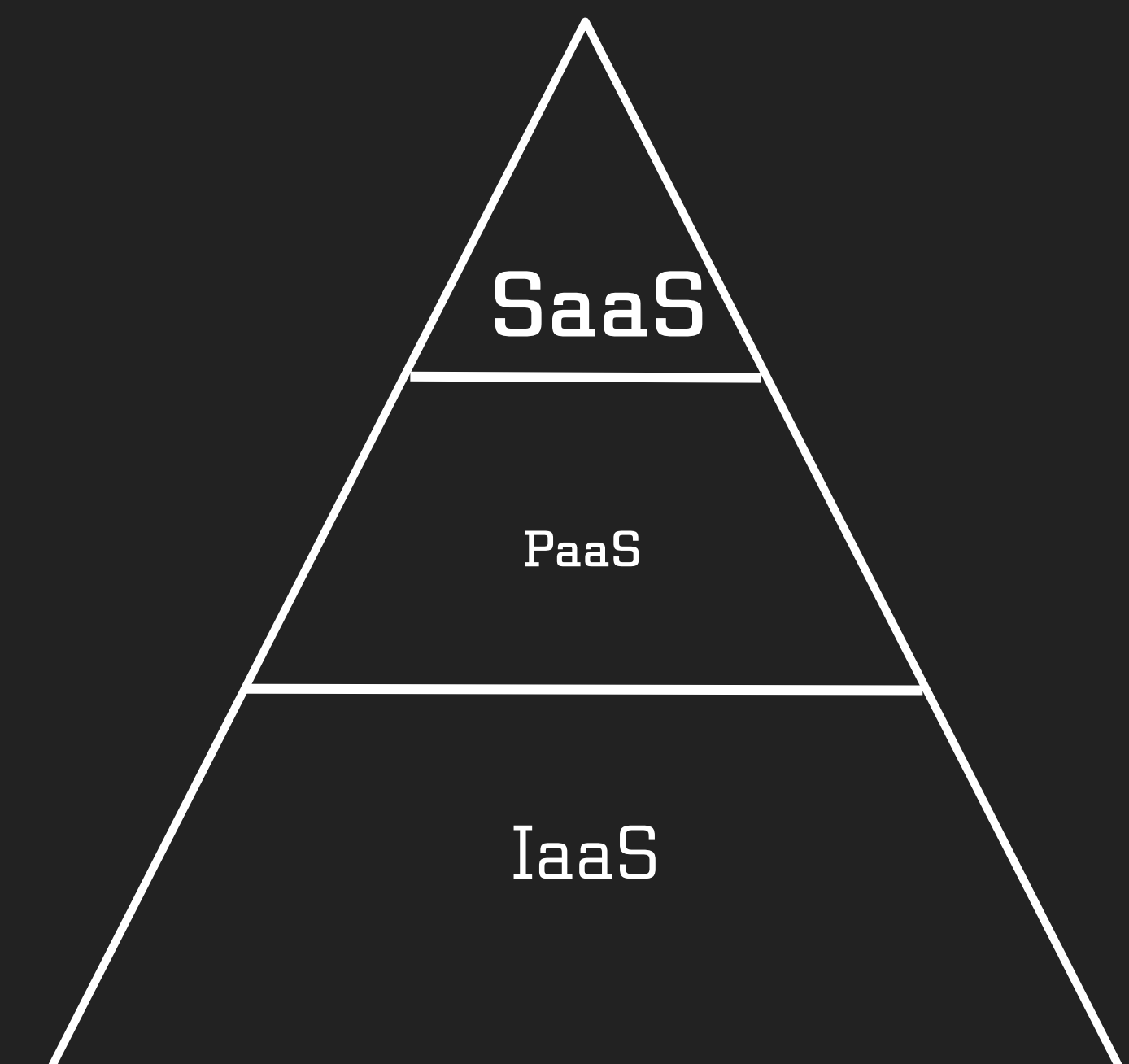
Servers

Storage

Networking

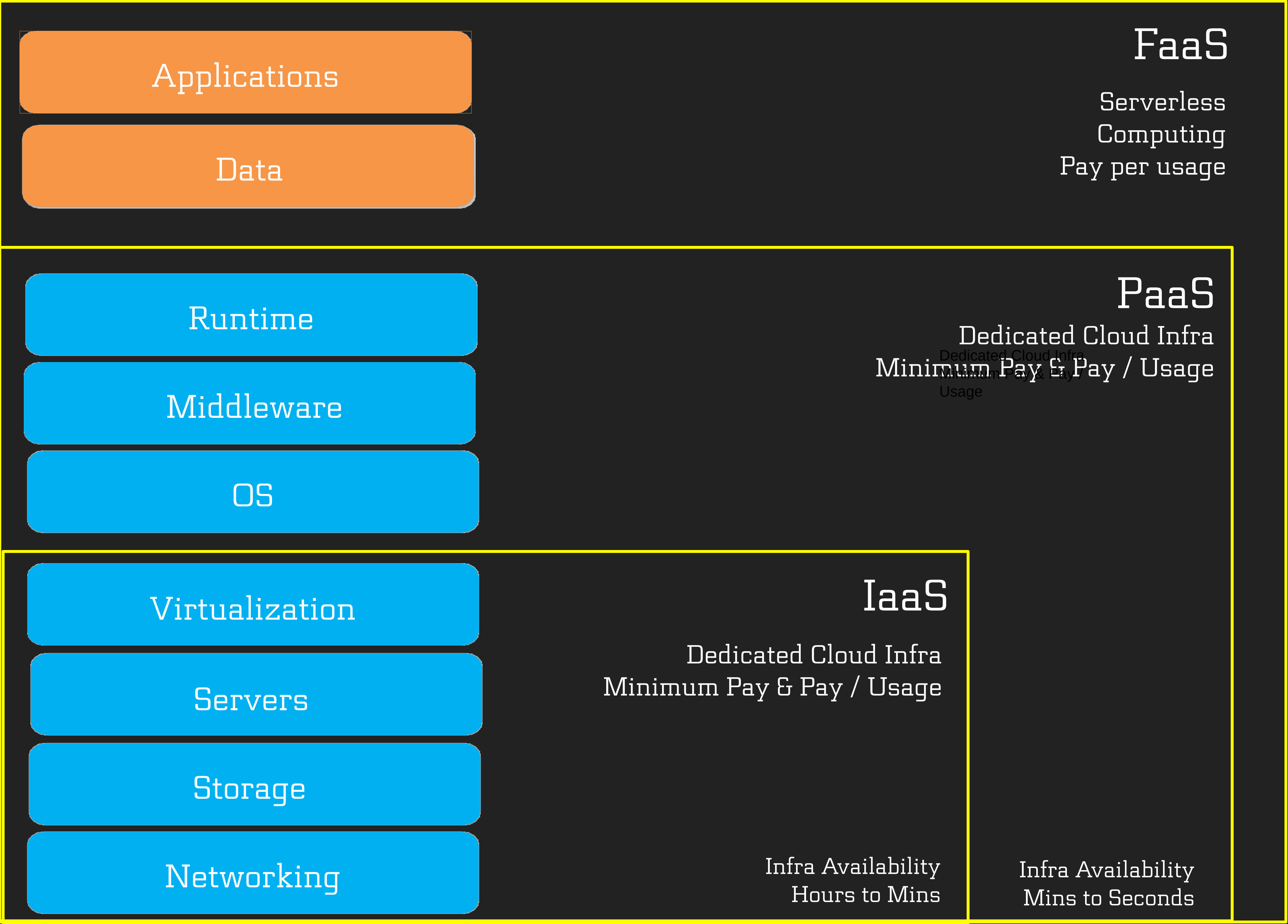
Examples

- ▷ 1. Google Workspace
- ▷ 2. Dropbox
- ▷ 3. Salesforce
- ▷ 4. Cisco WebEx
- ▷ 5. Concur
- ▷ 6. GoToMeeting
- ▷ 7. Google Gmail

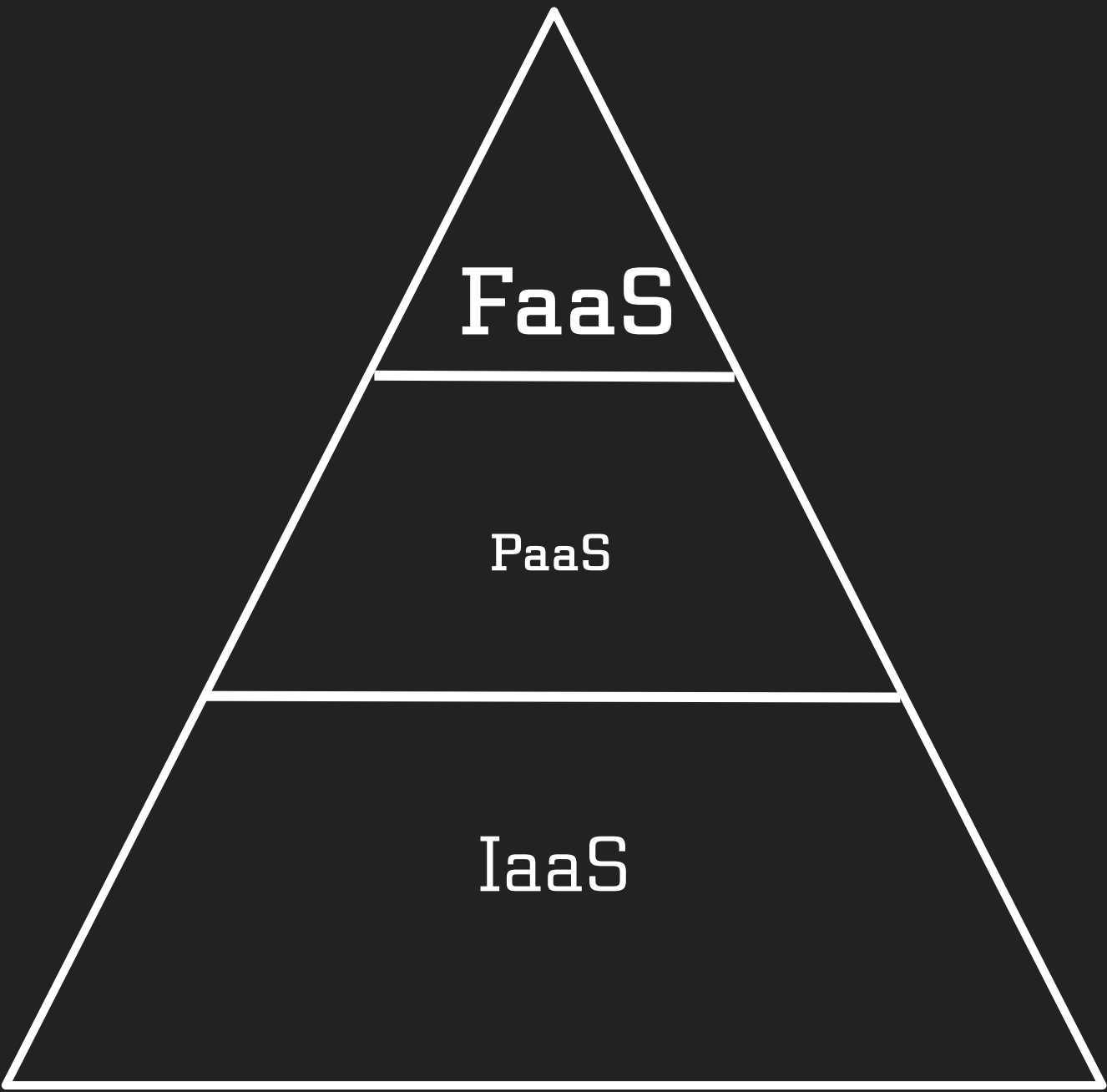


Function as a Service (FaaS)

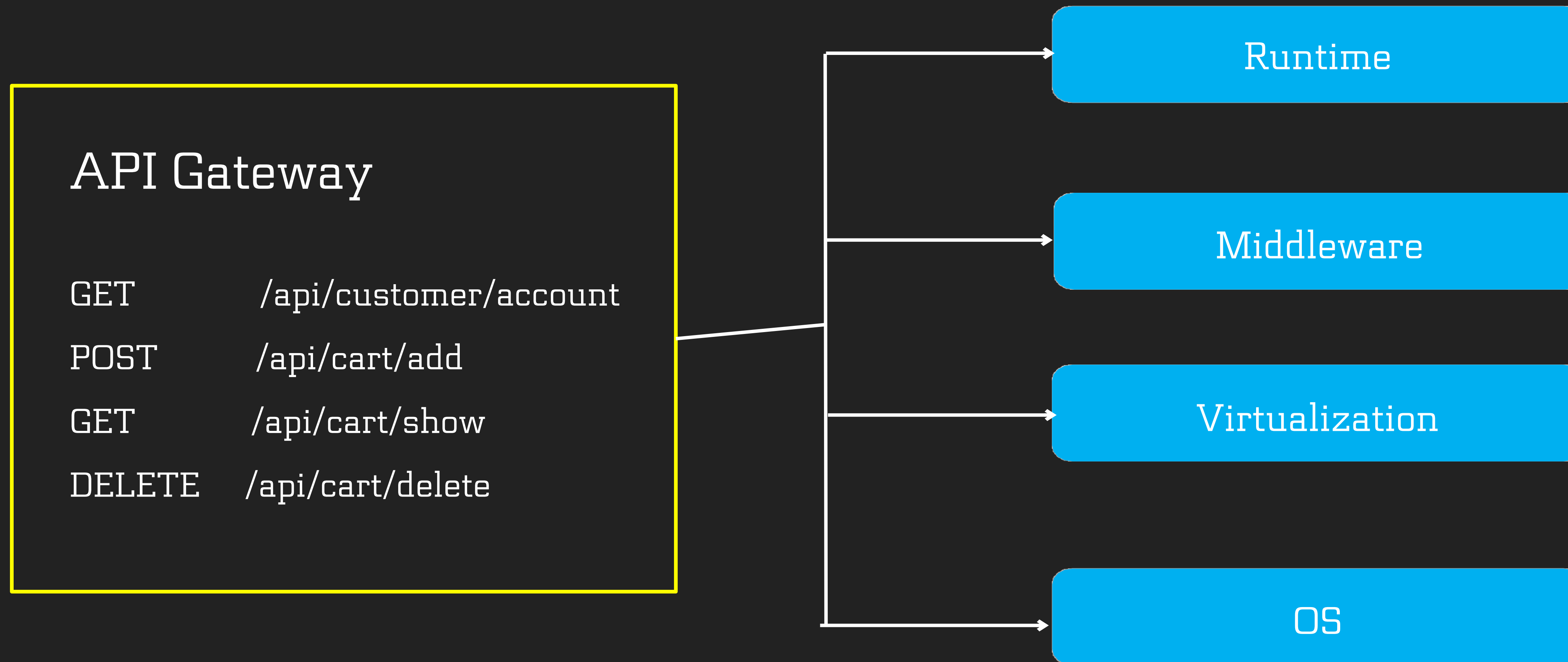
Application Deployed as small stateless functions



- ▶ Zero Hardware
- ▶ Scale on Demand
- ▶ Auto Scale to Zero
- ▶ Cost Effective



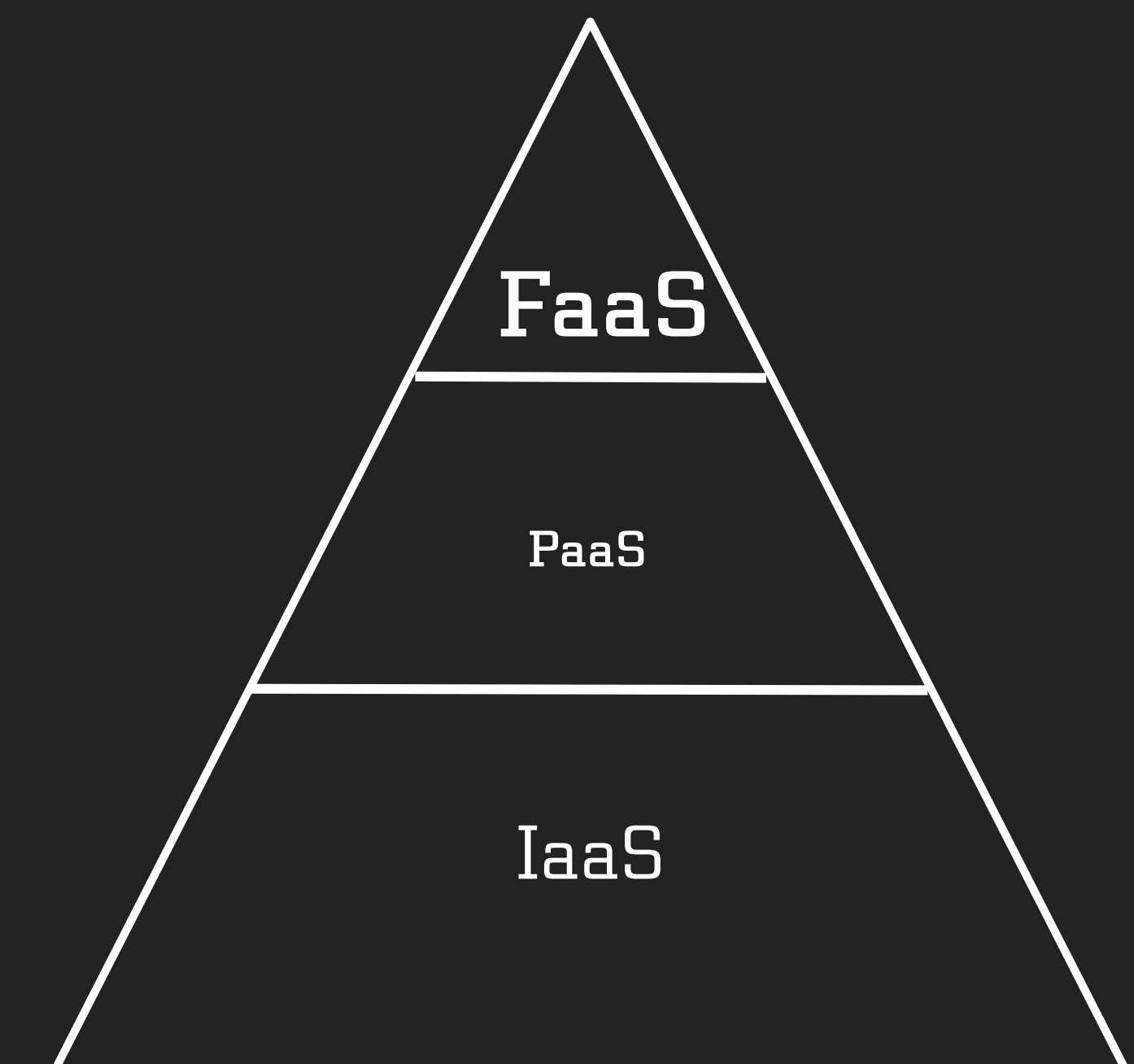
Function as a Service (FaaS)



Api Gateway Route the request to the function.

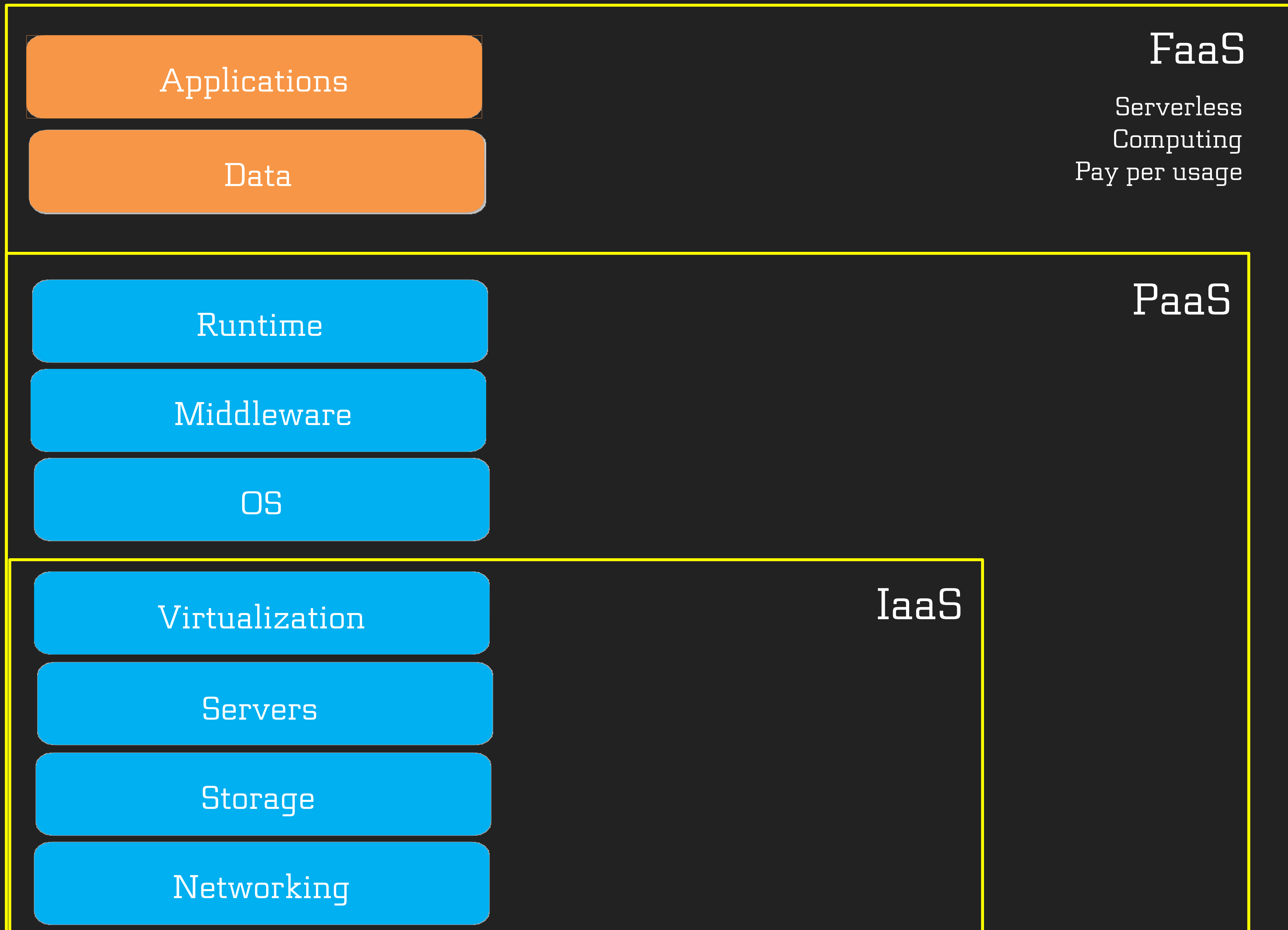
Function is terminated once the request is processed.

In Kubernetes Knative the function will wait for few seconds before it gets terminated to see if there are any pending requests to be processed.

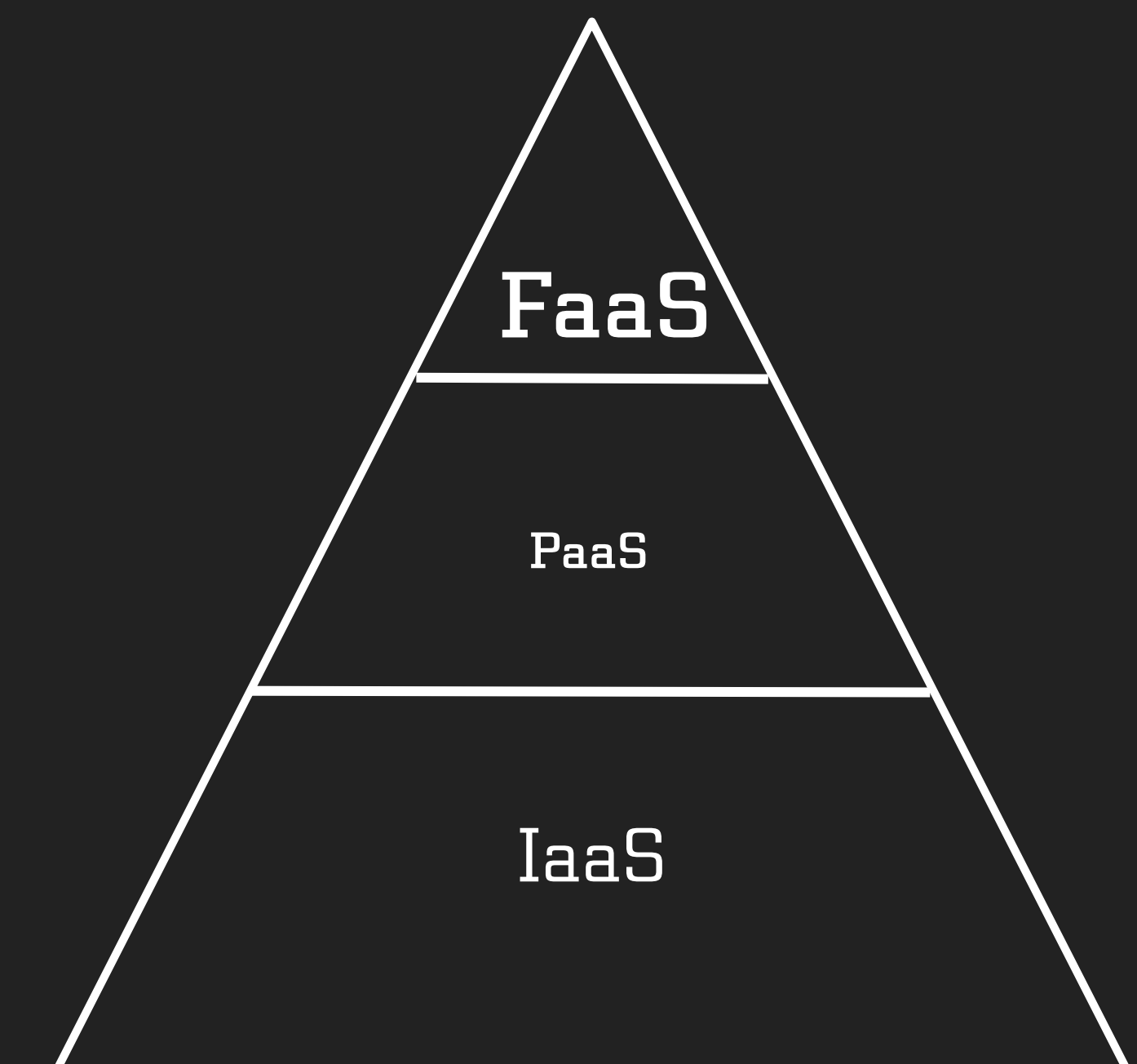


Function as a Service (FaaS)

Application Deployed as small stateless functions

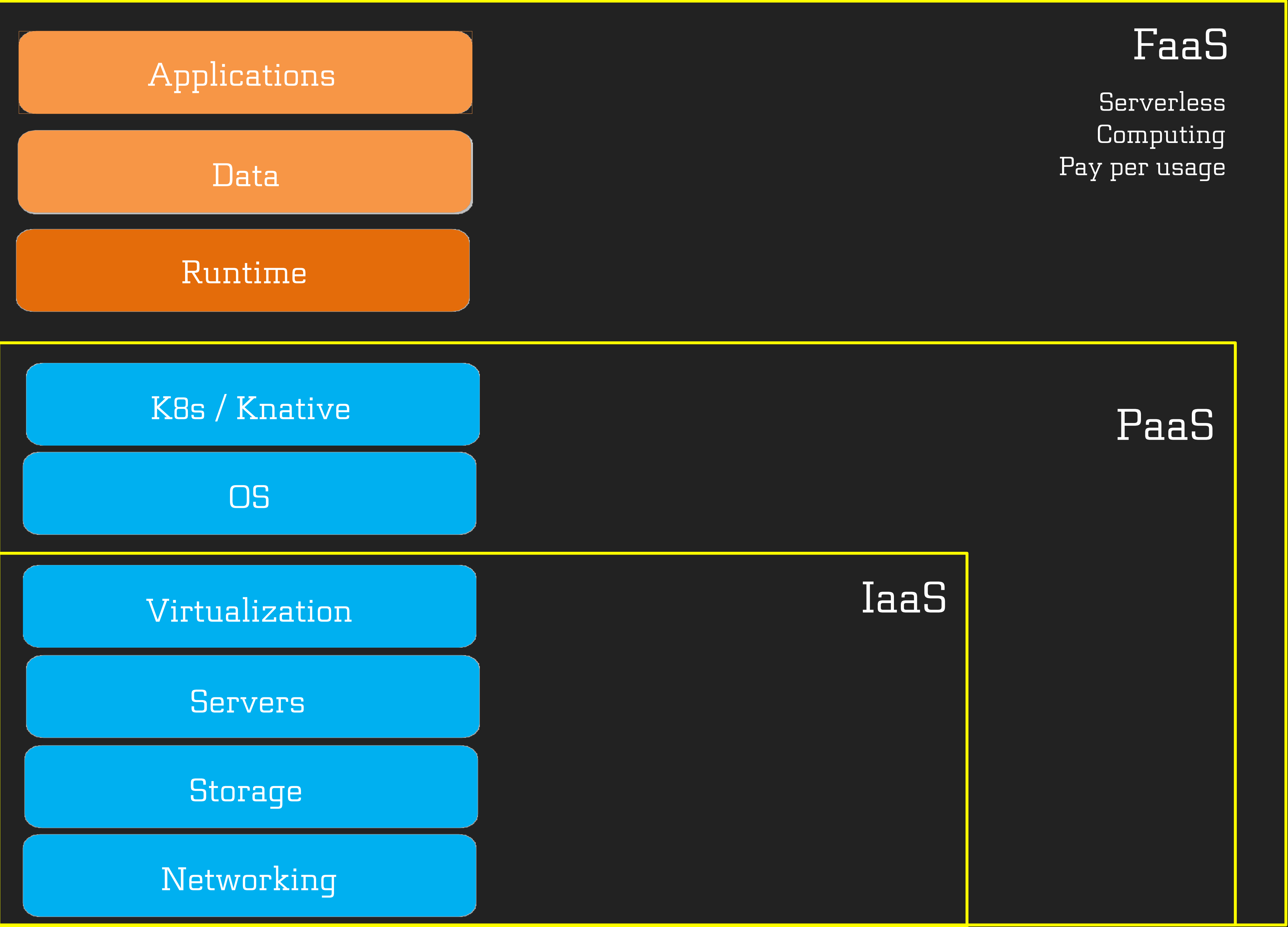


- ▷ Amazon Lambda
- ▷ Azure Functions
- ▷ Google Run (Knative)



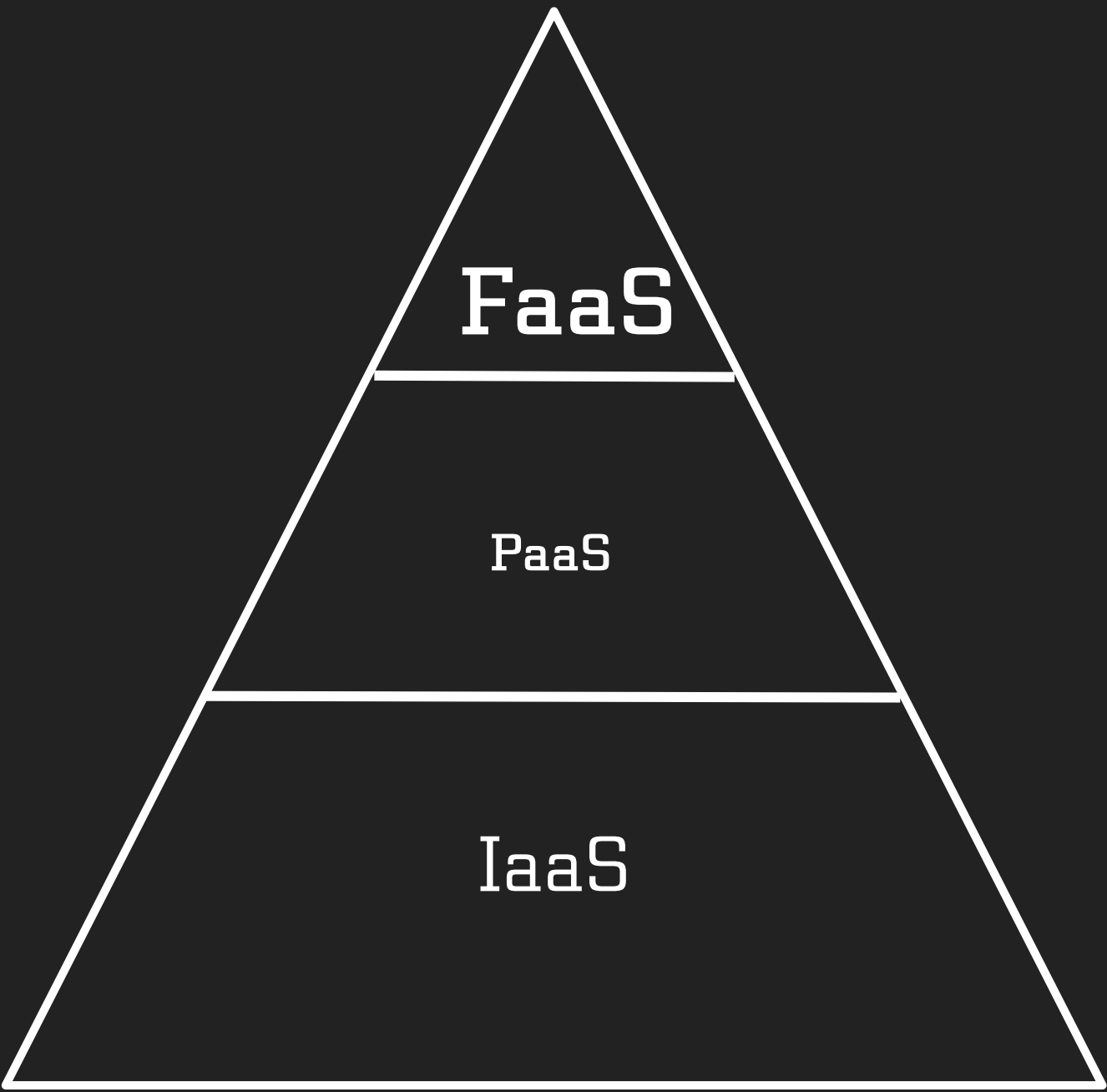
Function as a Service (FaaS)

Application Deployed as small stateless functions

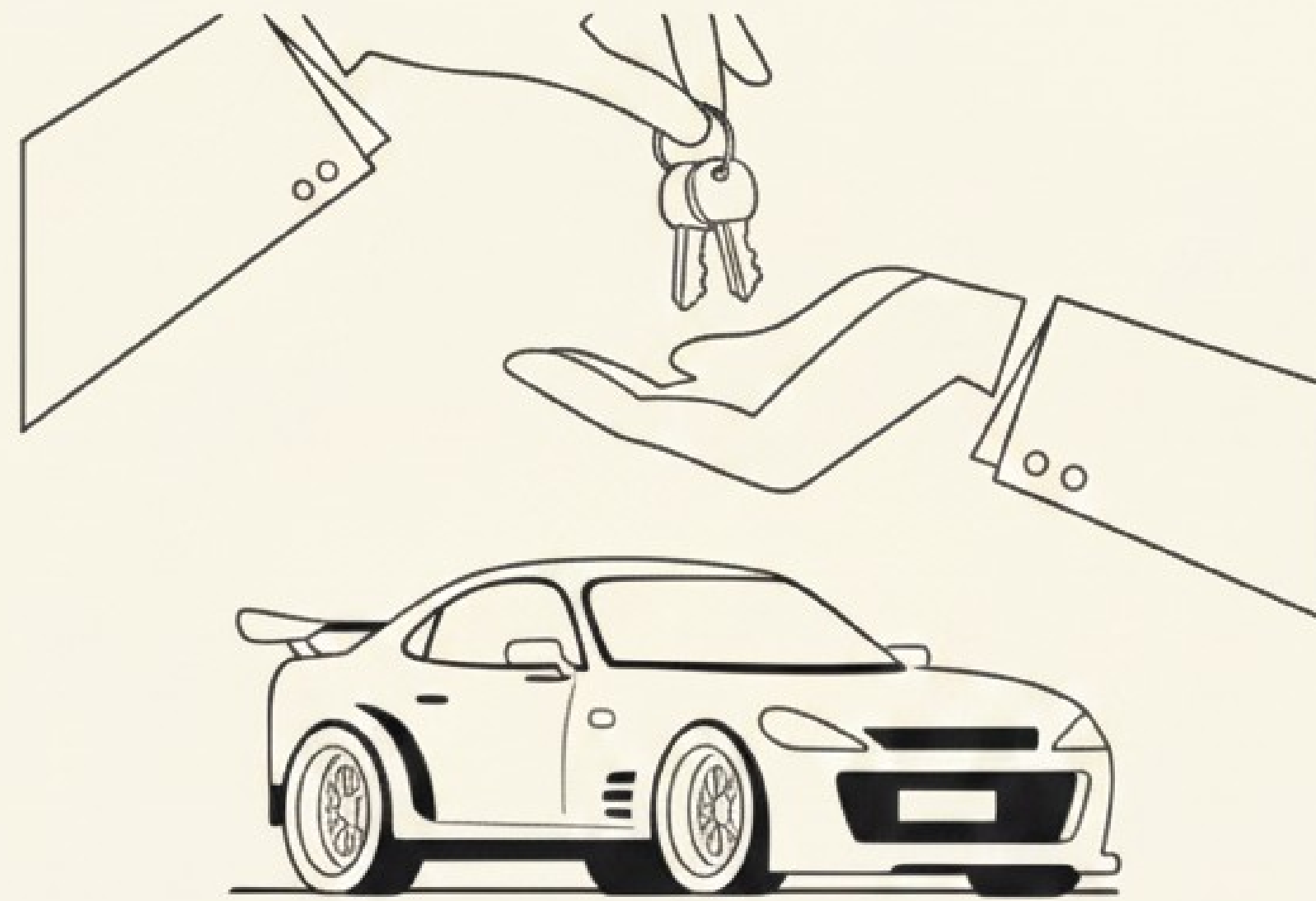


▶ Kubernetes Knative

Kubernetes Knative is Cloud agnostic FaaS. So, No Vendor Lock-in.



IaaS / PaaS / SaaS



IaaS: Lease a Car

You can decide what type of infrastructure you need for long-term usage of infrastructure. More flexibility brings in more complexity.



PaaS: Rent a Car

You choose what's available and pay as per your usage.



SaaS: Taxi Car - Uber

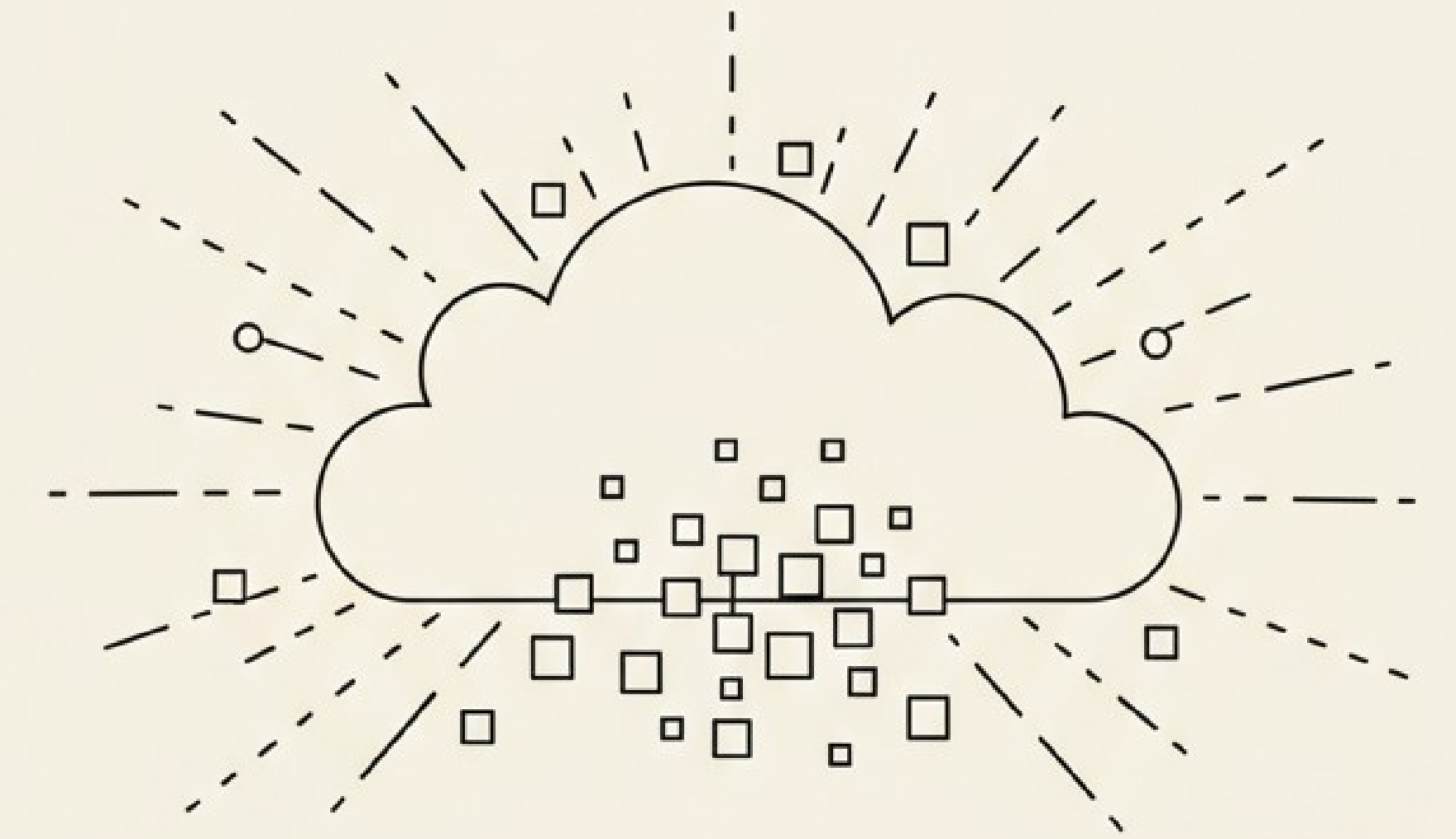
You don't decide what type of car you want – you use it to accomplish the task and pay as per your usage.

Summary of X as a Service

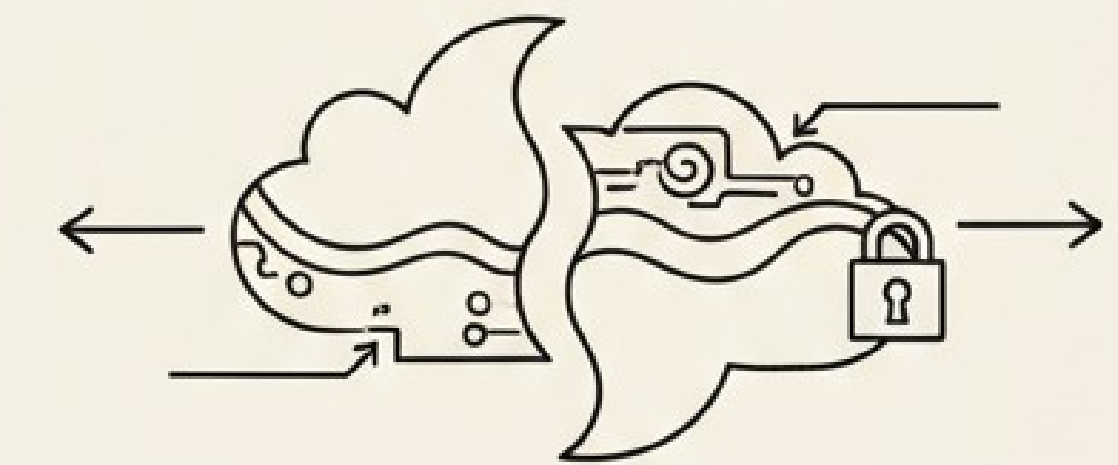
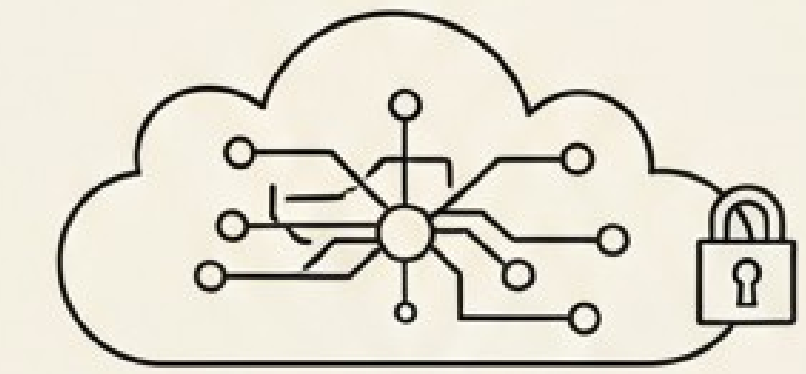
On Premise	IaaS	PaaS	PaaS	FaaS	SaaS
Applications	Applications	Applications	Applications	Applications	Applications
Data	Data	Data	Data	Data	Data
Runtime	Runtime	Runtime	Container	Container	Runtime
Middleware	Middleware	Middleware	K8s	K8s / Knative	Middleware
OS	OS	OS	OS	OS	OS
Virtualization	Virtualization	Virtualization	Virtualization	Virtualization	Virtualization
Servers	Servers	Servers	Servers	Servers	Servers
Storage	Storage	Storage	Storage	Storage	Storage
Networking	Networking	Networking	Networking	Networking	Networking
Customer Managed	Cloud Provider Managed				

Cloud Types

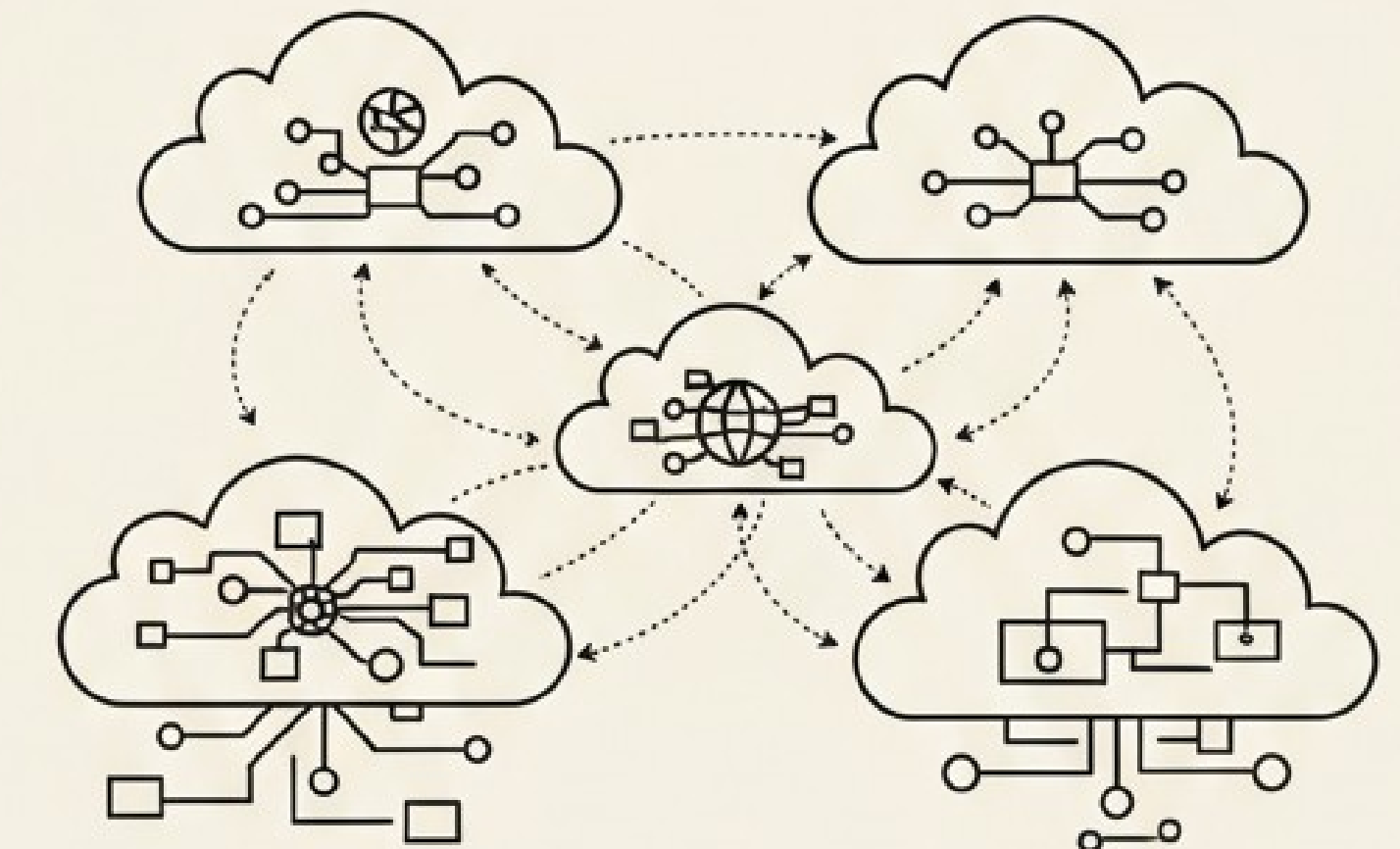
Public Cloud



Private Cloud

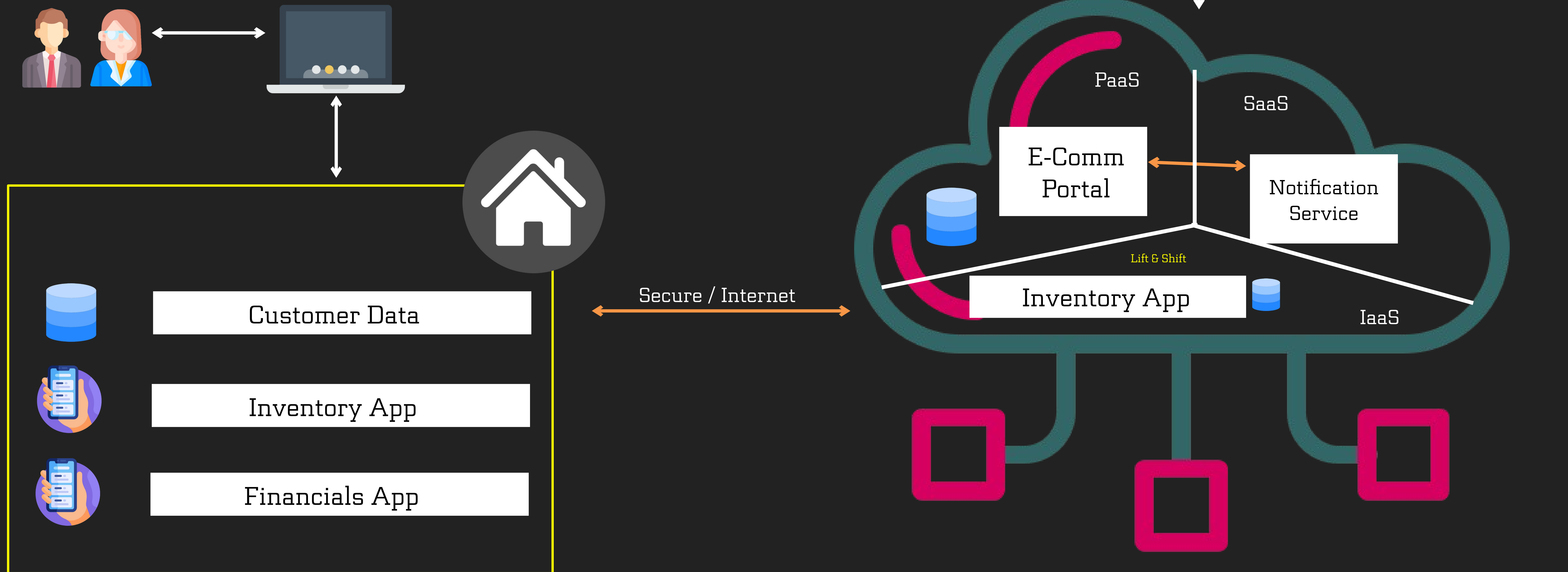


Hybrid Cloud



Multi-Cloud

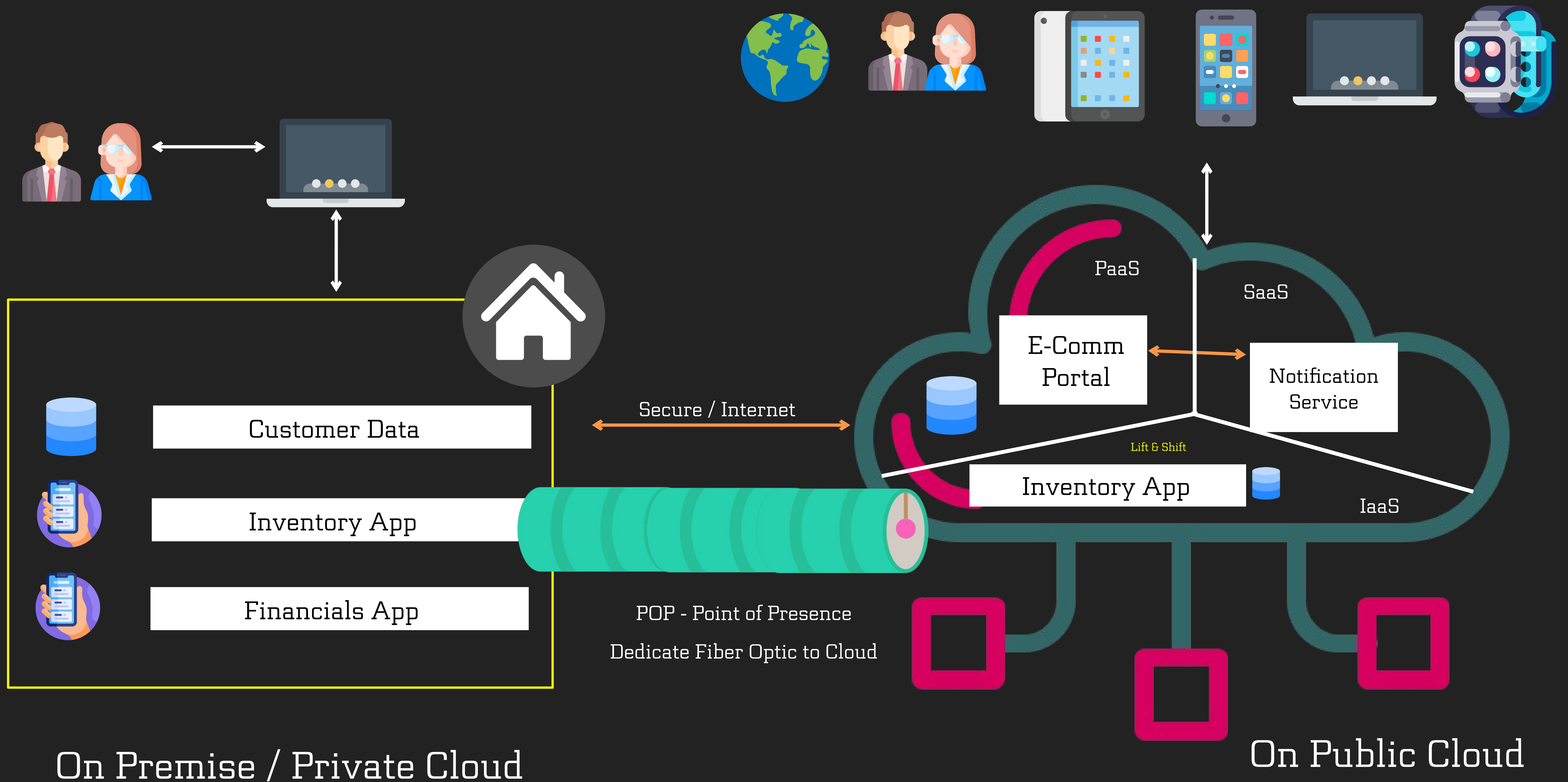
Hybrid Cloud



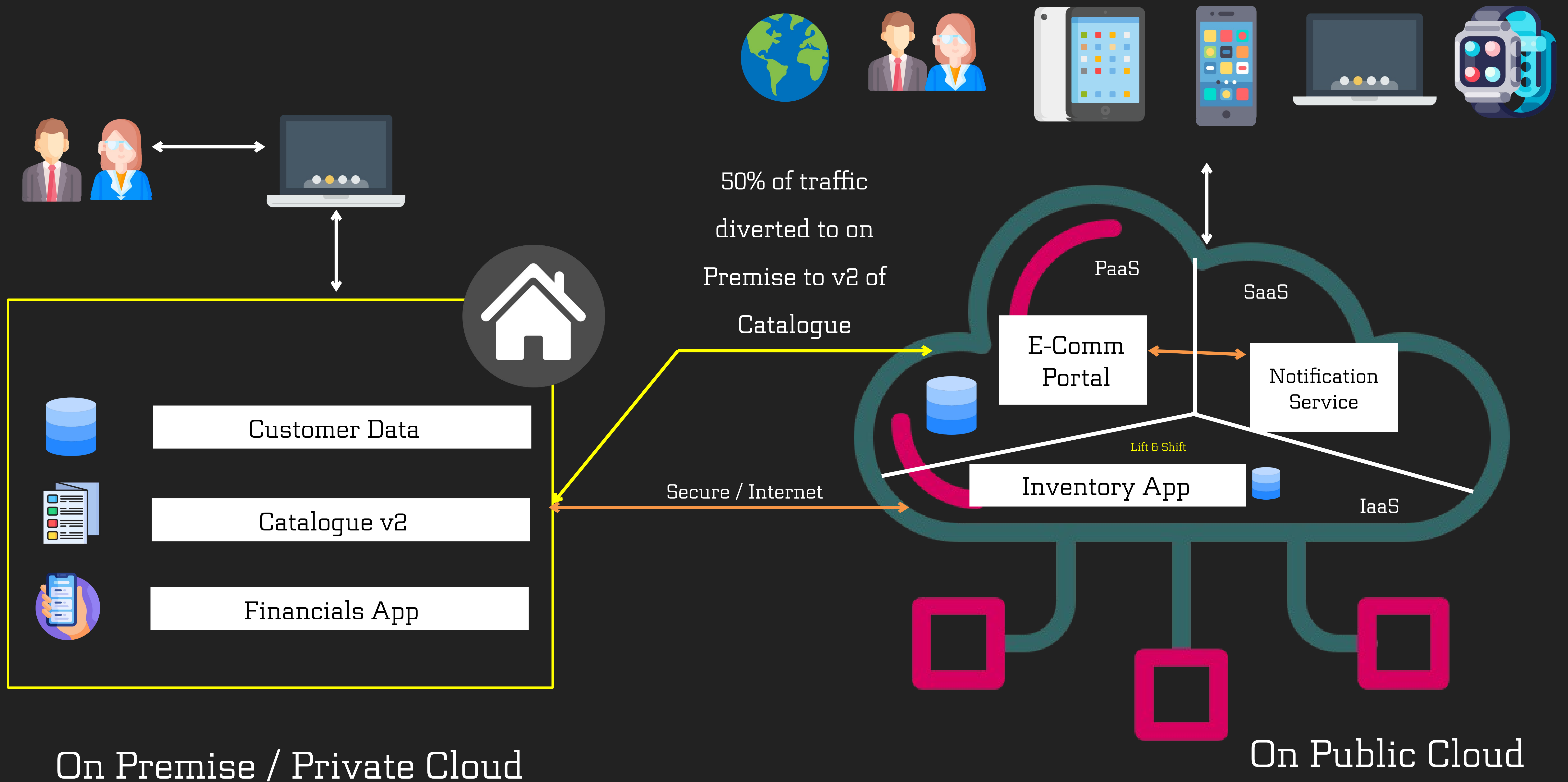
On Premise / Private Cloud

On Public Cloud

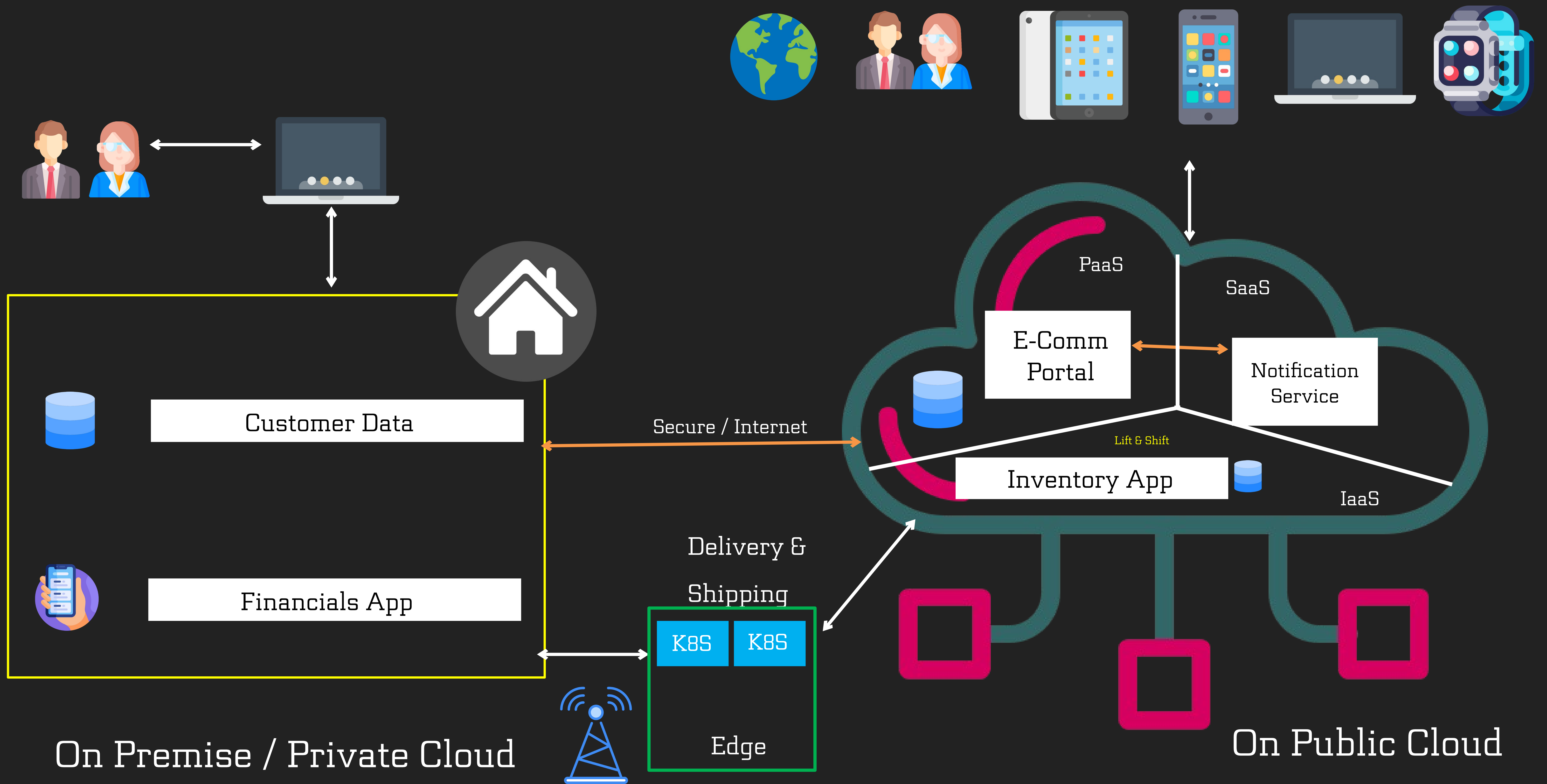
Hybrid Cloud – Connectivity



Hybrid Cloud – Service Mesh

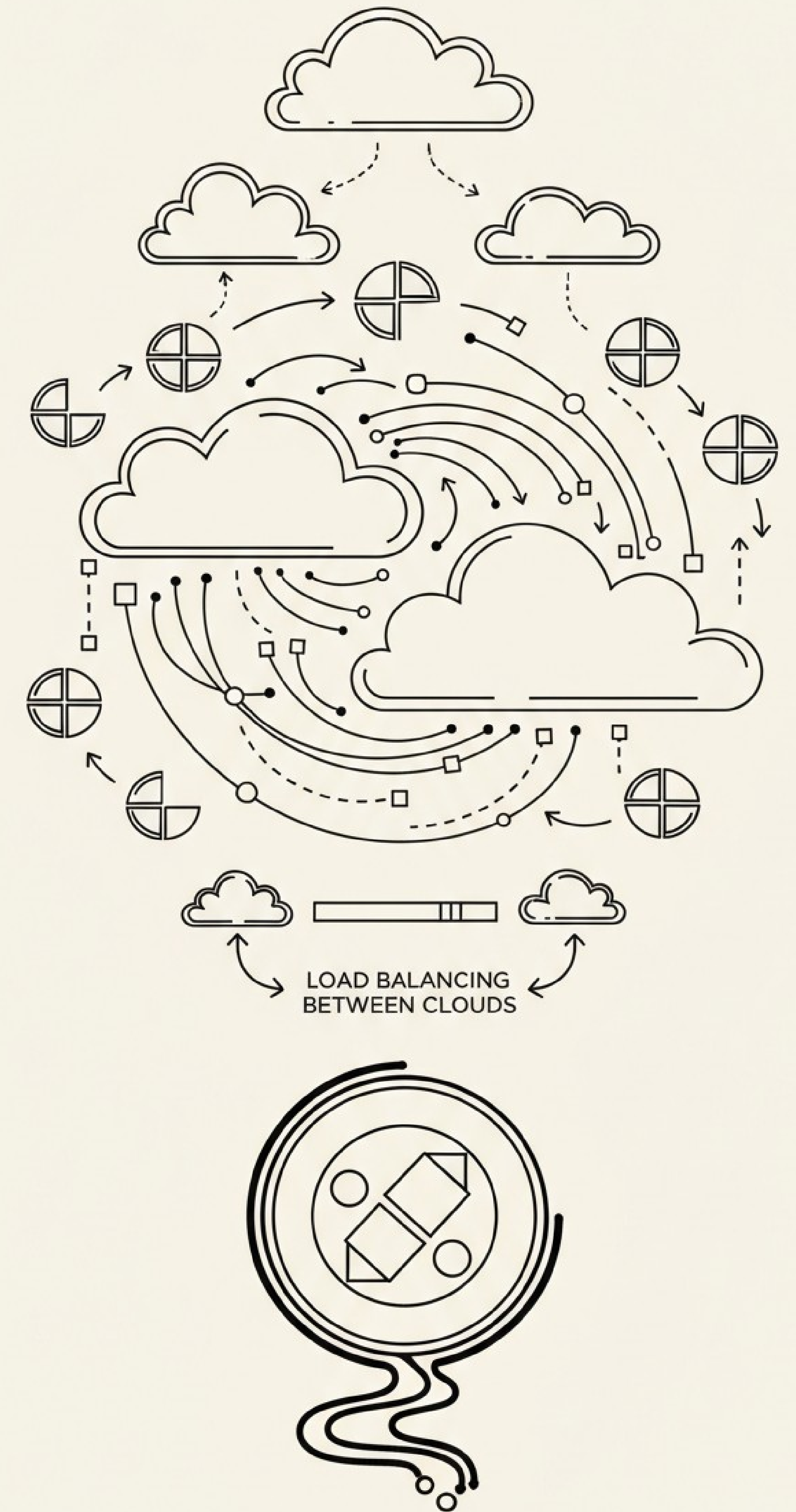


Hybrid Cloud with Edge



Multi Cloud

Distributed



Distributed Cloud / Multi Cloud

DevOps

Consistent Environment

Auto Scale Across Env

K8s

Cloud

K8s

On-Prem

K8s

Edge

DevOps

Portable Workloads

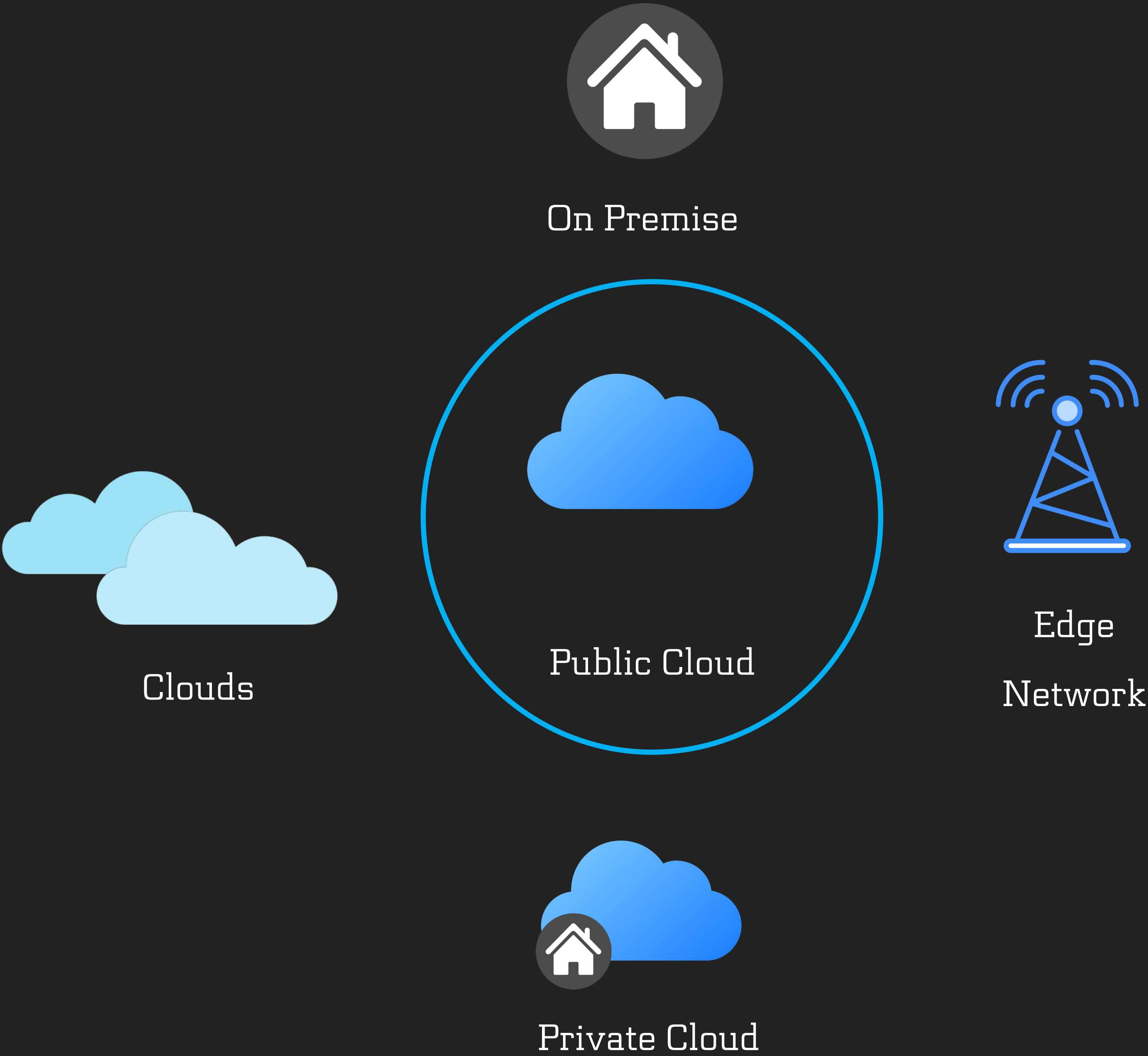
AI/ML Across All Environment

DevSecOps

Governance

Security Policies

Network Policies



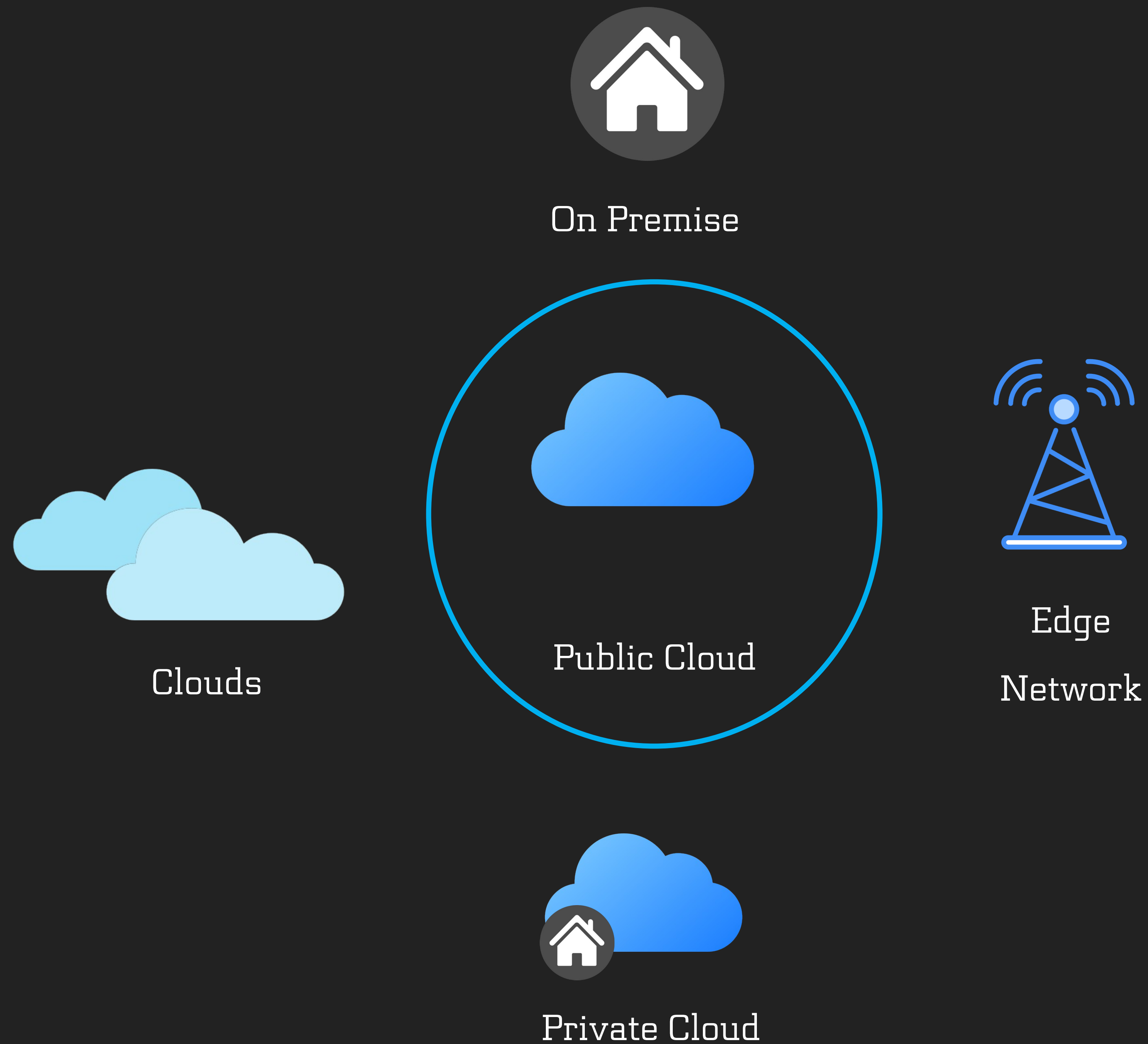
Technology Stack

Containers

Kubernetes

Service Mesh

Distributed Cloud / Multi Cloud



Examples

Google Anthos

Microsoft Azure Arc

Pros

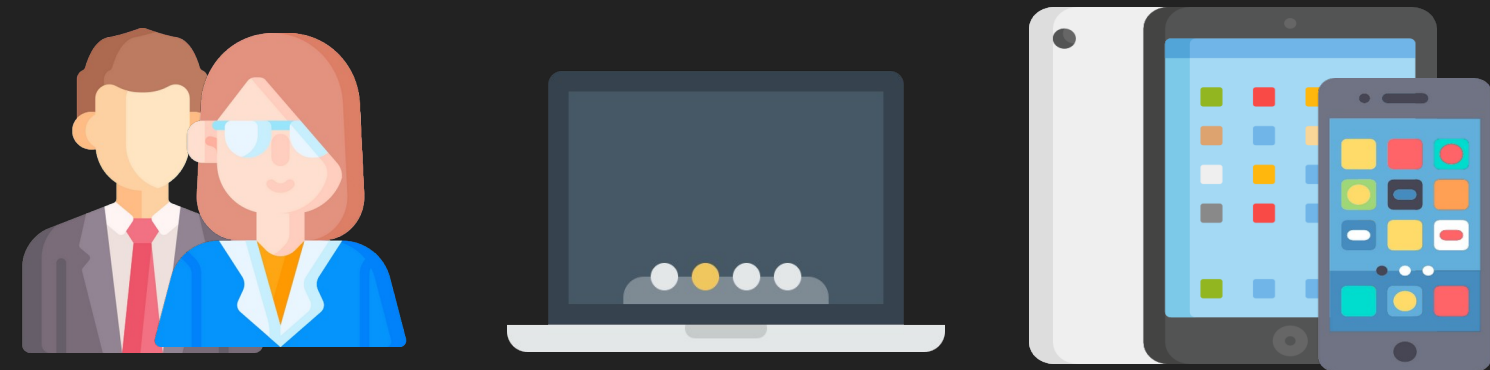
Geography Proximity

Availability Across Clouds

Seamless Communication

Multi cluster

Single cloud



K8s Cluster



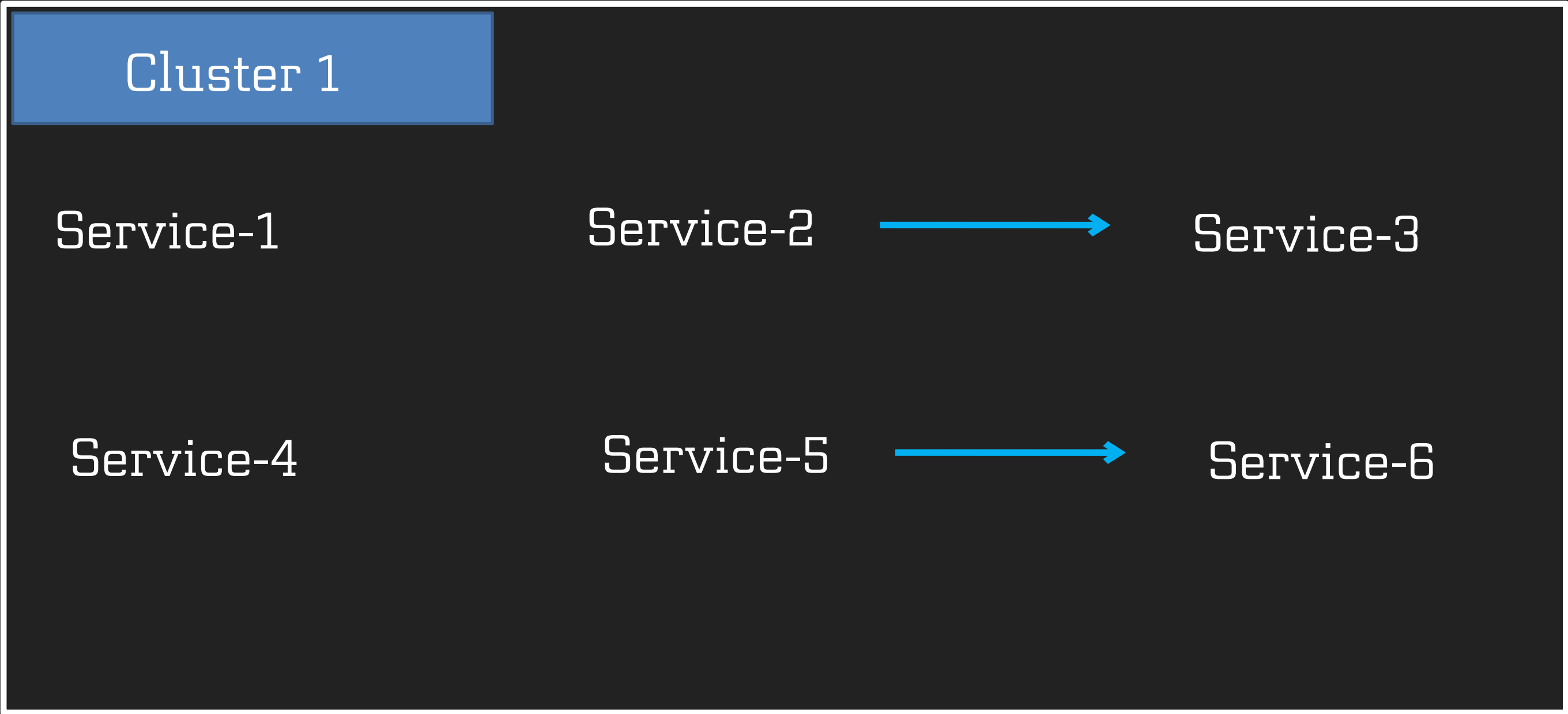
North-South
Communication

Gateway and Routing Rules



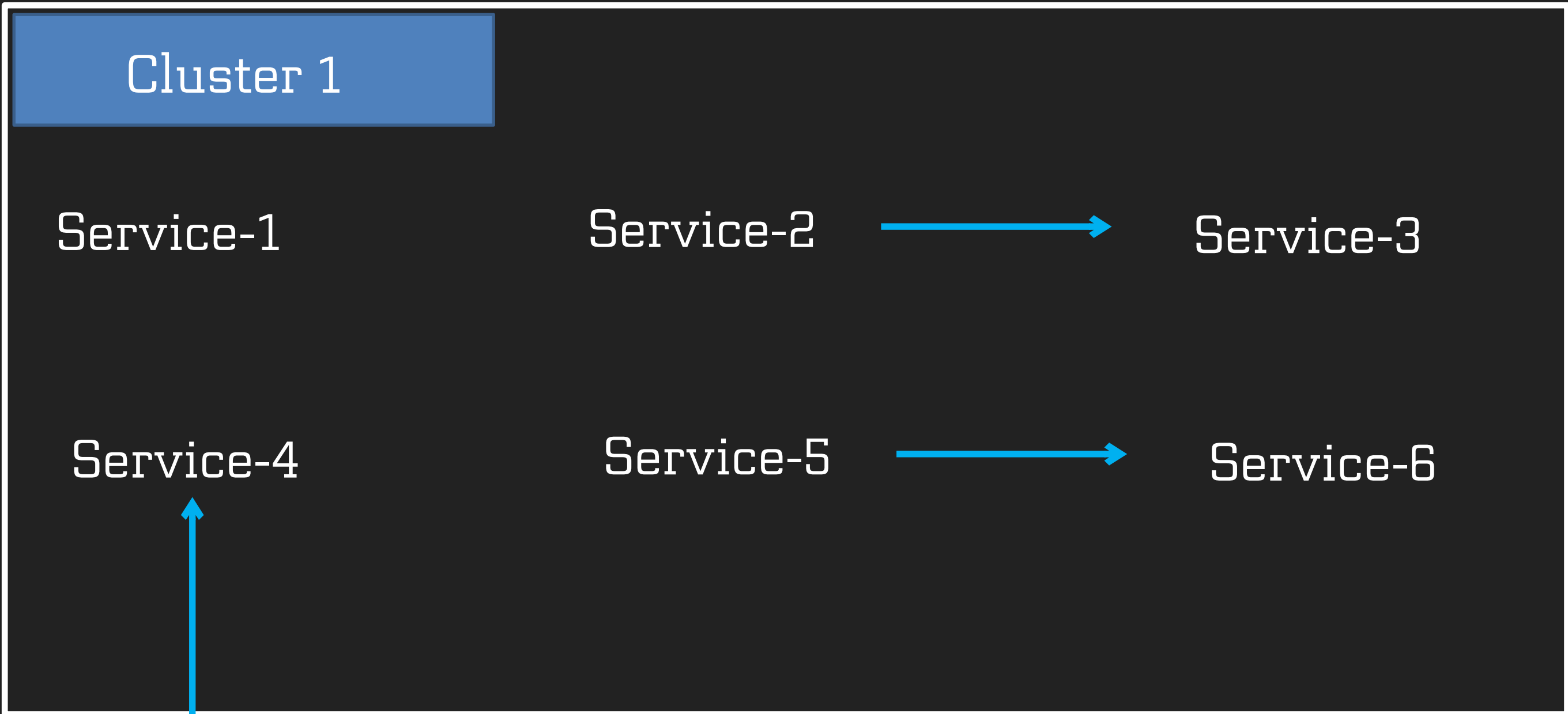
Secure
Communication

Service Definitions, Ports,
Load Balancing Algo



Fail-over cluster
(Clone Cluster 1)

Horizontal
Scaling within
cluster



East-West
Communication

Container (Pod),
Deployment Strategy, Replicas

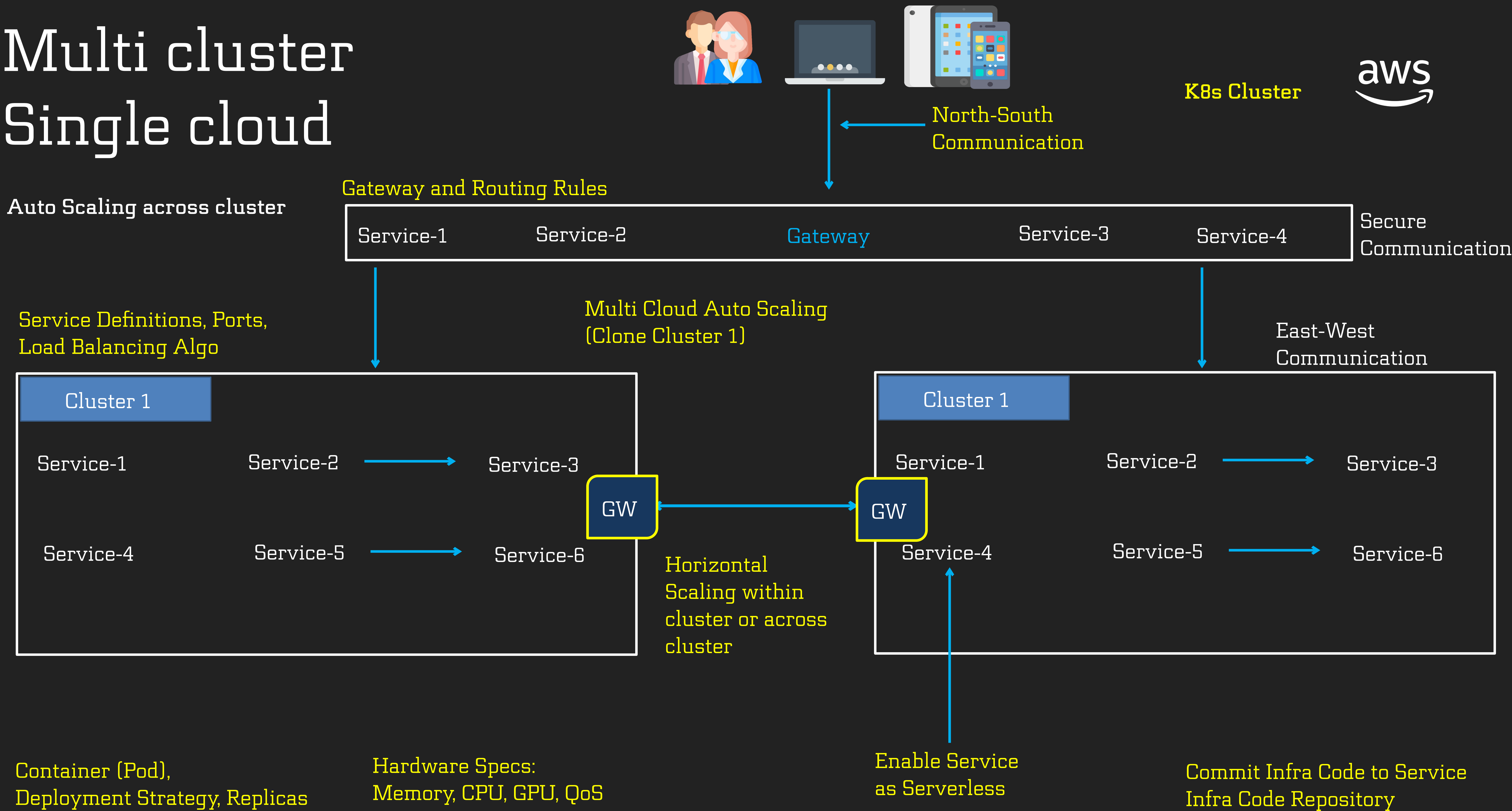
Hardware Specs:
Memory, CPU, GPU, QoS

Enable Service
as Serverless

Commit Infra Code to Service
Infra Code Repository

Multi cluster

Single cloud



Multi cluster

Multi cloud

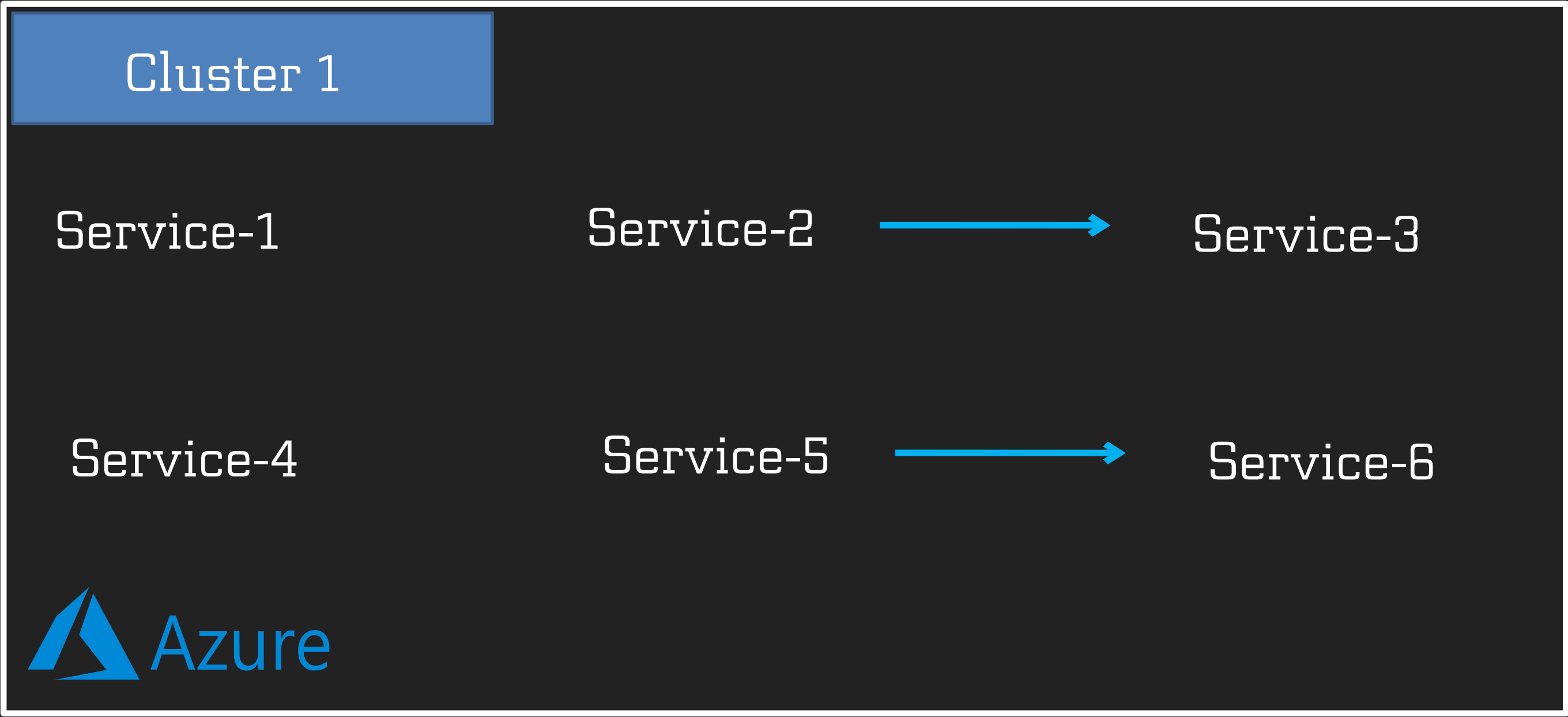
Fail over cluster

Gateway and Routing Rules



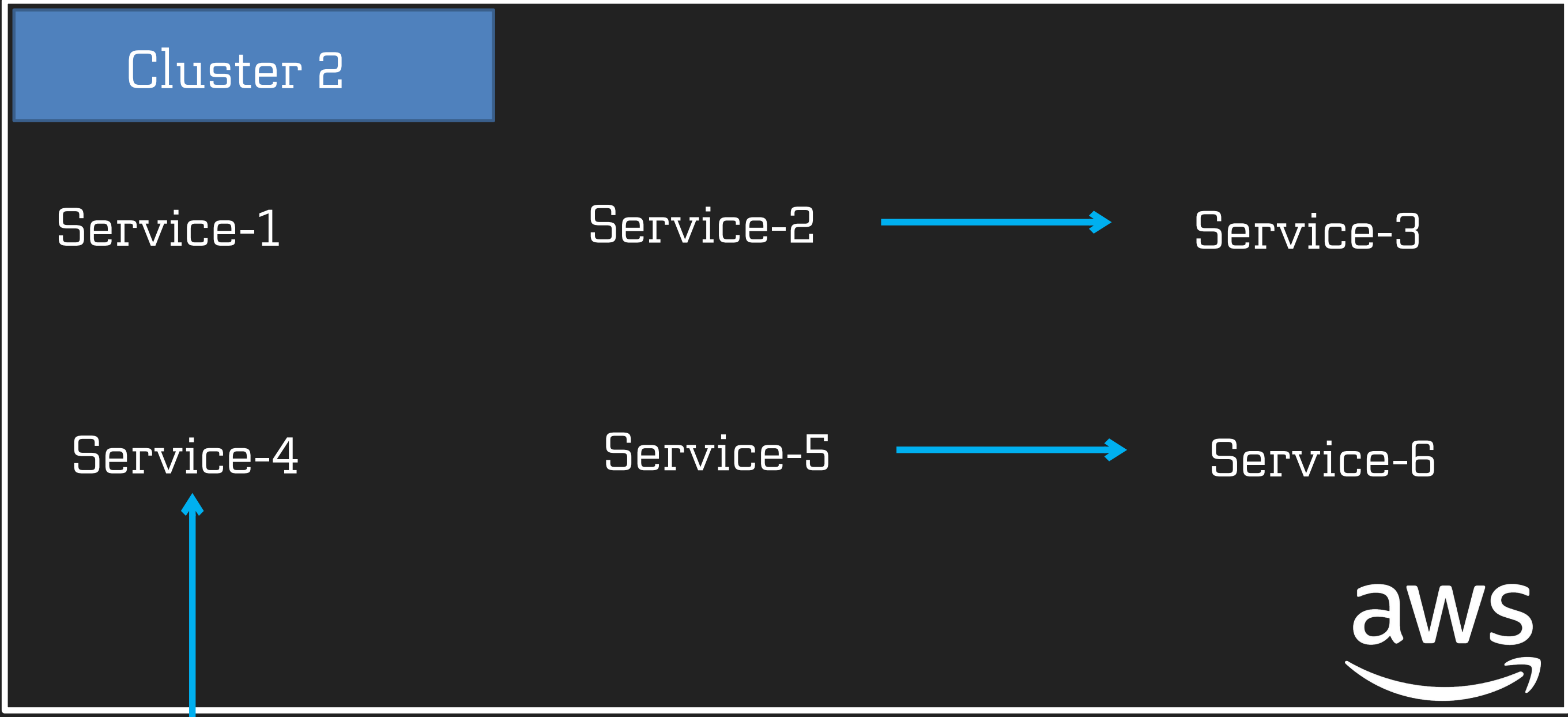
Secure Communication

Service Definitions, Ports, Load Balancing Algo



Fail-over cluster (Clone Cluster 1)

Horizontal Scaling within cluster



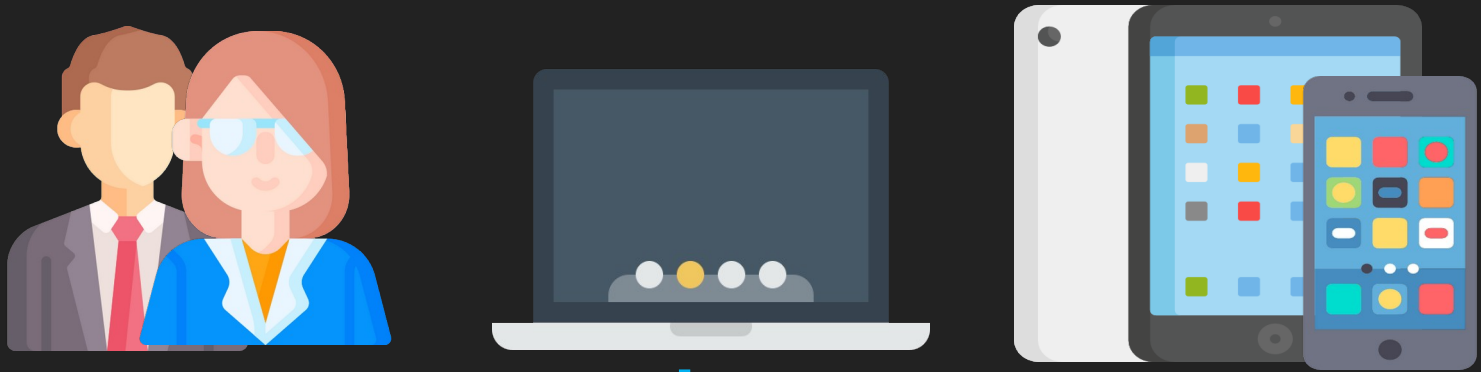
East-West Communication

Container (Pod), Deployment Strategy, Replicas

Hardware Specs: Memory, CPU, GPU, QoS

Enable Service as Serverless

Commit Infra Code to Service Infra Code Repository



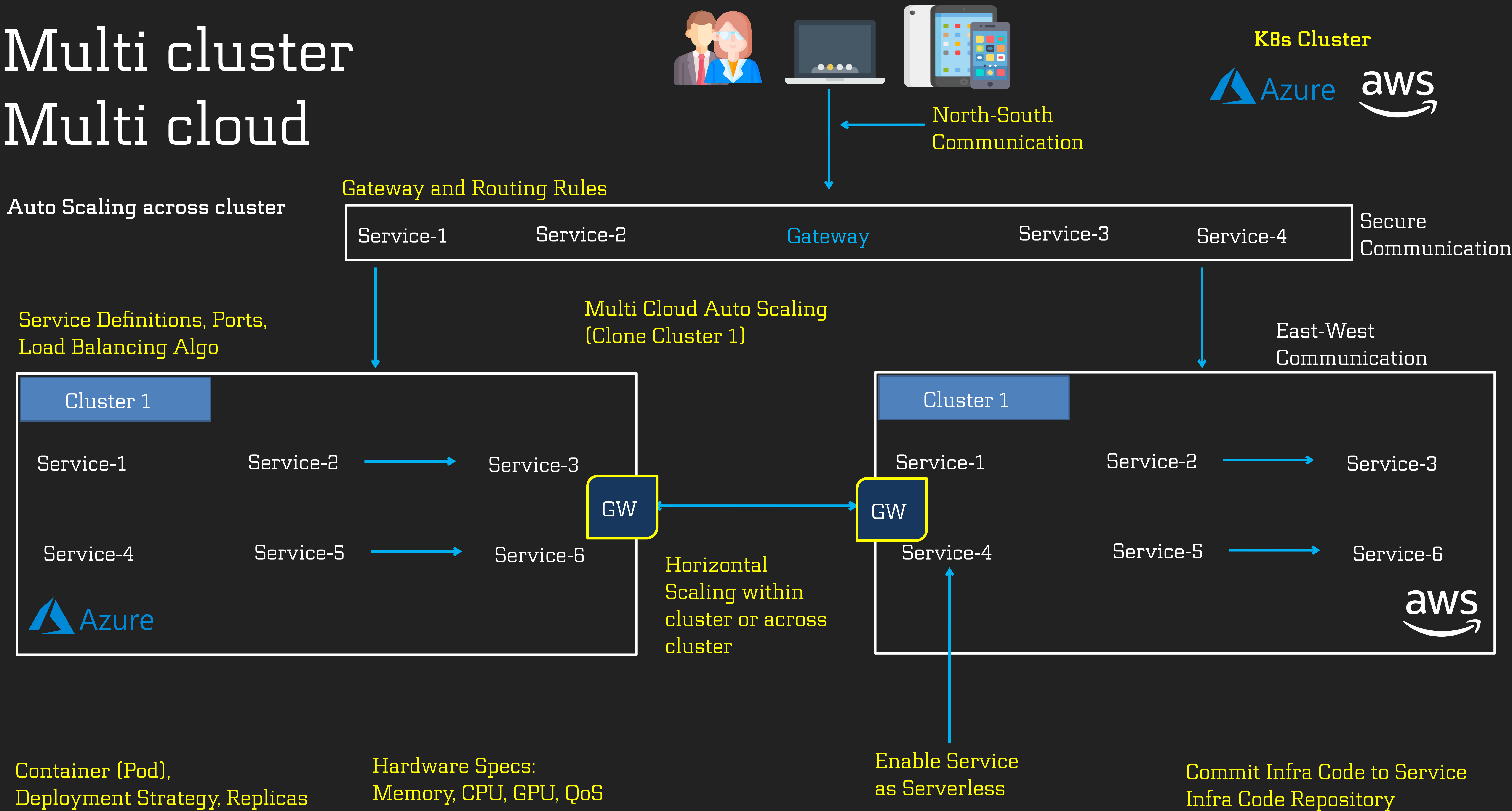
North-South Communication

K8s Cluster



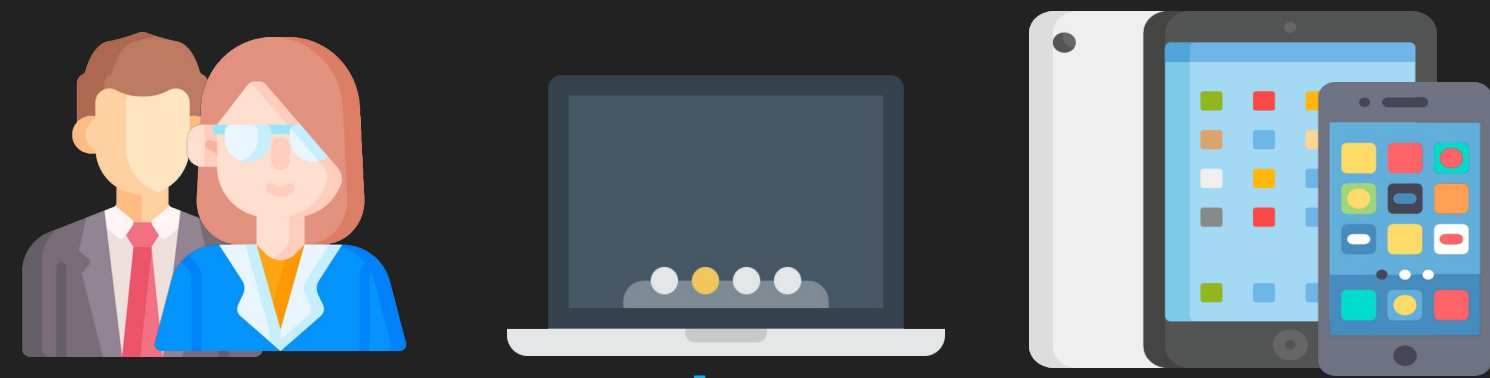
Multi cluster

Multi cloud



Multi cluster

Multi / Hybrid / Edge cloud



North-South
Communication



Auto Scaling across cluster

Gateway and Routing Rules



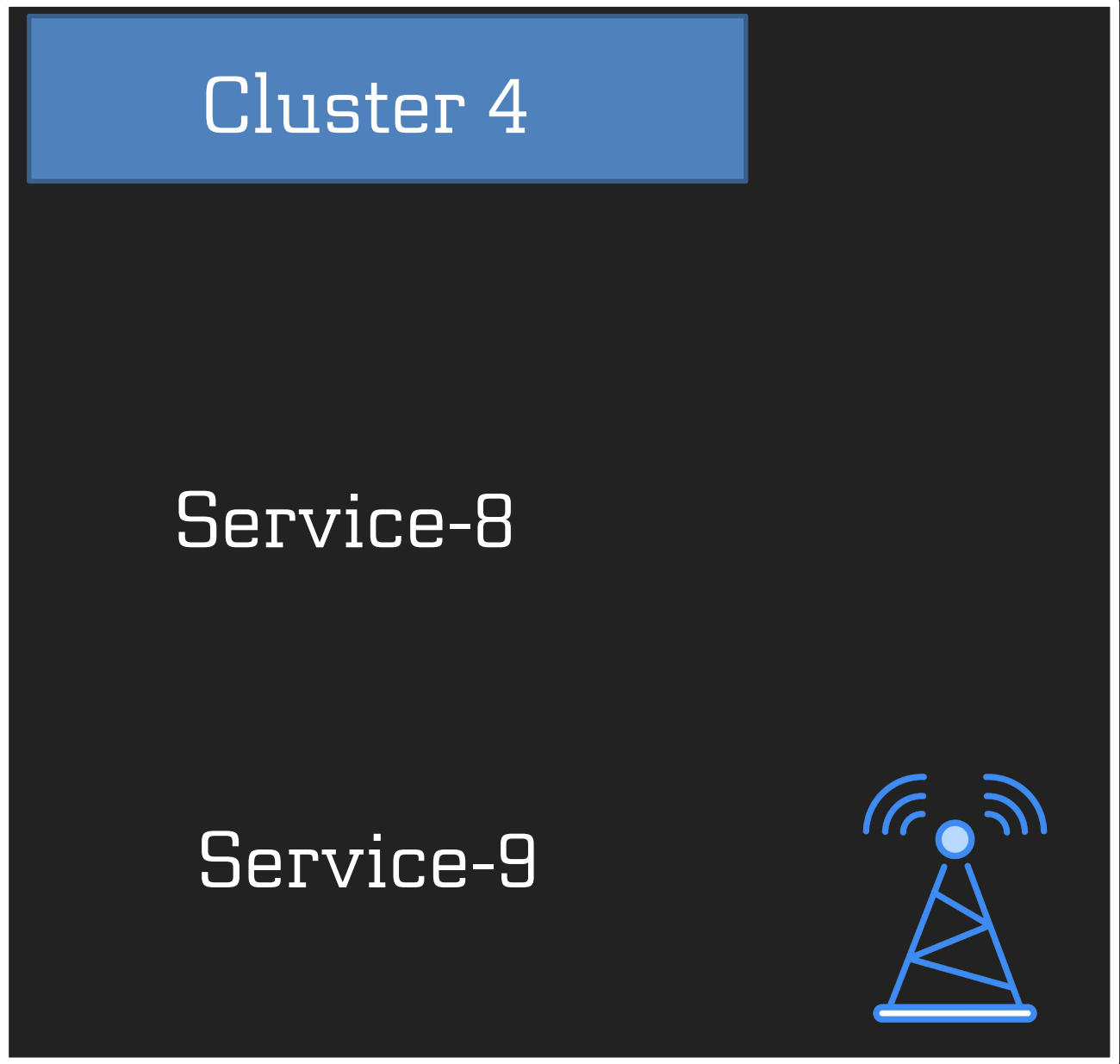
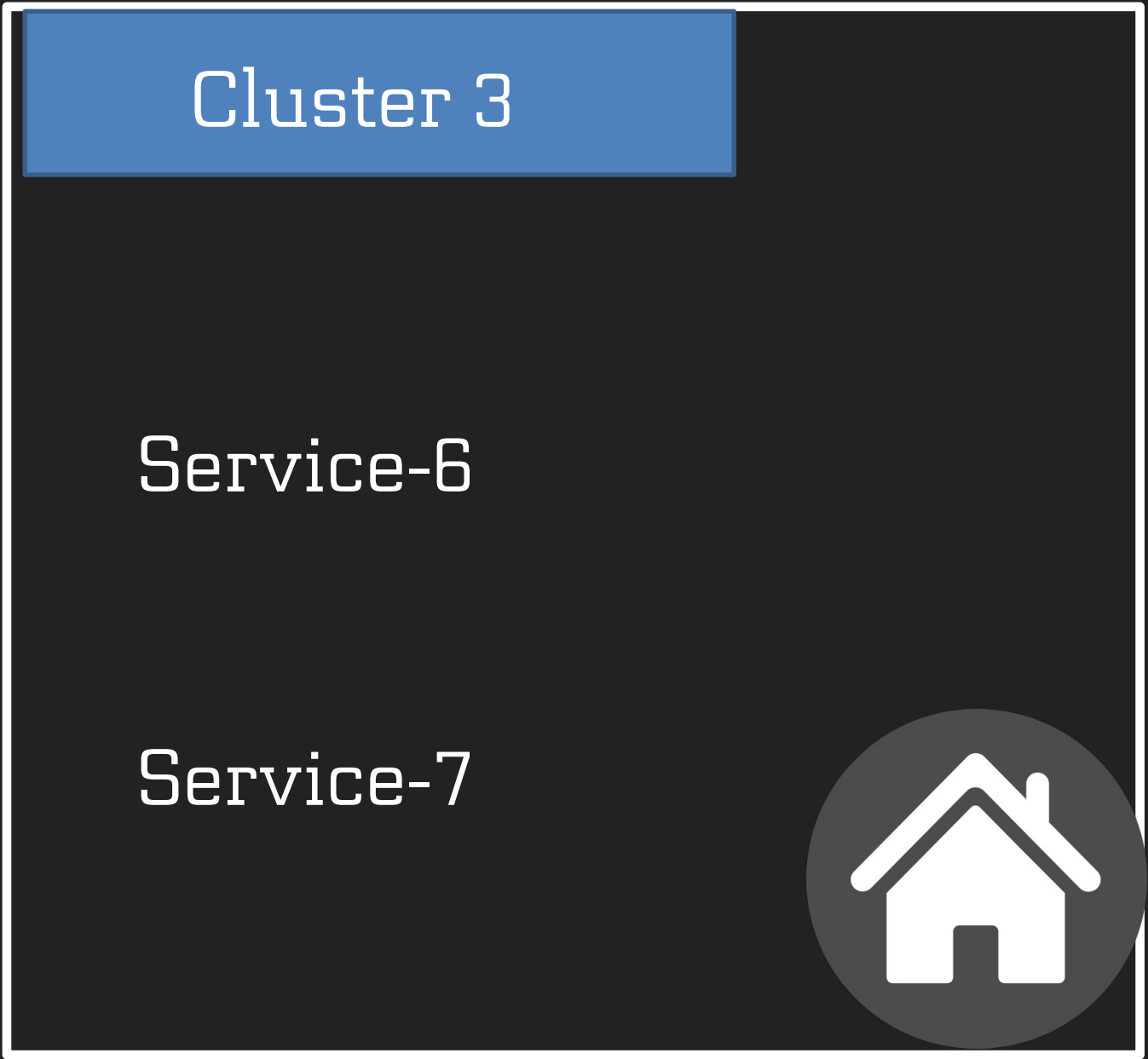
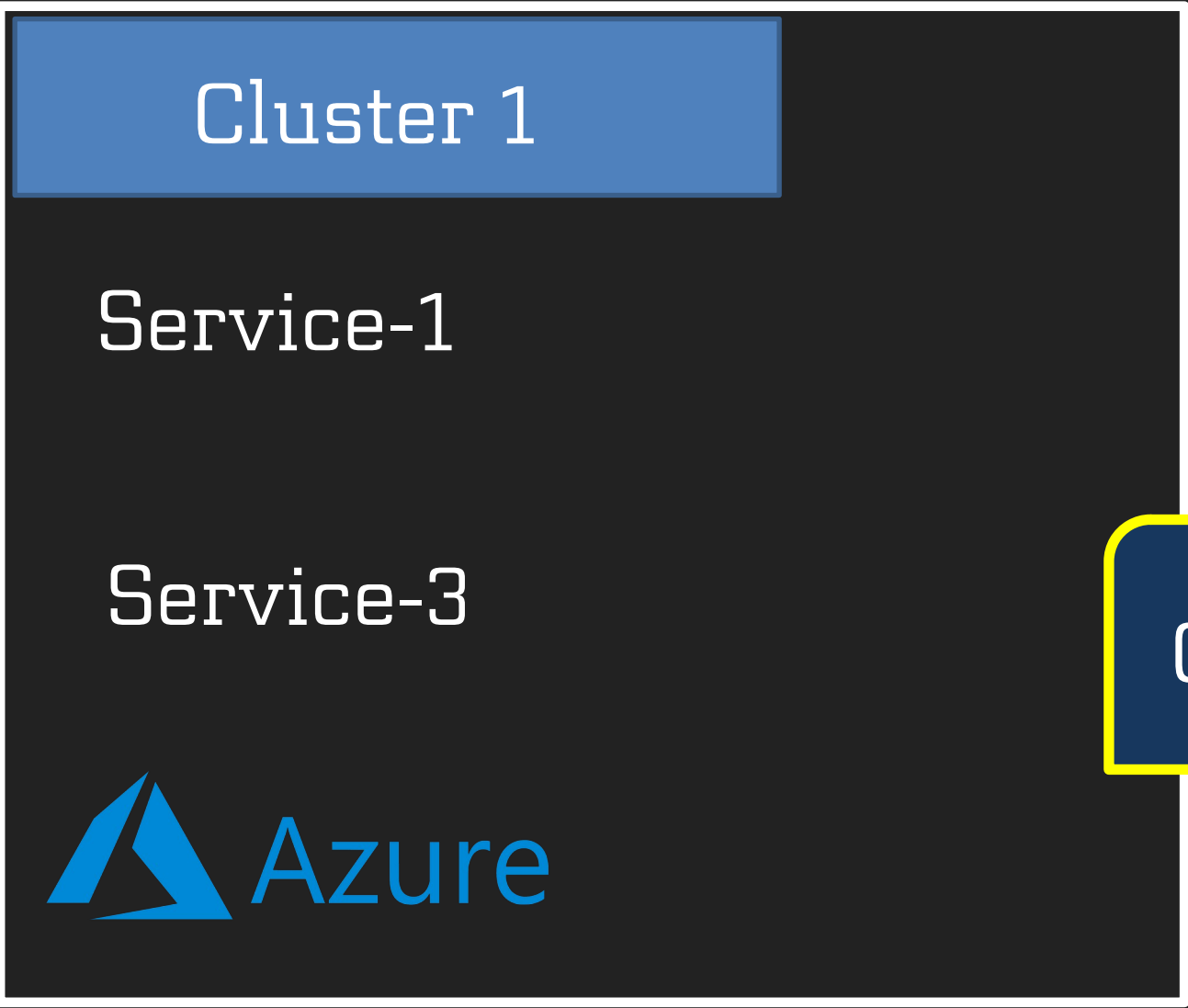
Secure
Communication

Service Definitions, Ports,
Load Balancing Algo

Horizontal
Scaling within
cluster or across
cluster

Multi Cloud Auto Scaling
(Clone Cluster 1)

East-West
Communication



On-Premise /
Private Cloud

Edge

Container (Pod),
Deployment Strategy,
Replicas

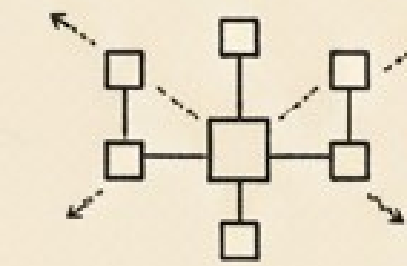
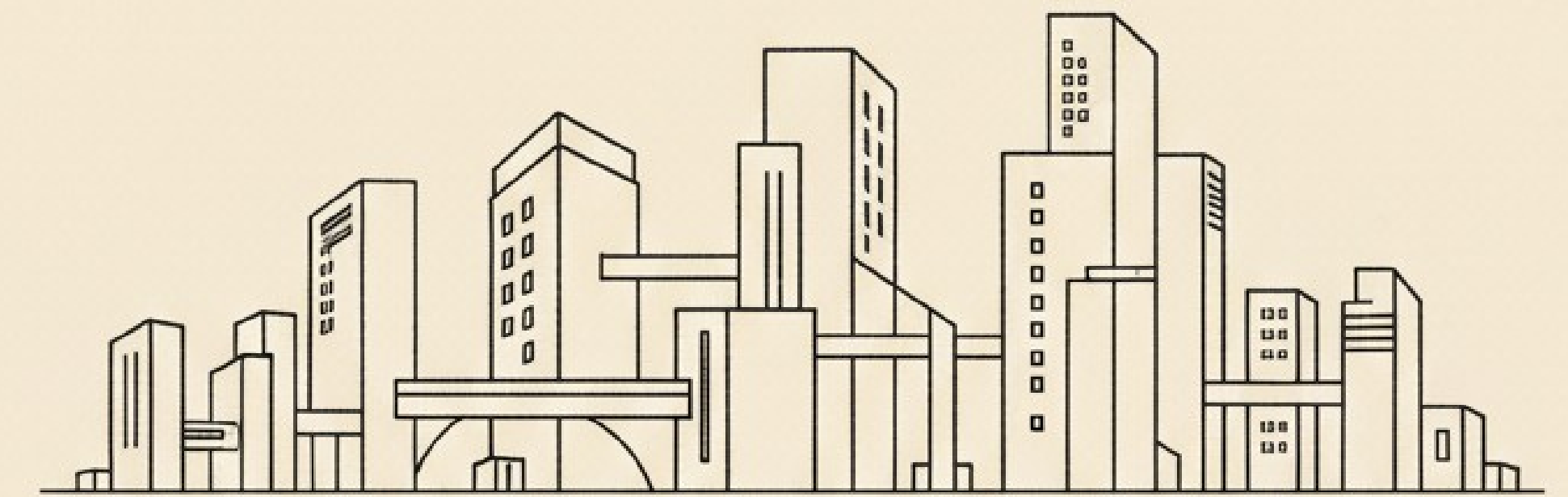
Hardware Specs:
Memory, CPU,
GPU, QoS

Enable Service
as Serverless

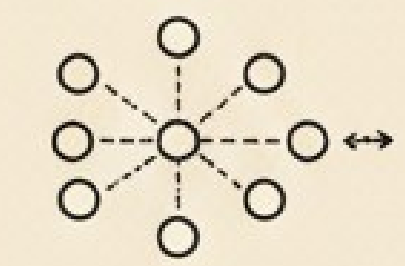
Commit Infra Code to Service Infra Code Repository

Modern System Architecture Concepts

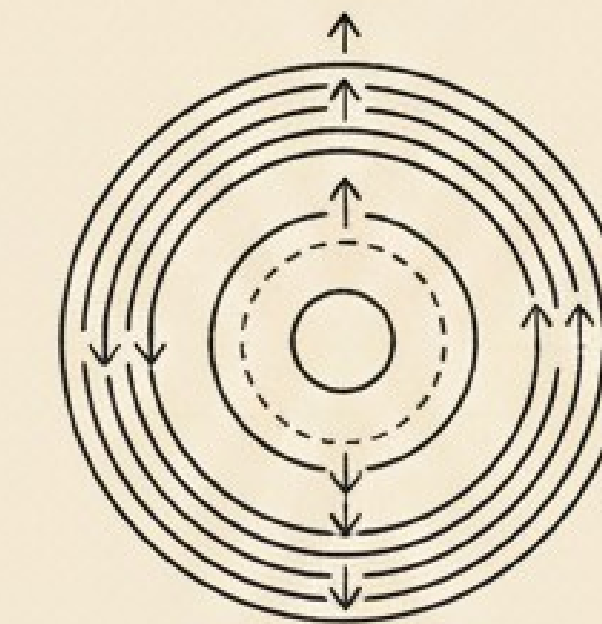
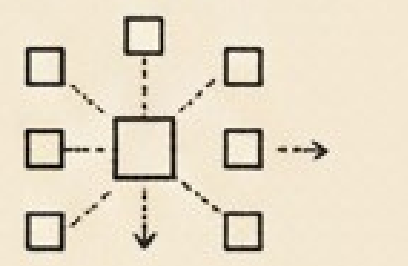
Modern Architecture Principles



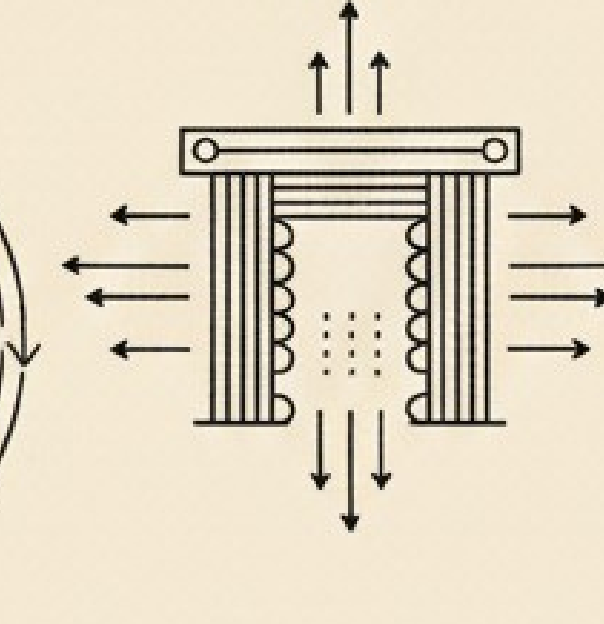
Microservices



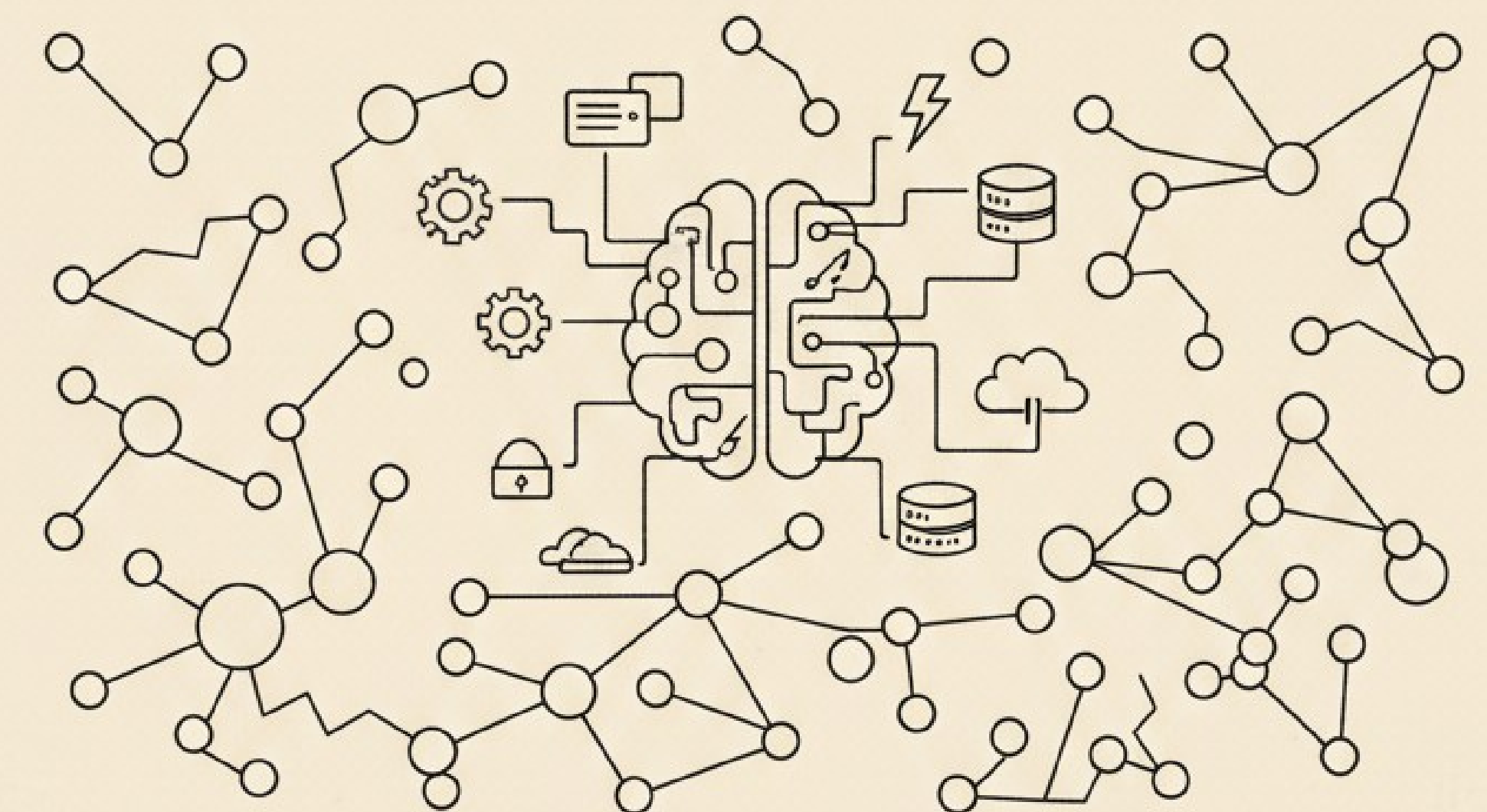
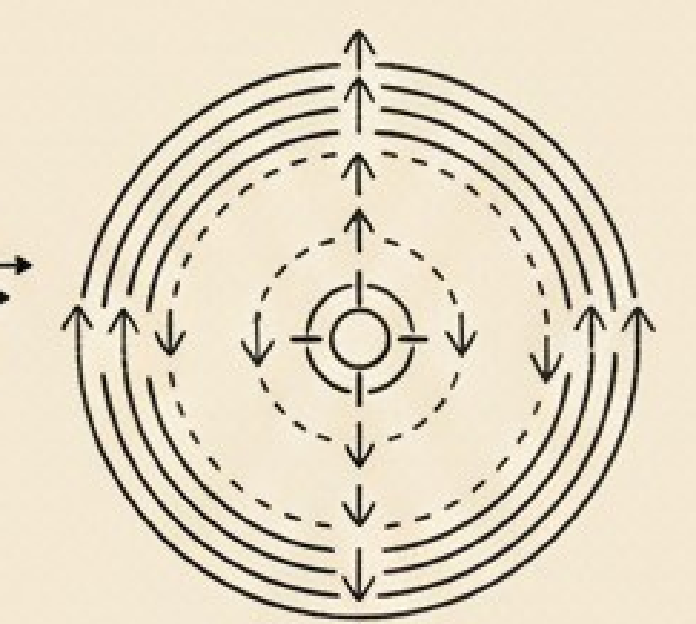
API Design



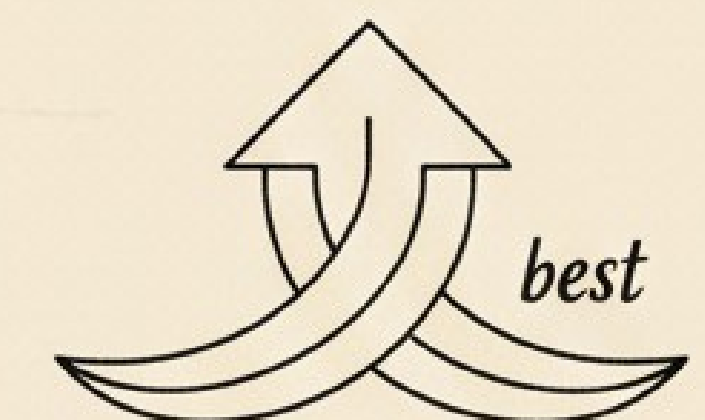
Scalability Patterns



System Design Components



Distributed Systems



Here are 12 essentials concepts

- ✓ Load Balancing
- ✓ Caching
- ✓ CDN (Content Delivery Network)
- ✓ Message Queues
- ✓ Publish/Subscribe (Pub/Sub)
- ✓ API Gateway
- ✓ Circuit Breaker
- ✓ Service Discovery
- ✓ Database Sharding
- ✓ Rate Limiting
- ✓ Consistent Hashing
- ✓ Auto Scaling

These concepts are the foundation of building systems that are:

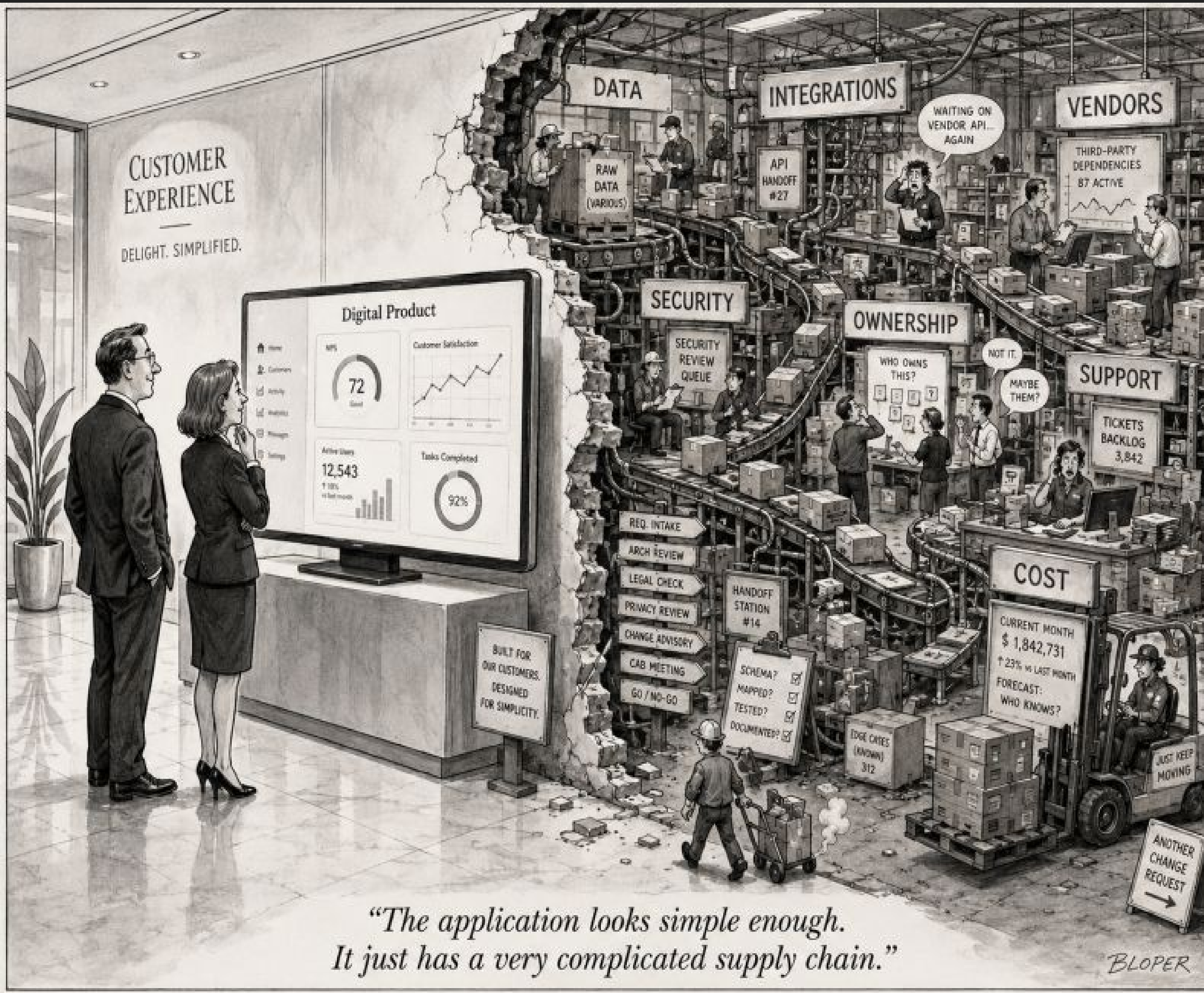
- ⚡ Scalable
- ⚡ Highly Available
- ⚡ Resilient
- ⚡ Secure
- ⚡ High Performance

Enterprise Architecture

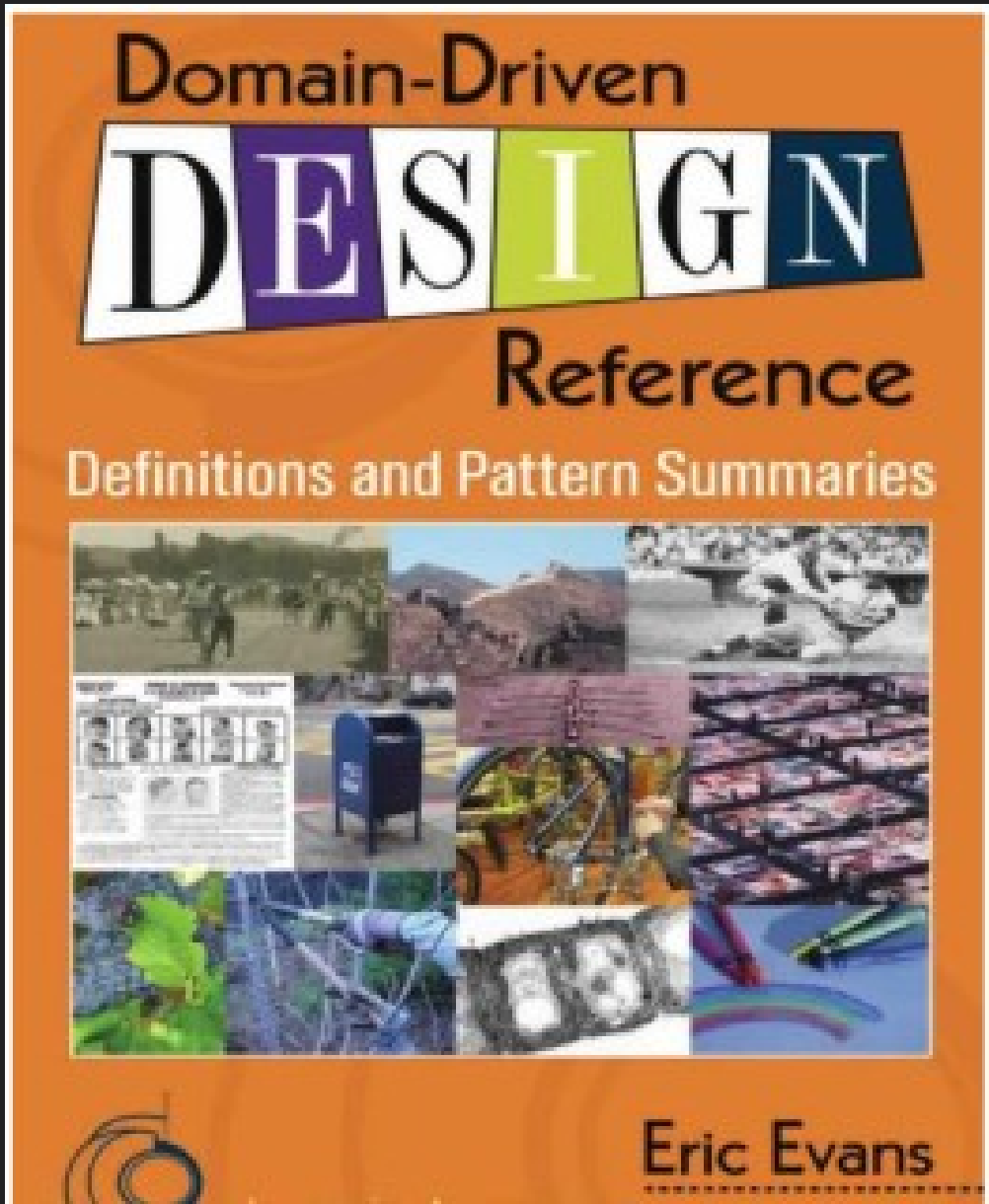
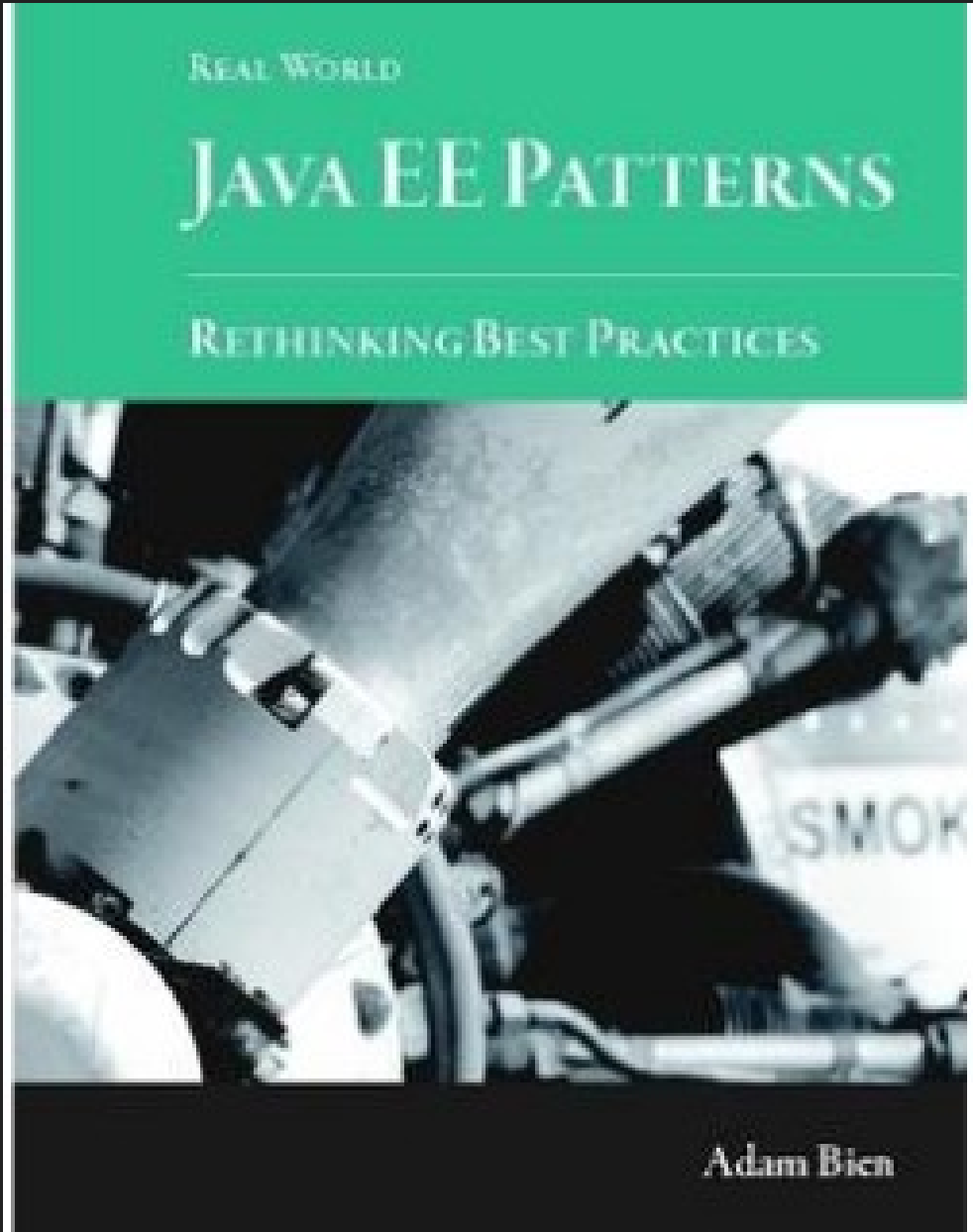
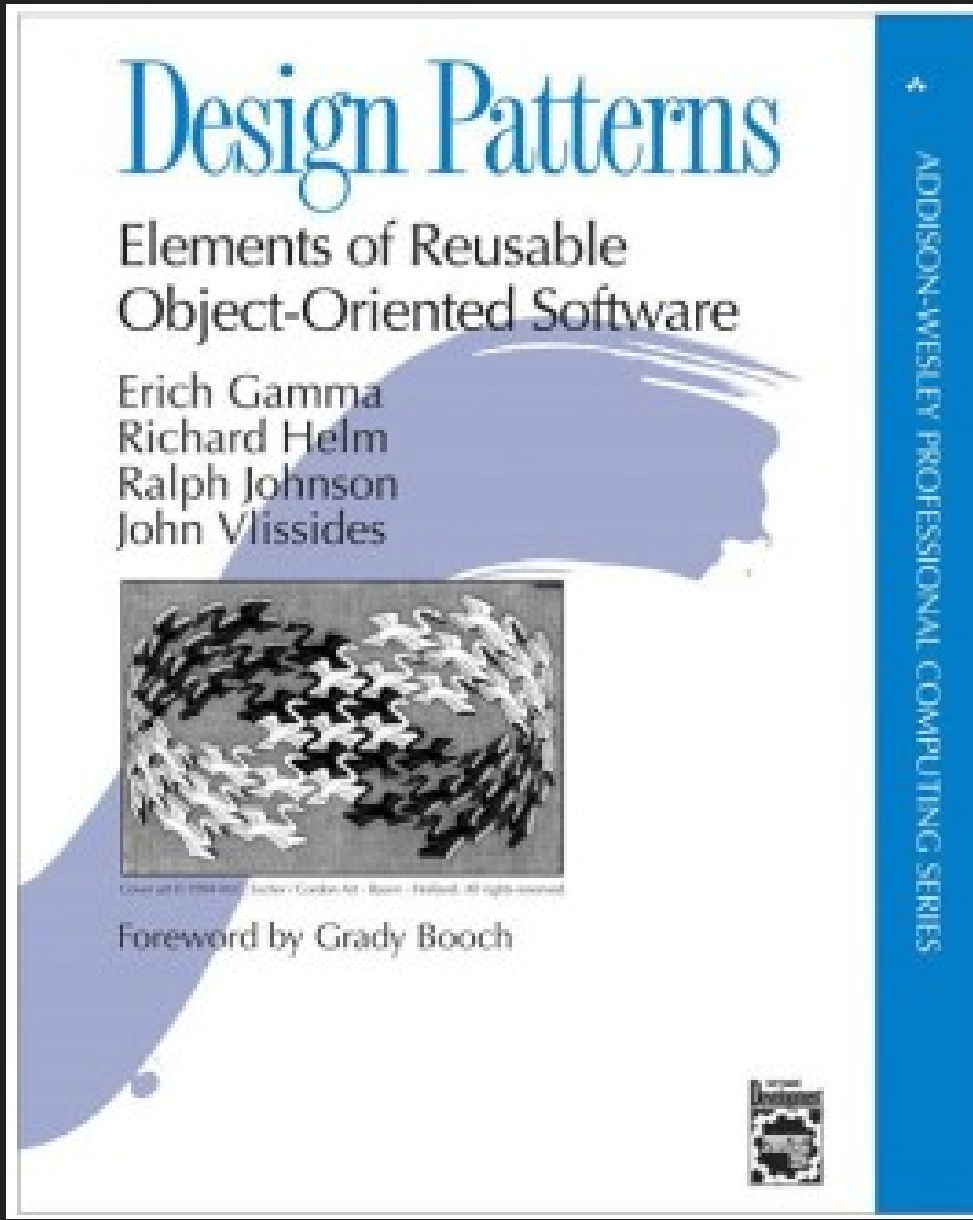
Enterprise Architecture is the supply chain of technology decisions.

*"The application looks simple enough.
It just has a very complicated supply chain."*

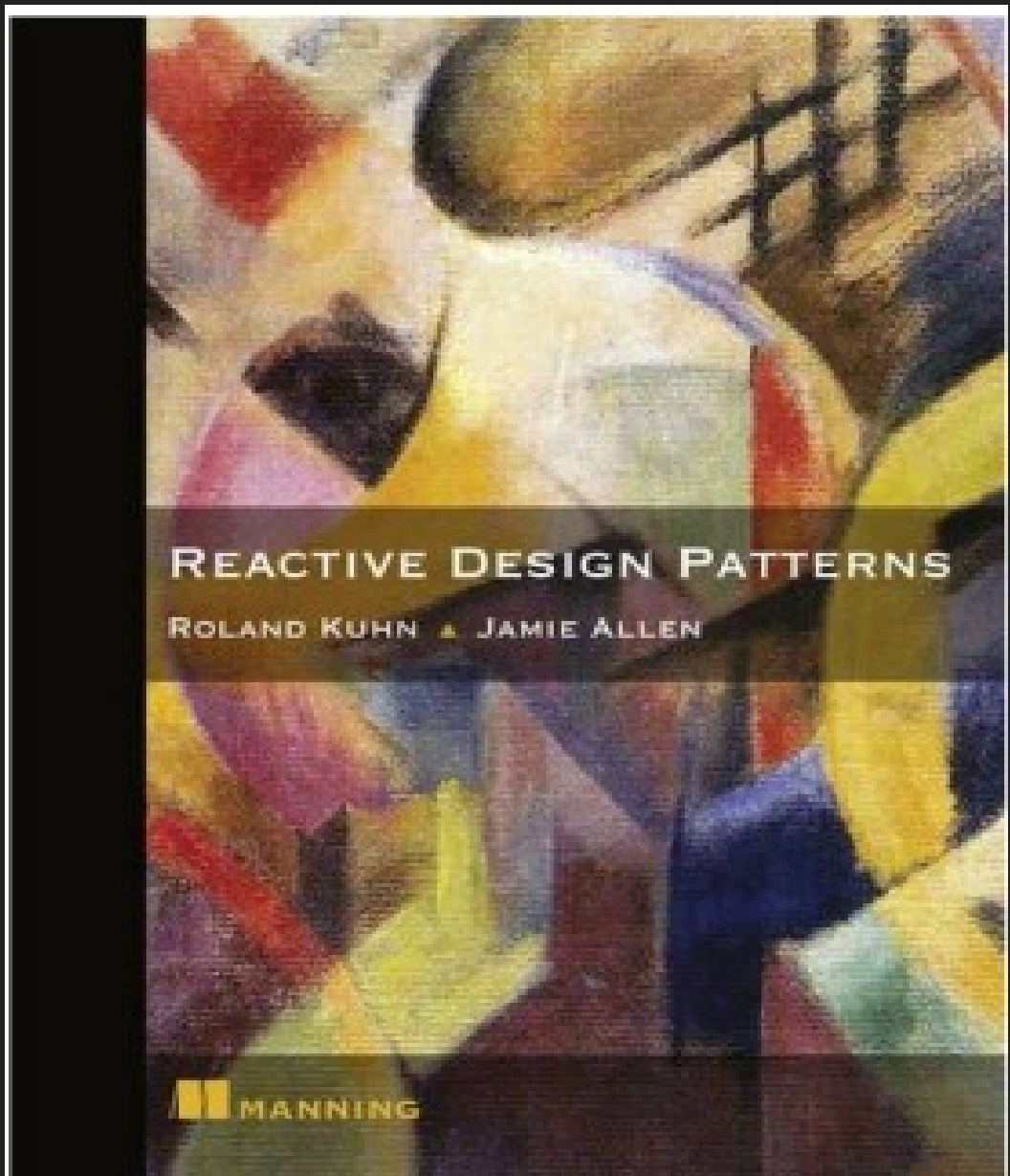
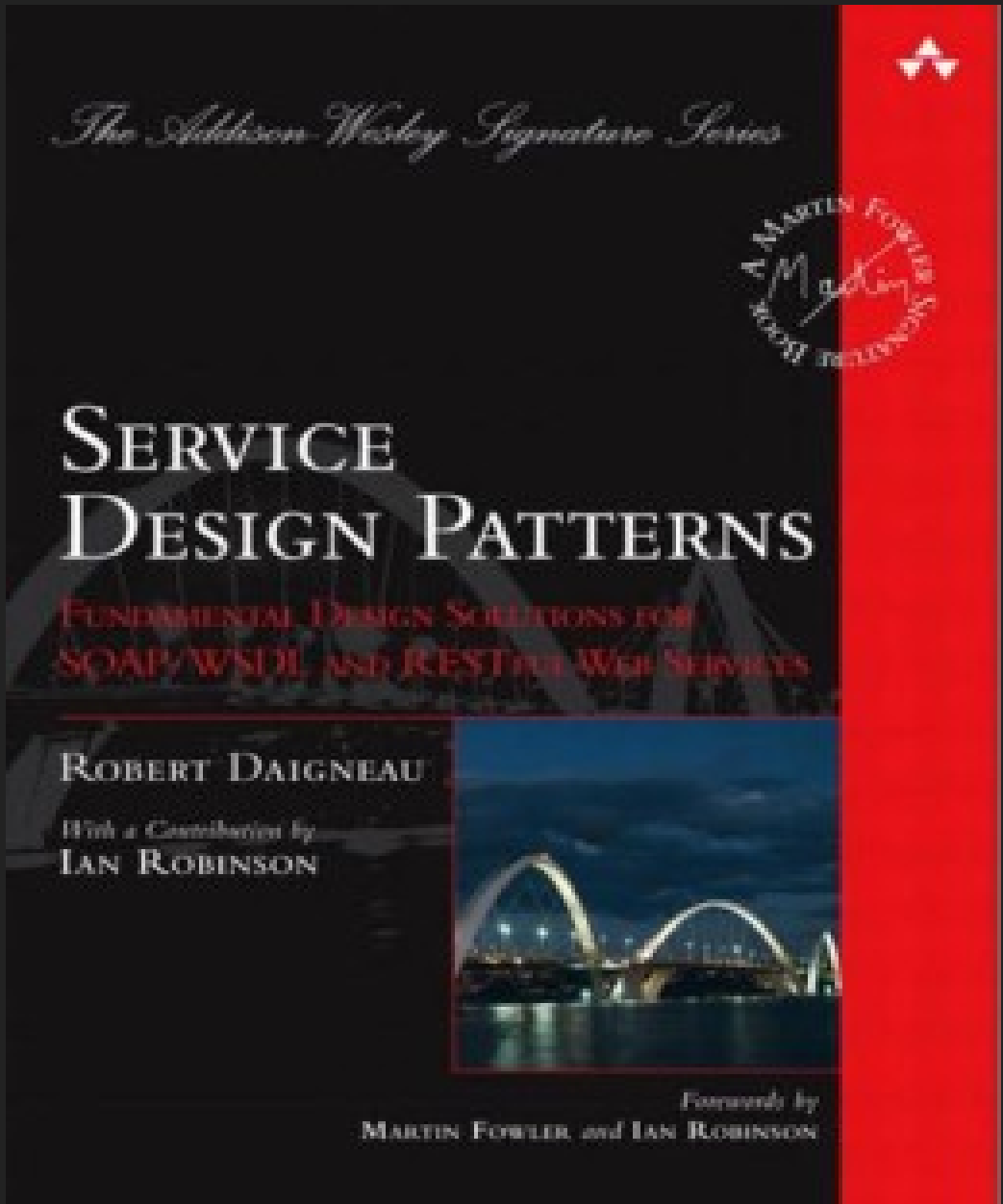
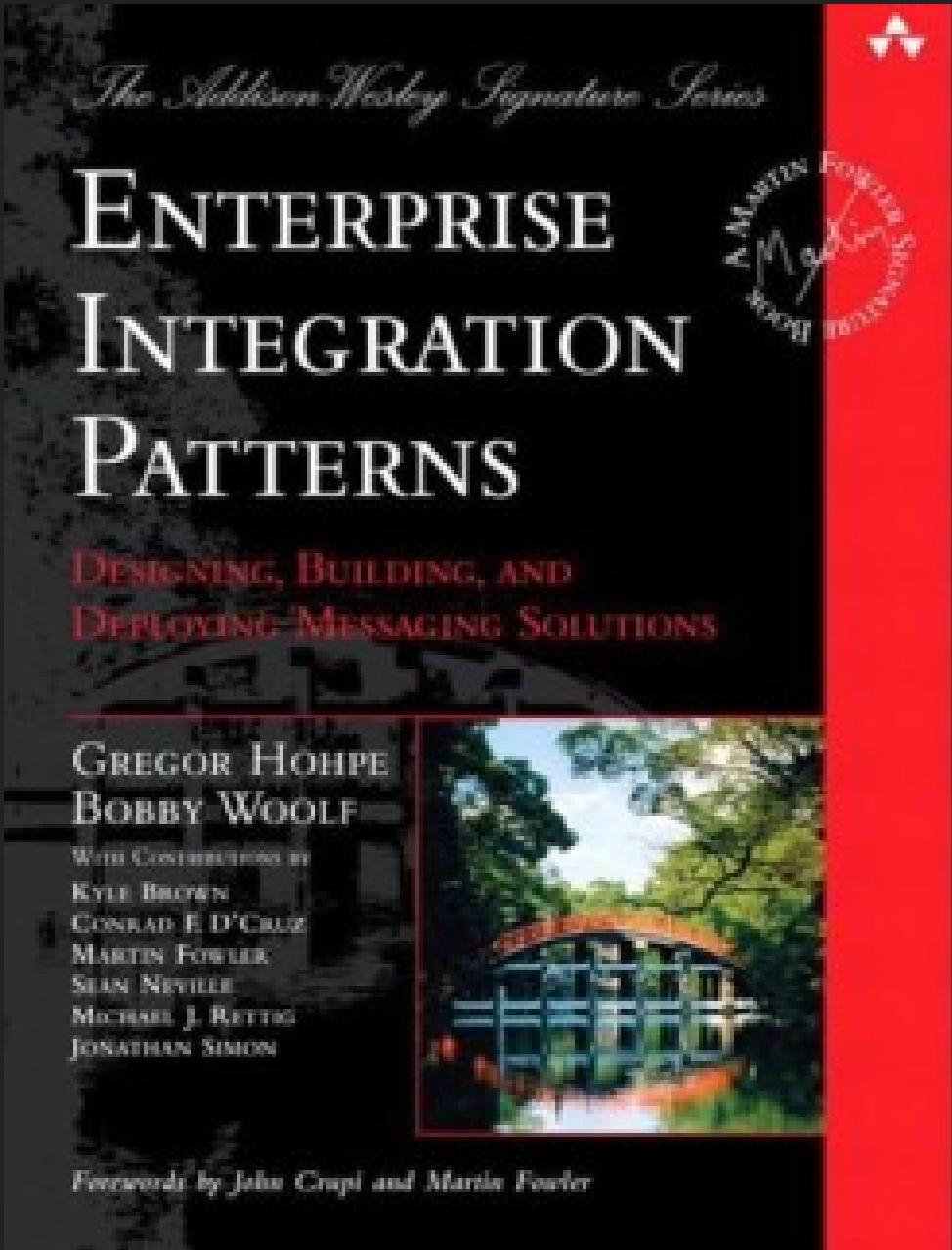
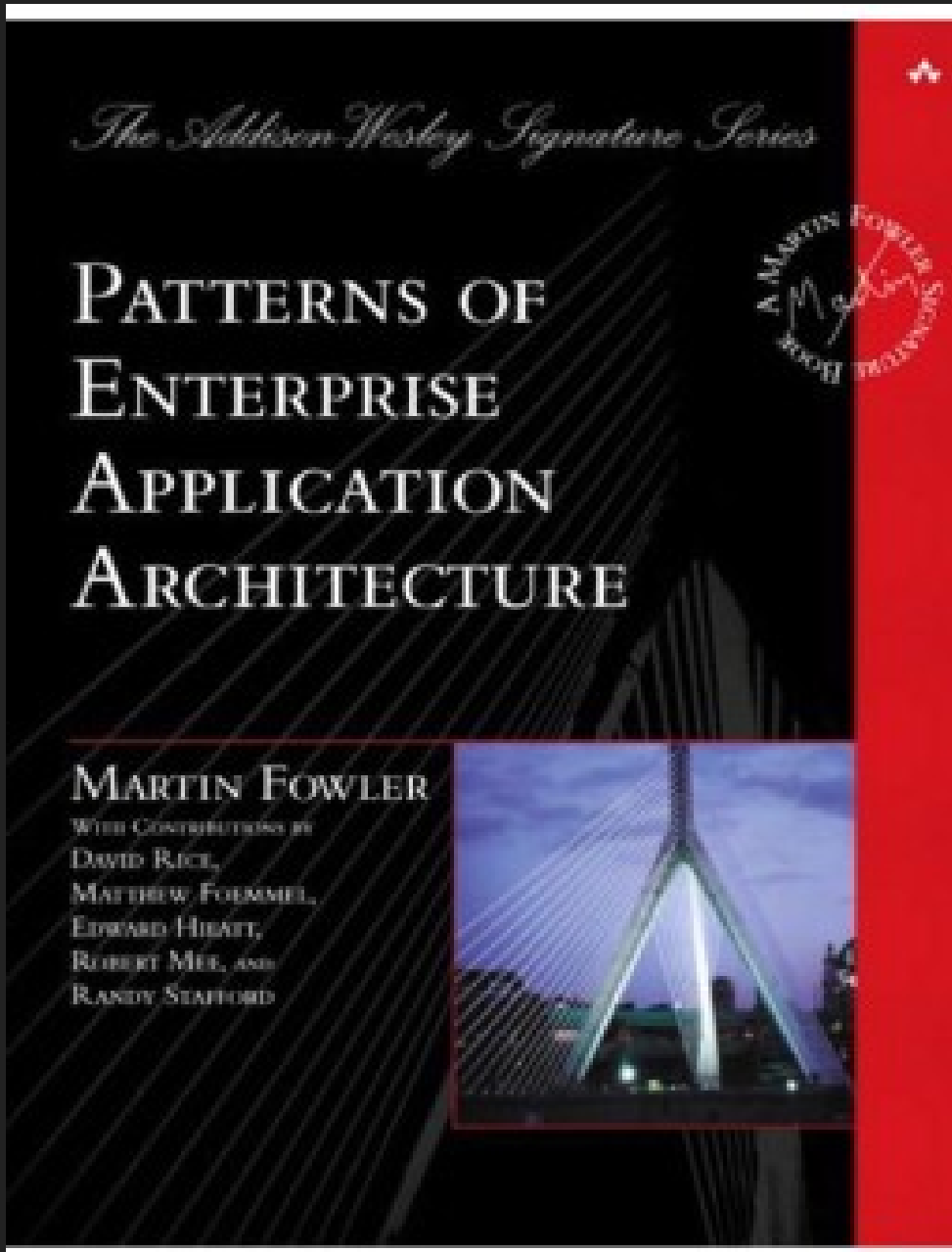
BLOPER



Design Patterns



Design Patterns are solutions to general problems that software developers faced during software development.



Your Cloud Journey Starts Now!

Cloud + Skills = Future.

Explore AWS Educated

No credit card required.



References

Agile Methodologies

1. July 15, 2015 - [Agile is Dead : GoTo 2015](#) By Dave Thomas
2. Apr 7, 2016 - [Agile Project Management with Kanban](#) | Eric Brechner | Talks at Google
3. Sep 27, 2017 - [Scrum vs Kanban - Two Agile Teams Go Head-to-Head](#)
4. Feb 17, 2019 - [Lean vs Agile vs Design Thinking](#)
5. Dec 17, 2020 - [Scrum vs Kanban | Differences & Similarities Between Scrum & Kanban](#)
6. Feb 24, 2021 - [Agile Methodology Tutorial for Beginners | Jira Tutorial | Agile Methodology Explained.](#)

References

Cloud Architecture

1. [Vmware: What is Cloud Architecture?](#)
2. [Redhat: What is Cloud Architecture?](#)
3. [Cloud Computing Architecture](#)
4. [Cloud Adoption Essentials:](#)

5. [Google: Hybrid and Multi Cloud](#) [IBM:](#)
6. [Hybrid Cloud Architecture Intro](#)
7. [IBM: Hybrid Cloud Architecture: Part 1](#)
8. [IBM: Hybrid Cloud Architecture: Part 2](#)
9. [Cloud Computing Basics: IaaS, PaaS, SaaS](#)

1. [IBM: IaaS Explained](#)
2. [IBM: PaaS Explained](#)
3. [IBM: SaaS Explained](#)
4. [IBM: FaaS Explained](#)
5. [IBM: What is Hypervisor?](#)

References

Microservices

[Microservices Definition by Martin Fowler](#)

[When to use Microservices By Martin Fowler](#)

GoTo: Sep 3, 2020: [When to use Microservices By Martin Fowler](#)

GoTo: Feb 26, 2020: [Monolith Decomposition Pattern](#)

Thought Works: [Microservices in a Nutshell](#)

[Microservices Prerequisites](#)

[What do you mean by Event Driven?](#)

[Understanding Event Driven Design Patterns for Microservices](#)

References – Microservices – Videos

1. Martin Fowler – Micro Services : <https://www.youtube.com/watch?v=2yko4TbC8cI&feature=youtu.be&t=15m53s>
2. GOTO 2016 – [Microservices at NetFlix Scale: Principles, Tradeoffs & Lessons Learned](#). By R Meshenberg
3. Mastering Chaos – [A NetFlix Guide to Microservices](#). By Josh Evans
4. GOTO 2015 – [Challenges Implementing Micro Services](#) By Fred George
5. GOTO 2016 – [From Monolith to Microservices at Zalando](#). By Rodrigue Scafer
6. GOTO 2015 – [Microservices @ Spotify](#). By Kevin Goldsmith
7. Modelling Microservices @ Spotify : <https://www.youtube.com/watch?v=7XDA044tl8k>
8. GOTO 2015 – [DDD & Microservices: At last, Some Boundaries](#) By Eric Evans
9. GOTO 2016 – [What I wish I had known before Scaling Uber to 1000 Services](#). By Matt Ranney
10. DDD Europe – [Tackling Complexity in the Heart of Software](#) By Eric Evans, April 11, 2016
- 11.
12. AWS re:Invent 2016 – [From Monolithic to Microservices: Evolving Architecture Patterns](#). By Emerson L, Gilt D. Chiles
13. AWS 2017 – [An overview of designing Microservices based Applications on AWS](#). By Peter Dalbhanjan
14. GOTO Jun, 2017 – [Effective Microservices in a Data Centric World](#). By Randy Shoup.
15. GOTO July, 2017 – [The Seven \(more\) Deadly Sins of Microservices](#). By Daniel Bryant
16. Sept, 2017 – [Airbnb, From Monolith to Microservices: How to scale your Architecture](#). By Melanie Cubula
17. GOTO Sept, 2017 – [Rethinking Microservices with Stateful Streams](#). By Ben Stopford.
GOTO 2017 – [Microservices without Servers](#). By Glynn Bird.



References

Domain Driven Design

1. Oct 27, 2012 [What I have learned about DDD Since the book](#). By Eric Evans
2. Mar 19, 2013 [Domain Driven Design](#) By Eric Evans
3. Jun 02, 2015 [Applied DDD in Java EE 7 and Open Source World](#)
4. Aug 23, 2016 [Domain Driven Design the Good Parts](#) By Jimmy Bogard
5. Sep 22, 2016 GOTO 2015 – [DDD & REST Domain Driven API's for the Web](#). By Oliver Gierke
6. Jan 24, 2017 Spring Developer – [Developing Micro Services with Aggregates](#). By Chris Richardson
7. May 17. 2017 DEVONX – [The Art of Discovering Bounded Contexts](#). By Nick Tune
8. Dec 21, 2019 [What is DDD - Eric Evans - DDD Europe 2019](#). By Eric Evans
9. Oct 2, 2020 - [Bounded Contexts - Eric Evans - DDD Europe 2020](#). By. Eric Evans
10. Oct 2, 2020 - [DDD By Example - Paul Rayner - DDD Europe 2020](#). By Paul Rayner

References

Event Sourcing and CQRS

1. IBM: [Event Driven Architecture](#) – Mar 21, 2021
2. Martin Fowler: [Event Driven Architecture](#) – GOTO 2017
3. Greg Young: [A Decade of DDD, Event Sourcing & CQRS](#) – April 11, 2016
4. Nov 13, 2014 GOTO 2014 – [Event Sourcing](#). By Greg Young
5. Mar 22, 2016 [Building Micro Services with Event Sourcing and CQRS](#)
6. Apr 15, 2016 YOW! Nights – [Event Sourcing](#). By Martin Fowler
7. May 08, 2017 [When Micro Services Meet Event Sourcing](#). By Vinicius Gomes

References

Kafka

1. [Understanding Kafka](#)
2. [Understanding RabbitMQ](#)
3. IBM: [Apache Kafka](#) – Sept 18, 2020
4. Confluent: [Apache Kafka Fundamentals](#) – April 25, 2020
5. Confluent: [How Kafka Works](#) – Aug 25, 2020
6. Confluent: [How to integrate Kafka into your environment](#) – Aug 25, 2020
7. [Kafka Streams](#) – Sept 4, 2021
8. Kafka: [Processing Streaming Data with KSQL](#) – Jul 16, 2018
9. Kafka: [Processing Streaming Data with KSQL](#) – Nov 28, 2019

References

Databases: Big Data / Cloud Databases

1. [Google: How to Choose the right database?](#)
 2. [AWS: Choosing the right Database](#)
 3. [IBM: NoSQL Vs. SQL](#)
 4. [A Guide to NoSQL Databases](#)
 5. [How does NoSQL Databases Work?](#)
 6. [What is Better? SQL or NoSQL?](#)
 7. [What is DBaaS?](#)
 8. [NoSQL Concepts](#)
 9. [Key Value Databases](#)
 10. [Document Databases](#)
 11. [Jun 29, 2012 – Google I/O 2012 - SQL vs NoSQL: Battle of the Backends](#)
 12. [Feb 19, 2013 - Introduction to NoSQL • Martin Fowler • GOTO 2012](#)
 13. [Jul 25, 2018 - SQL vs NoSQL or MySQL vs MongoDB](#)
 14. [Oct 30, 2020 - Column vs Row Oriented Databases Explained](#)
 15. [Dec 9, 2020 - How do NoSQL databases work? Simply Explained!](#)
1. [Graph Databases](#)
 2. [Column Databases](#)
 3. [Row Vs. Column Oriented Databases](#)
 4. [Database Indexing Explained](#)
 5. [MongoDB Indexing](#)
 6. [AWS: DynamoDB Global Indexing](#)
 7. [AWS: DynamoDB Local Indexing](#)
 8. [Google Cloud Spanner](#)
 9. [AWS: DynamoDB Design Patterns](#)
 10. [Cloud Provider Database Comparisons](#)
 11. [CockroachDB: When to use a Cloud DB?](#)

References

Docker / Kubernetes / Istio

[IBM: Virtual Machines and Containers](#)

[IBM: What is a Hypervisor?](#)

[IBM: Docker Vs. Kubernetes](#)

[IBM: Containerization Explained](#)

[IBM: Kubernetes Explained](#)

[IBM: Kubernetes Ingress in 5 Minutes](#)

[Microsoft: How Service Mesh works in Kubernetes](#)

[IBM: Istio Service Mesh Explained](#)

[IBM: Kubernetes and OpenShift](#)

[IBM: Kubernetes Operators](#)

[Consideration for Kubernetes Deployments](#)

[Istio – Metrics](#)

[Monitoring Istio Mesh with Grafana](#)

[Visualize your Istio Service Mesh](#)

[Security and Monitoring with Istio](#)

[Observing Services using Prometheus, Grafana, Kiali](#)

[Istio Cookbook: Kiali Recipe](#)

[Kubernetes: Open Telemetry](#)

[Open Telemetry](#)

[How Prometheus works](#)

[IBM: Observability vs. Monitoring](#)

References

Testing – TDD / BDD

1. Feb 6, 2020 – [An introduction to TDD](#)
2. Aug 14, 2019 – [Component Software Testing](#)
3. May 30, 2020 – [What is Component Testing?](#)
4. Apr 23, 2013 – [Component Test By Martin Fowler](#)
5. Jan 12, 2011 – [Contract Testing By Martin Fowler](#)
6. Jan 16, 2018 – [Integration Testing By Martin Fowler](#)
7. [Testing Strategies in Microservices Architecture](#)
8. [Practical Test Pyramid By Ham Vocke](#)

Testing tools

Simoorg : LinkedIn's own failure inducer framework. It was designed to be easy to extend and most of the important components are plug- gable.

Pumba : A chaos testing and network emulation tool for Docker.

Chaos Lemur : Self-hostable application to randomly destroy virtual machines in a BOSH-managed environment, as an aid to resilience testing of high-availability systems.

Chaos Lambda : Randomly terminate AWS ASG instances during business hours.

Blockade : Docker-based utility for testing network failures and partitions in distributed applications.

Chaos-http-proxy : Introduces failures into HTTP requests via a proxy server.

Monkey-ops : Monkey-Ops is a simple service implemented in Go, which is deployed into an OpenShift V3.X and generates some chaos within it. Monkey-Ops seeks some OpenShift components like Pods or Deployment Configs and randomly terminates them.

Chaos Dingo : Chaos Dingo currently supports performing operations on Azure VMs and VMSS deployed to an Azure Resource Manager-based resource group.

Tugbot : Testing in Production (TiP) framework for Docker.

References

1. [What is Continuous Integration?](#)
2. [What is Continuous Delivery?](#)
3. [CI / CD Pipeline](#)
4. [What is CI / CD Pipeline?](#)
5. [CI / CD Explained](#)
6. [CI / CD Pipeline using Java Example Part 1](#)
7. [CI / CD Pipeline using Ansible Part 2](#)
8. [Declarative Pipeline vs Scripted Pipeline](#)
9. [Complete Jenkins Pipeline Tutorial](#)
10. [Common Pipeline Mistakes](#)
11. [CI / CD for a Docker Application](#)

References

DevOps

[IBM: What is DevOps?](#)

[IBM: Cloud Native DevOps Explained](#)

[IBM: Application Transformation](#)

[IBM: Virtualization Explained](#)

[What is DevOps? Easy Way](#)

[DevOps?! How to become a DevOps Engineer???](#)

Amazon: <https://www.youtube.com/watch?v=mBU3AJ3j1rq>

NetFlix: <https://www.youtube.com/watch?v=UTKIT6STSVM>

DevOps and SRE: <https://www.youtube.com/watch?v=uTEL8Ff1Zvk>

SLI, SLO, SLA : <https://www.youtube.com/watch?v=tEylFyxbDLE>

DevOps and SRE : Risks and Budgets : <https://www.youtube.com/watch?v=y2ILKr8kCJU>

SRE @ Google: https://www.youtube.com/watch?v=d2wn_E1jxn4

References

1. Lewis, James, and Martin Fowler. "[Microservices: A Definition of This New Architectural Term](#)", March 25, 2014.
2. Miller, Matt. "[Innovate or Die: The Rise of Microservices](#)". *the Wall Street Journal*, October 5, 2015.
3. Newman, Sam. [*Building Microservices*](#). O'Reilly Media, 2015.
4. Alagarasan, Vijay. "[Seven Microservices Anti-patterns](#)", August 24, 2015.
5. Cockcroft, Adrian. "[State of the Art in Microservices](#)", December 4, 2014.
6. Fowler, Martin. "[Microservice Prerequisites](#)", August 28, 2014.
7. Fowler, Martin. "[Microservice Tradeoffs](#)", July 1, 2015.
8. Humble, Jez. "[Four Principles of Low-Risk Software Release](#)", February 16, 2012.
9. [Zuul Edge Server](#), Ketan Gote, May 22, 2017
10. [Ribbon, Hystrix using Spring Feign](#), Ketan Gote, May 22, 2017
11. [Eureka Server with Spring Cloud](#), Ketan Gote, May 22, 2017
12. [Apache Kafka, A Distributed Streaming Platform](#), Ketan Gote, May 20, 2017
13. [Functional Reactive Programming](#), Araf Karsh Hamid, August 7, 2016
14. [Enterprise Software Architectures](#), Araf Karsh Hamid, July 30, 2016

15. [Docker and Linux Containers](#), Araf Karsh Hamid, April 28, 2015

References

- 16.MSDN – Microsoft <https://msdn.microsoft.com/en-us/library/dn568103.aspx>
- 17.Martin Fowler : CQRS – <http://martinfowler.com/bliki/CQRS.html>
- 18.Udi Dahan : CQRS – <http://www.udidahan.com/2009/12/09/clarified-cqrs/>
- 19.Greg Young : CQRS - <https://www.youtube.com/watch?v=JHGkaShoyNs>
- 20.Bertrand Meyer – CQS - http://en.wikipedia.org/wiki/Bertrand_Meyer
- 21.CQS : http://en.wikipedia.org/wiki/Command-query_separation
- 22.CAP Theorem : http://en.wikipedia.org/wiki/CAP_theorem
- 23.CAP Theorem : <http://www.julianbrowne.com/article/viewer/brewers-cap-theorem>
- 24.CAP 12 years how the rules have changed
- 25.EBay Scalability Best Practices : <http://www.infoq.com/articles/ebay-scalability-best-practices>
- 26.Pat Helland (Amazon) : [Life beyond distributed transactions](#)
- 27.Stanford University: Rx <https://www.youtube.com/watch?v=y9xudo3C1Cw>
- 28.Princeton University: [SAGAS \(1987\) Hector Garcia Molina / Kenneth Salem](#)
29. Rx Observable : <https://dzone.com/articles/using-rx-java-observable>

it's a wrap

thank you for coming